香港中文大學統計學系
Department of Statistics
THE CHINESE UNIVERSITY OF HONG KONG

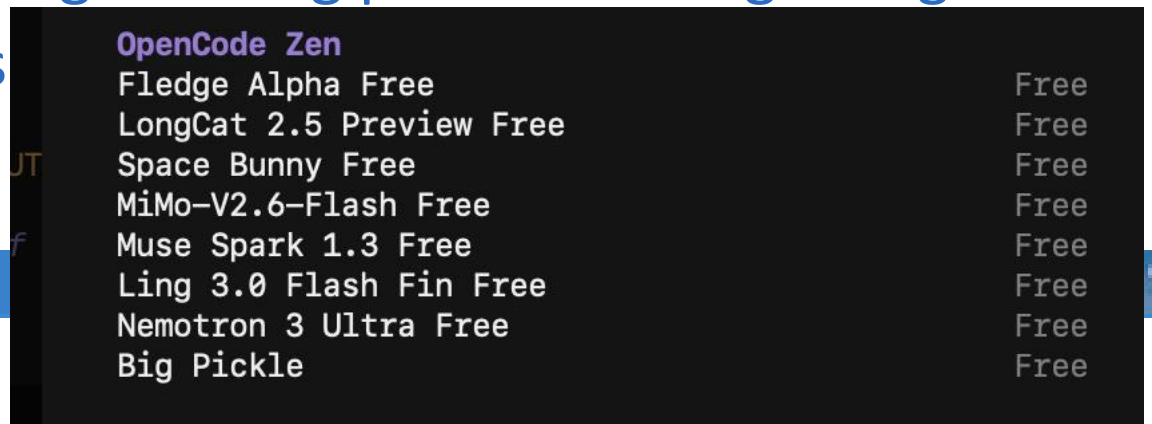
STAT6207
Applied Deep Learning

L4, 2 Oct 2026
By Dr. Ming Cheung



Announcement

- Midterm exam will be on 23 Oct 26 at room: CYT201, 202, 209B
- There will be two parts
  - Written: MC and short question
  - Programming: You can use your computer and agent coder
- No communication is allowed,
- You can bring a cheat sheet (A4, dounle page)
- Make sure that you can run all demo and assignments on your computer
- Duration 2.5 hours, you will need to complete the written part first
- You will know the questions in Programming part at the beginning
- You will need to use free models



Announcement – Final Project

- 4-5 persons in a group
- The topic will be announced after mid term



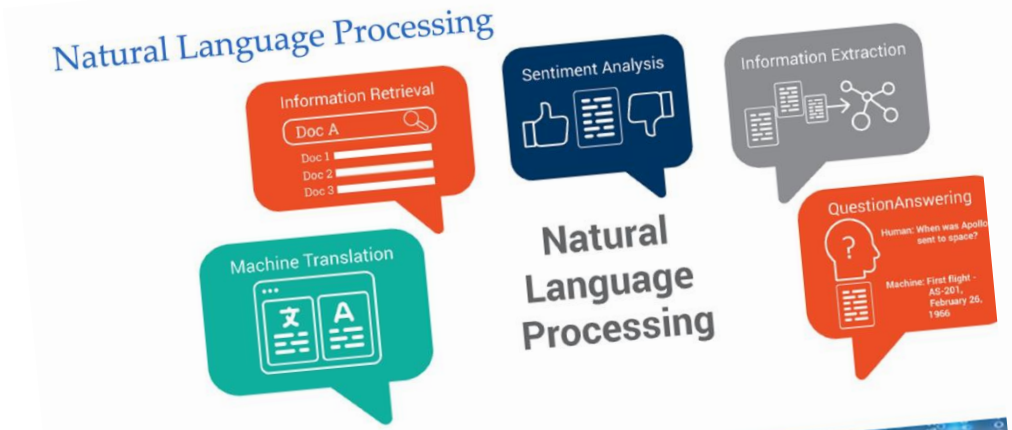
Lecture Room - next lecture: CYT LT1A

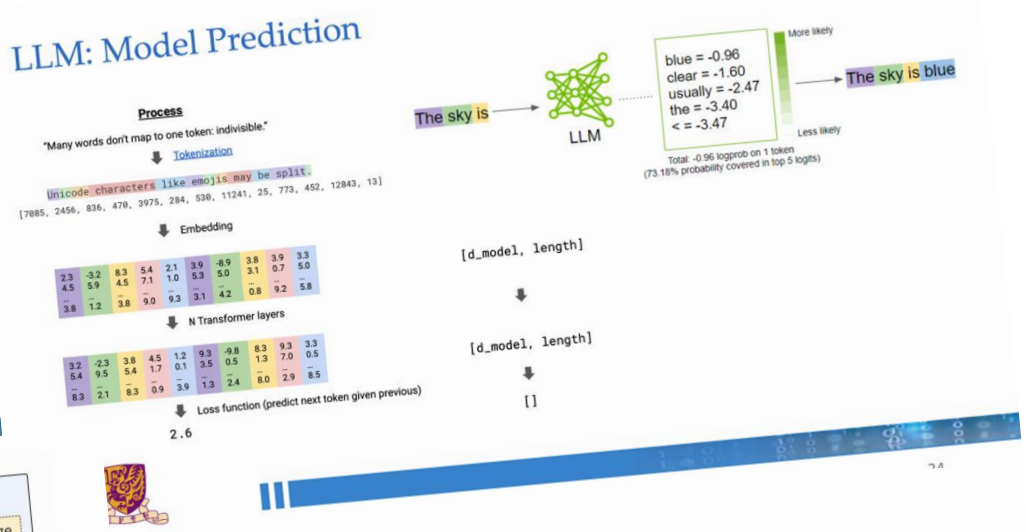
Lecture	Week	Lecture Rooms
L4	2-Oct	CYT LT1A
L5	9-Oct	CYT LT1A
L6	16-Oct	BMS G18
L7 (Mid Term)	23-Oct	CYT201, 202, 209B
L8	30-Oct	CYT LT1A
L9	6-Nov	UCC C1
L10	13-Nov	CYT LT1A
L11	20-Nov	UCC C1
L12	27-Nov	UCC C1
L13	4-Dec	BMS G18
L14	11-Dec	BMS G18

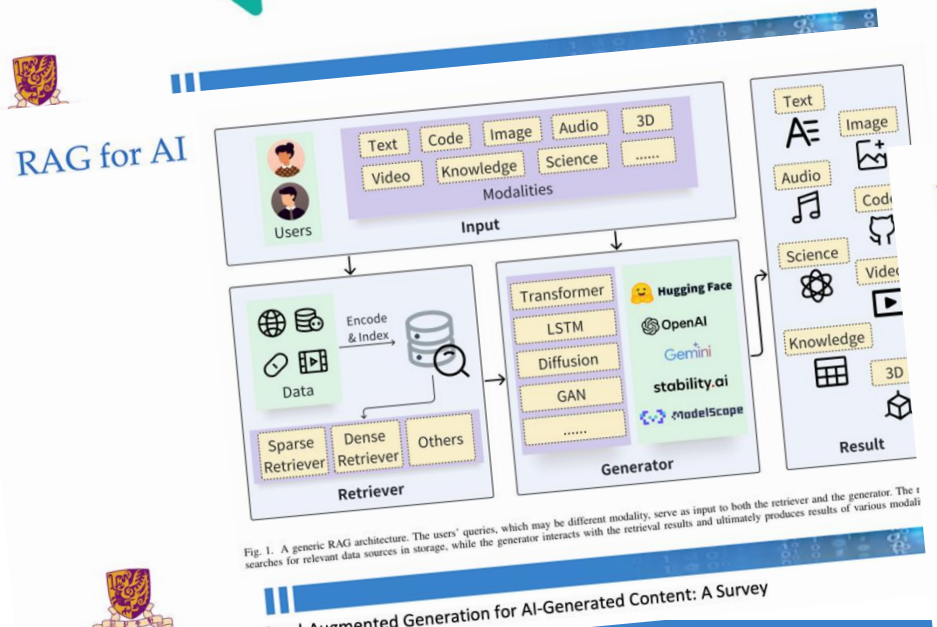


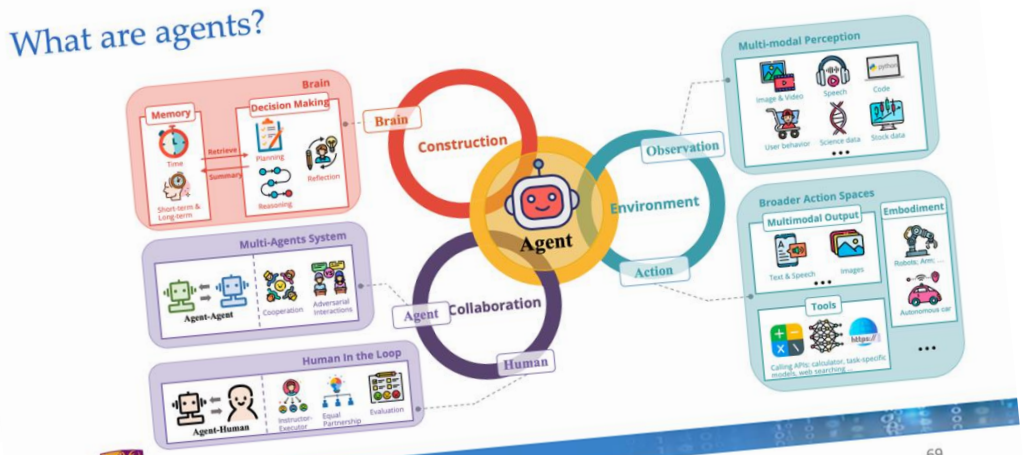
Selected Assignment 1

Last Lecture







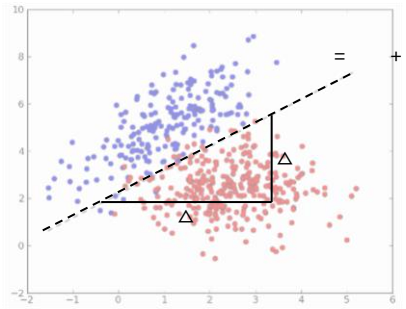


Course so far

Machine learning → **Transfer learning** → **NLP & LLM**

What is Machine Learning - Classification

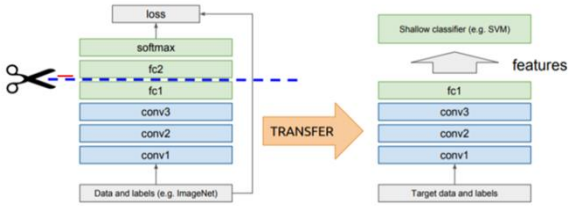
- Using a straight line equation as the separator.
- Variables: and
- Constants: and
- On graph:
  - = gradient
  - = y-axis intercept



50

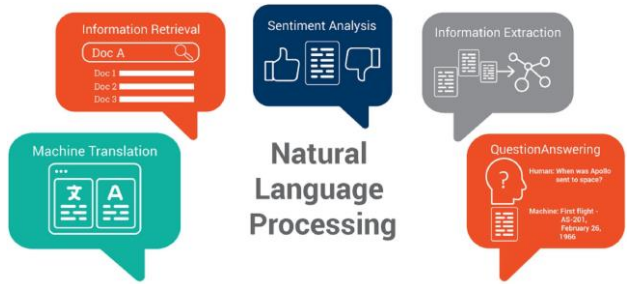
Representation-based Deep Transfer Learning

- Use outputs of one or mare layers to train on another model



28

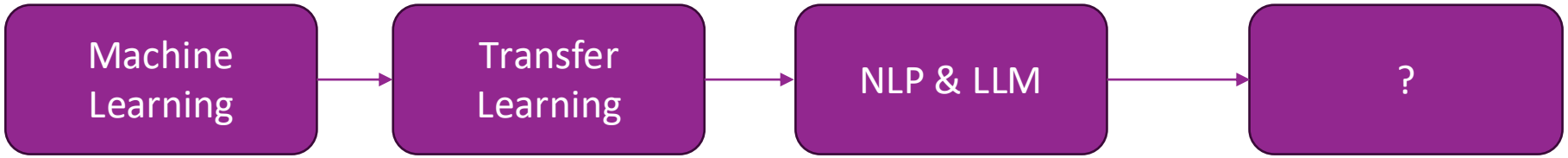
Natural Language Processing



11

Course so far

- How do we understand text and sequent data?
- How to we make LLM effective?





Agenda

- RNN & LSTM
- Transformer
- KV caches & Model efficiency

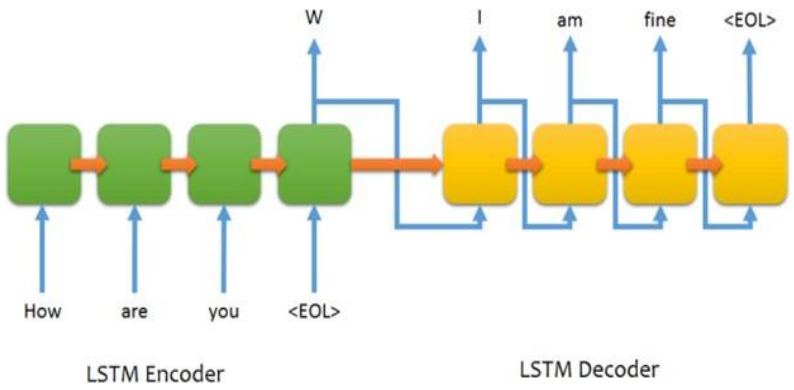


Recall: Introduction

- neural network models for labelled data:

$$\{x_i, y_i\}_{i=1}^{N} \rightarrow P(y|x)$$

where each sample is an **input** x_i and **an output** y_i.



- Each training example consists of **a sequence of inputs**
  - $(\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(\tau i)})$ and **a sequence of outputs** $(\mathbf{y}^{(1)}, \ldots, \mathbf{y}^{(\tau i)})$,
  - (*"How", "are", "you", –, –, –*), (*–, –, –, "I", "am", "fine"*).

- Training data can be from dialogue or machinetranslation



Recall: One Hot Vectors

- a binary vector used to represent categorical /text data
- Each category is represented by a unique index in the vector.
- without any implicit relationships between categories.

Island		Biscoe	Dream	Torgensen
Biscoe	→	1	0	0
Torgensen		0	0	1
Dream		0	1	0

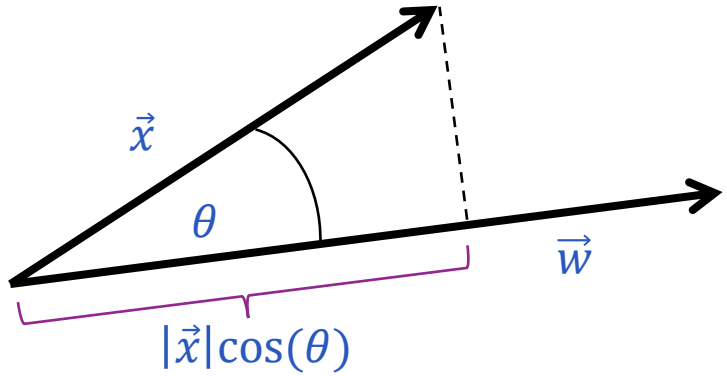


Recall: Linear Algebra

- Inner product: measuring the similarity of 2 vectors

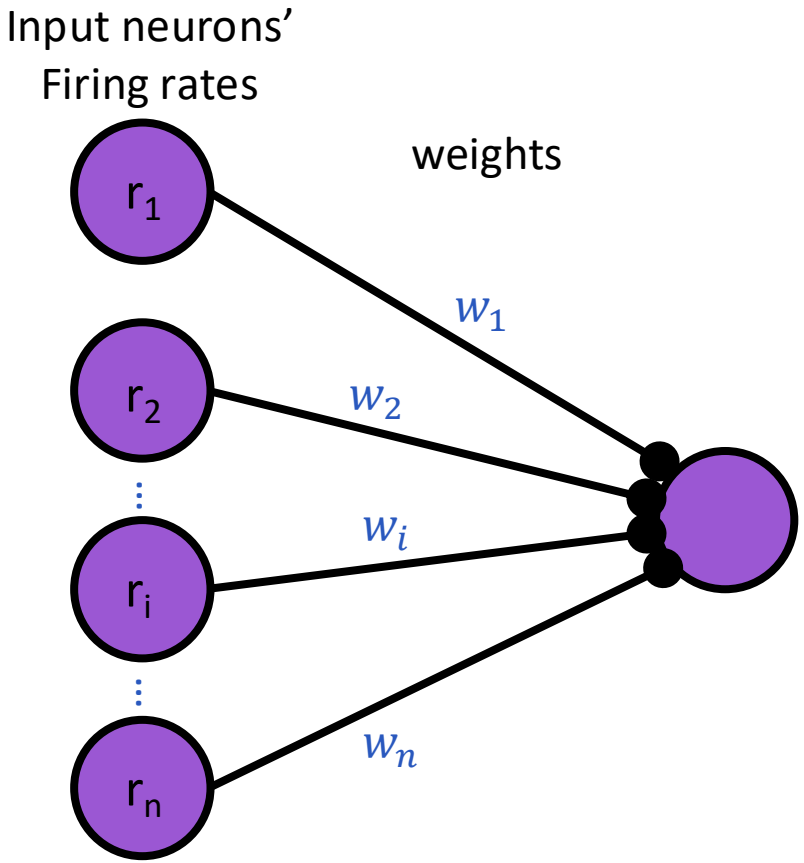
$$(w_1, w_2, \dots, w_N)\begin{pmatrix} x_1 \\ \cdots \\ x_N \end{pmatrix} = w_1x_1 + w_2x_2 + \dots + w_Nx_N = \sum_{i=1}^{N} w_ix_i$$

```
a = np.array([1,2,3,4])
b = np.array([0,5,1,6])
e=np.inner(a,b)
```



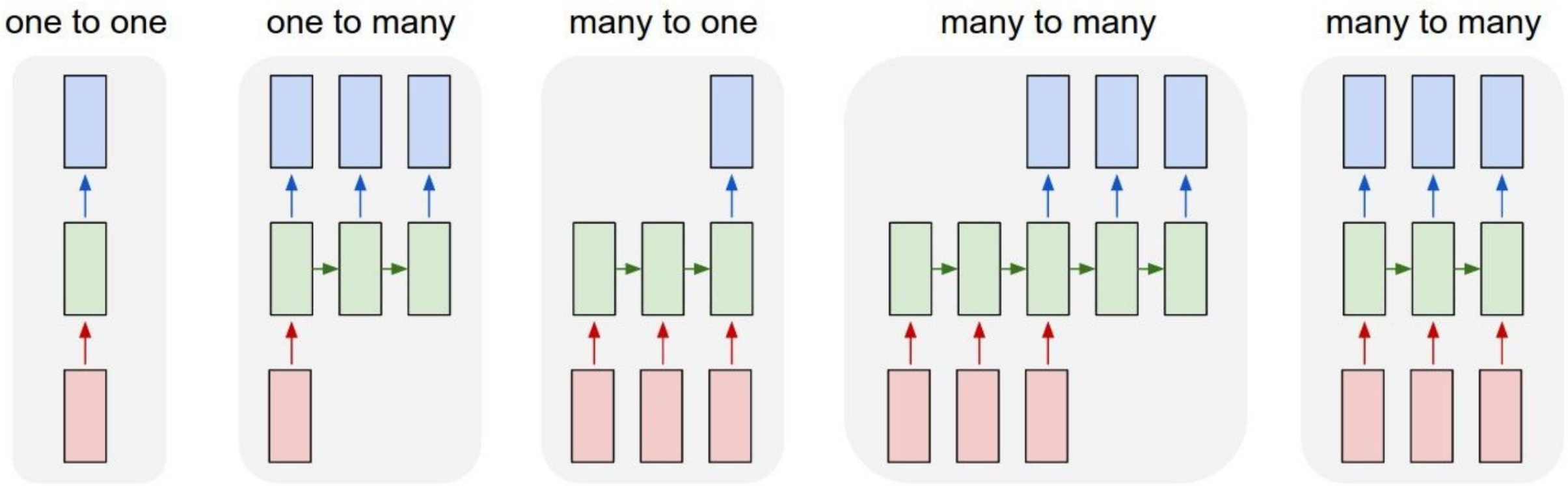
$$inner(\vec{x}, \vec{w}) = |\vec{x}||\vec{w}|\cos(\theta)$$

Recall: Classification and prediction



$$(w_1 \quad w_2 \quad \cdots \quad w_N)\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{pmatrix} = w_1x_1 + w_2x_2 + \cdots + w_Nx_N$$

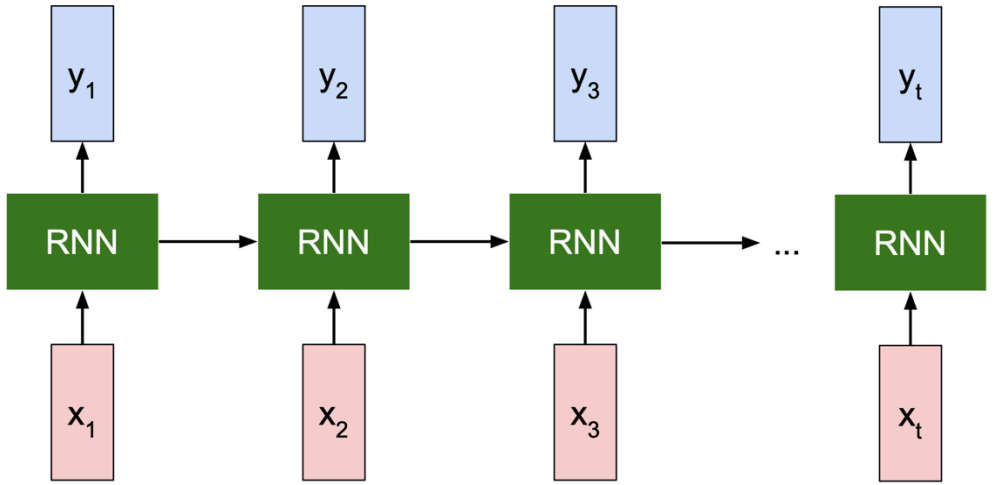
Possible use-cases





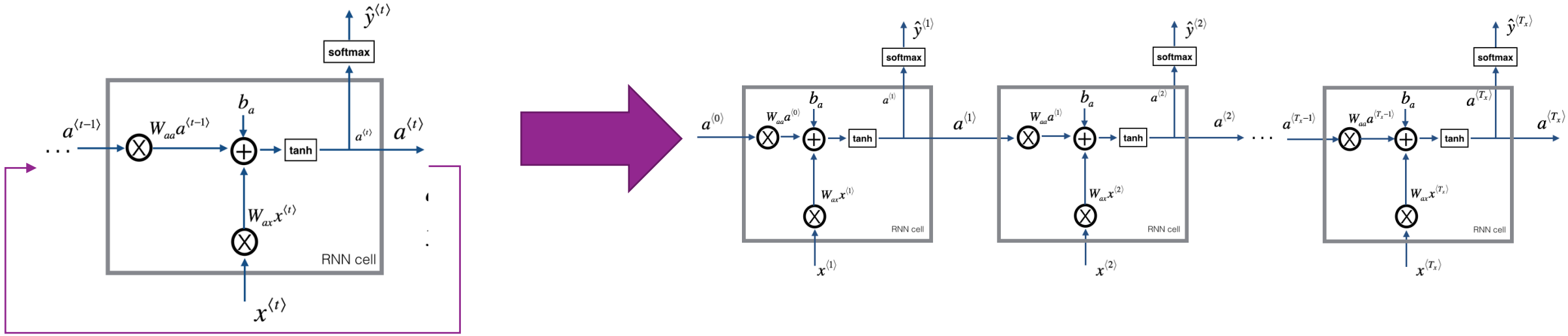
Recurrent Neural Networks (RNN)

- process sequential data by considering previous inputs and outputs.
- useful for language modeling, speech recognition, and machine translation
- variants such as **LSTM** and **GRU** networks:
  - Long Short-Term Memory
  - Gated Recurrent Unit



Recurrent Neural Networks (RNN)

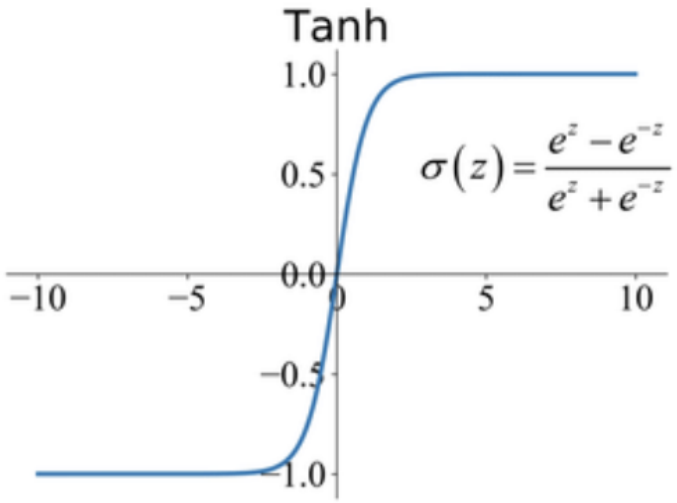
- The input $\mathbf{x}^{(t)}$ **and hidden states** $\mathbf{h}^{(t)}$ **and embedding vectors**
- The current $\mathbf{h}^{(t)}$ depends on the current input $\mathbf{x}^{(t)}$ and the previous state $\mathbf{h}^{(t-1)}$

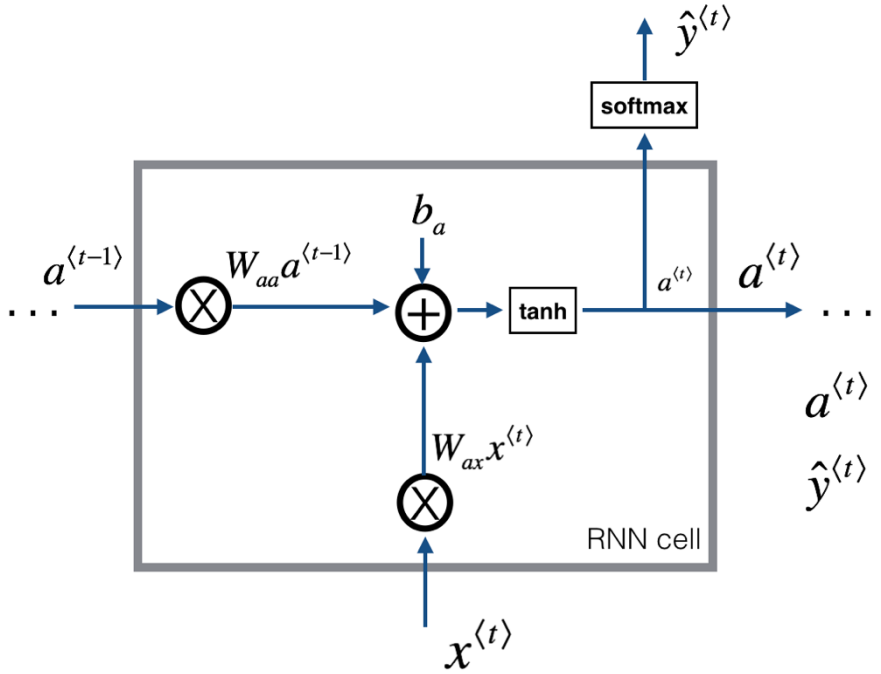


where W_{aa}, W_{ax}, W_{ay} are model parameters (independent of time t)

A RNN Cell

- Input: 1) the current input data; 2) the hidden state from the last timestep
- Output: 1) a new hidden state and an output (prediction)
- Note that $W_{aa}, W_{ax}, W_{ay}, b_a, b_y$ are shared across cells



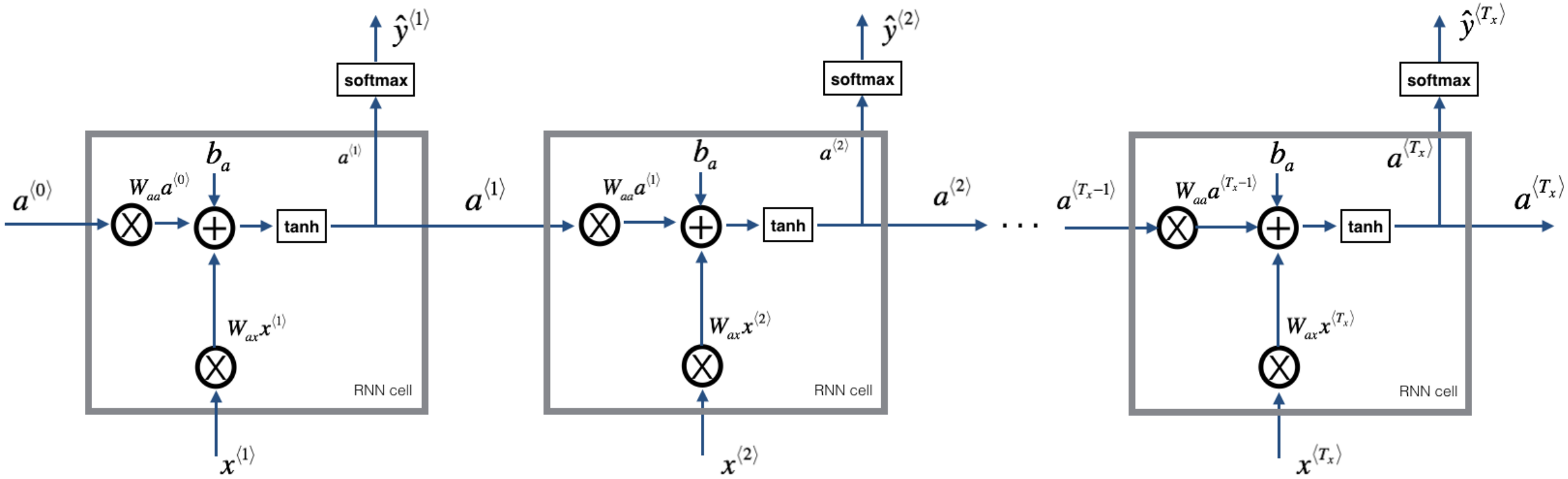


The new hidden state keeps track of context and dependencies between inputs.

$$a^{\langle t \rangle} = \tanh(W_{ax}x^{\langle t \rangle} + W_{aa}a^{\langle t-1 \rangle} + b_a)$$

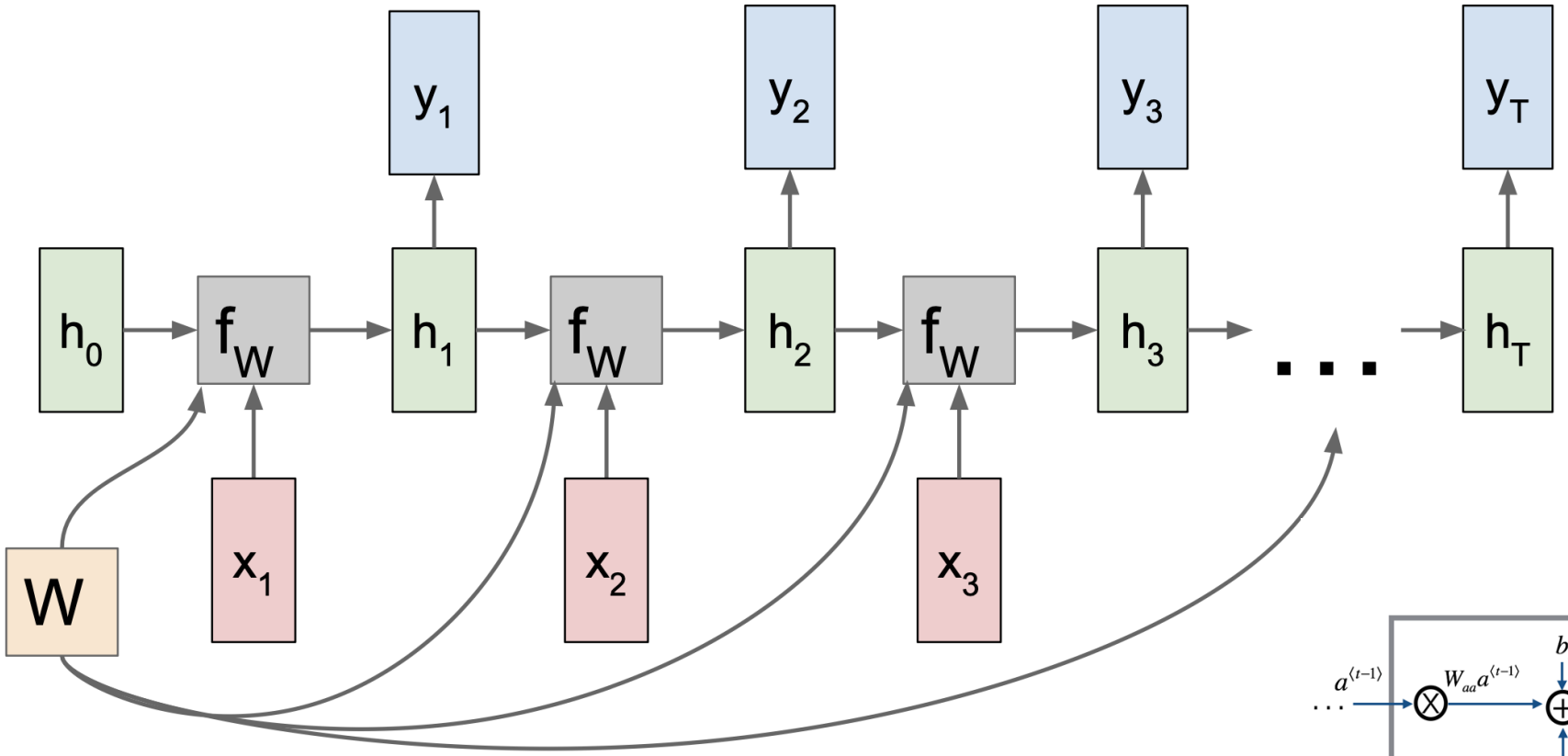
$$\hat{y}^{\langle t \rangle} = soft\max(W_{ya}a^{\langle t \rangle} + b_y)$$

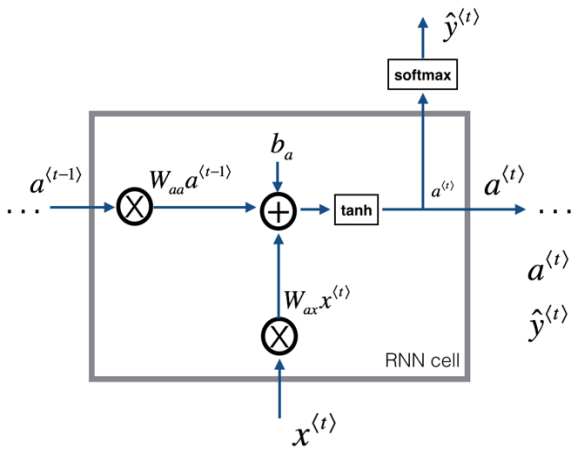
RNN Example


$\hat{y}^{\langle 1 \rangle}$
softmax
$a^{\langle 0 \rangle}$
$W_{aa}a^{\langle 0 \rangle}$
b_a
$a^{\langle 1 \rangle}$
tanh
$W_{ax}x^{\langle 1 \rangle}$
RNN cell
$x^{\langle 1 \rangle}$
$a^{\langle 1 \rangle}$
$\hat{y}^{\langle 2 \rangle}$
softmax
$W_{aa}a^{\langle 1 \rangle}$
b_a
$a^{\langle 2 \rangle}$
tanh
$W_{ax}x^{\langle 2 \rangle}$
RNN cell
$x^{\langle 2 \rangle}$
$a^{\langle 2 \rangle}$
$a^{\langle T_x-1 \rangle}$
$\hat{y}^{\langle T_x \rangle}$
softmax
$W_{aa}a^{\langle T_x-1 \rangle}$
b_a
$a^{\langle T_x \rangle}$
tanh
$W_{ax}x^{\langle T_x \rangle}$
RNN cell
$x^{\langle T_x \rangle}$
$a^{\langle T_x \rangle}$



RNN Example





$$a^{\langle t \rangle} = \tanh(W_{ax}x^{\langle t \rangle} + W_{aa}a^{\langle t-1 \rangle} + b_a)$$

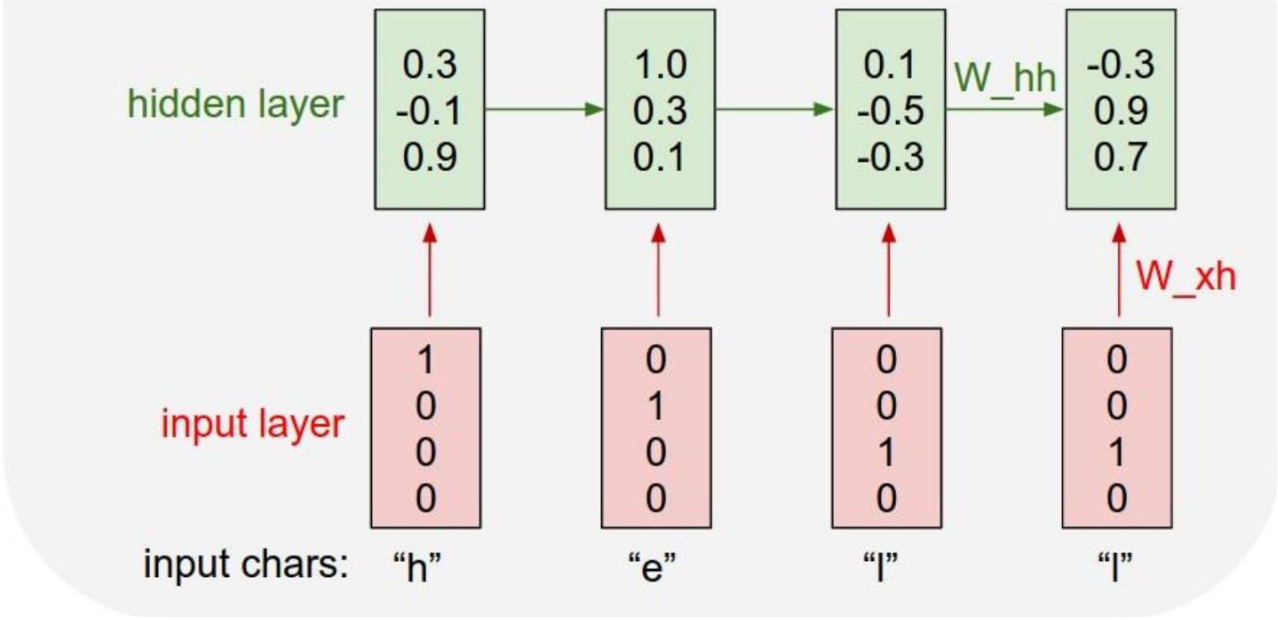
$$\hat{y}^{\langle t \rangle} = soft\max(W_{ya}a^{\langle t \rangle} + b_y)$$

RNN Example: Character-level model

- Vocabulary: [h, e, l, o]

- Example training sequence: “Hello”

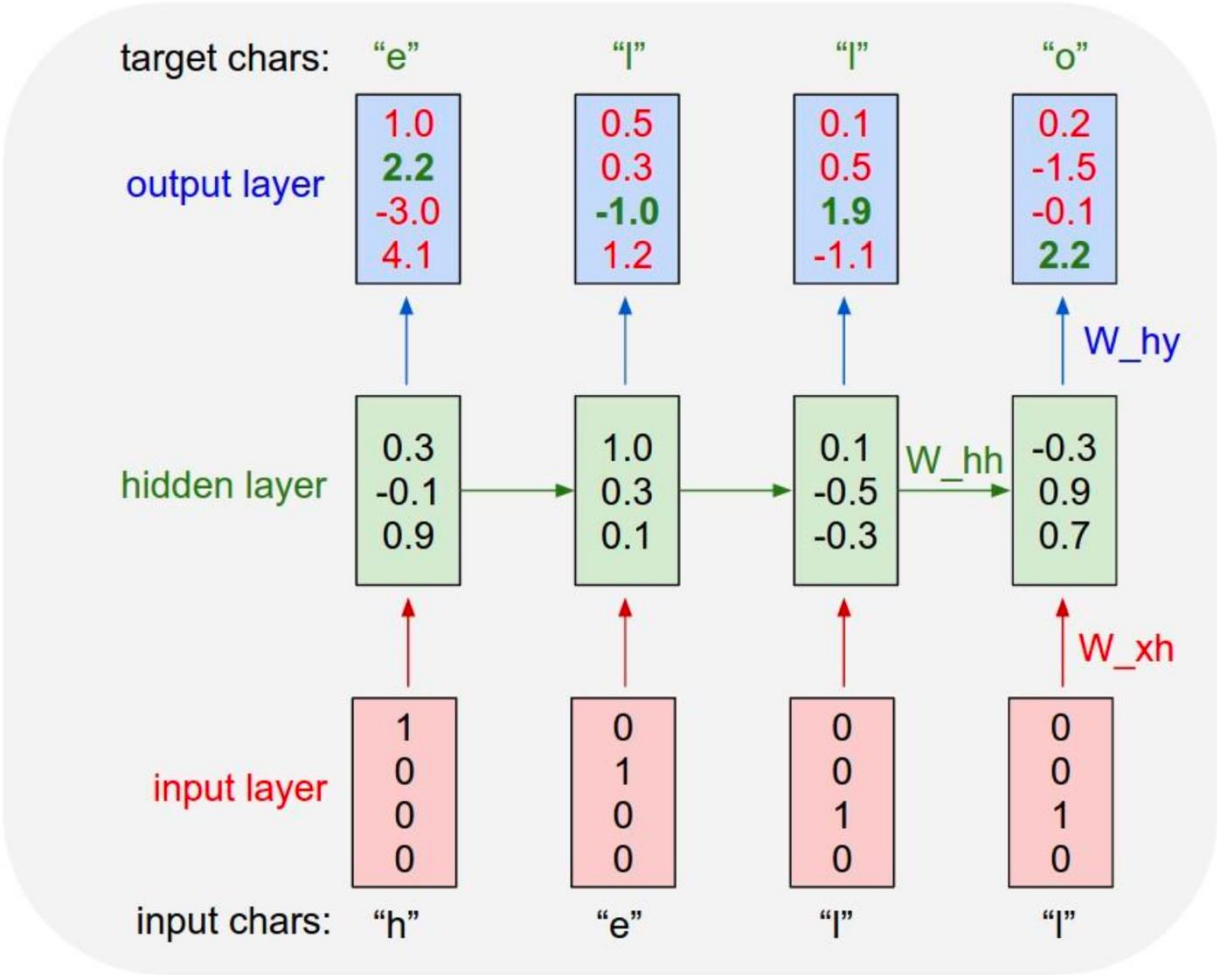
$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$$



Example: Character-level model

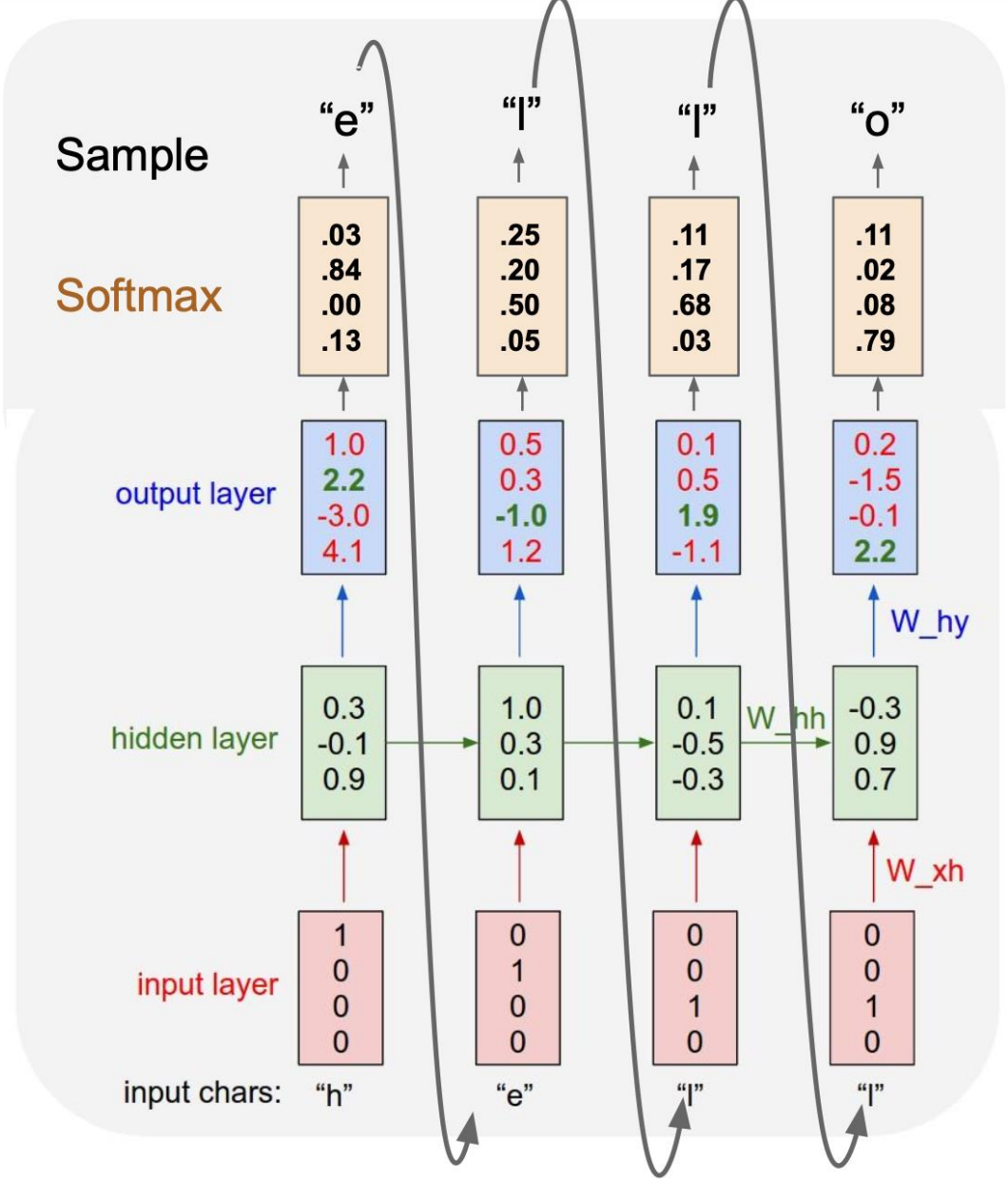
$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$$

- Vocabulary: [h, e, l, o]
- Example training sequence: “Hello”
- Training:



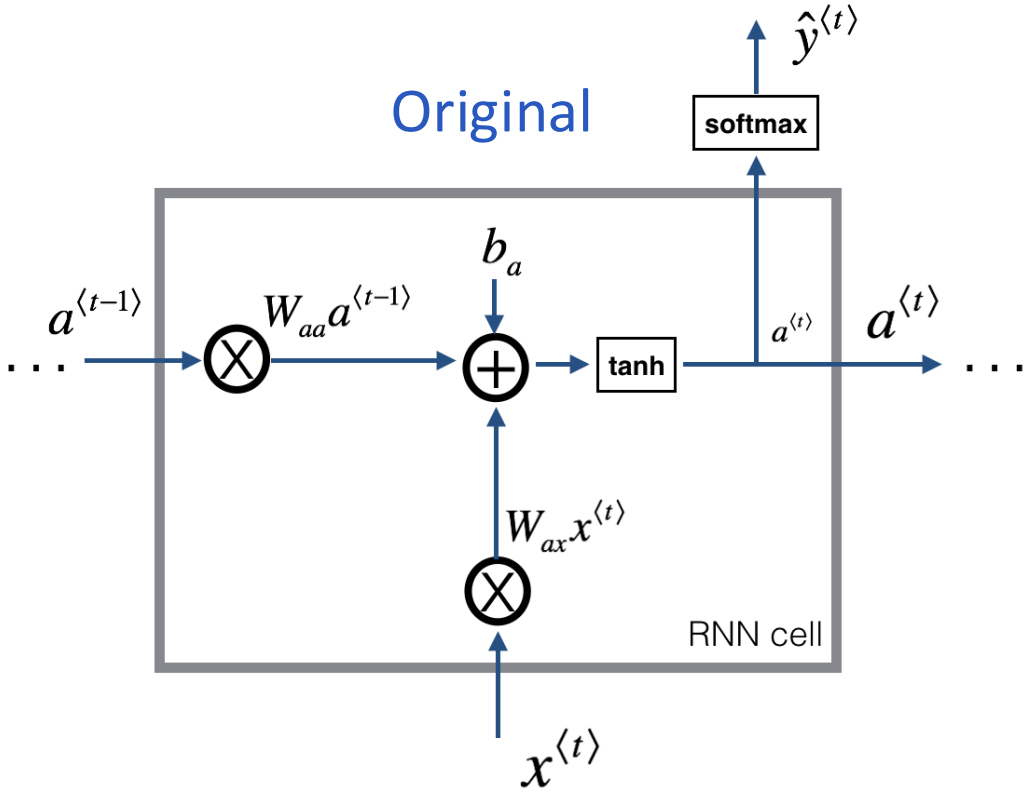
Example: Character-level model

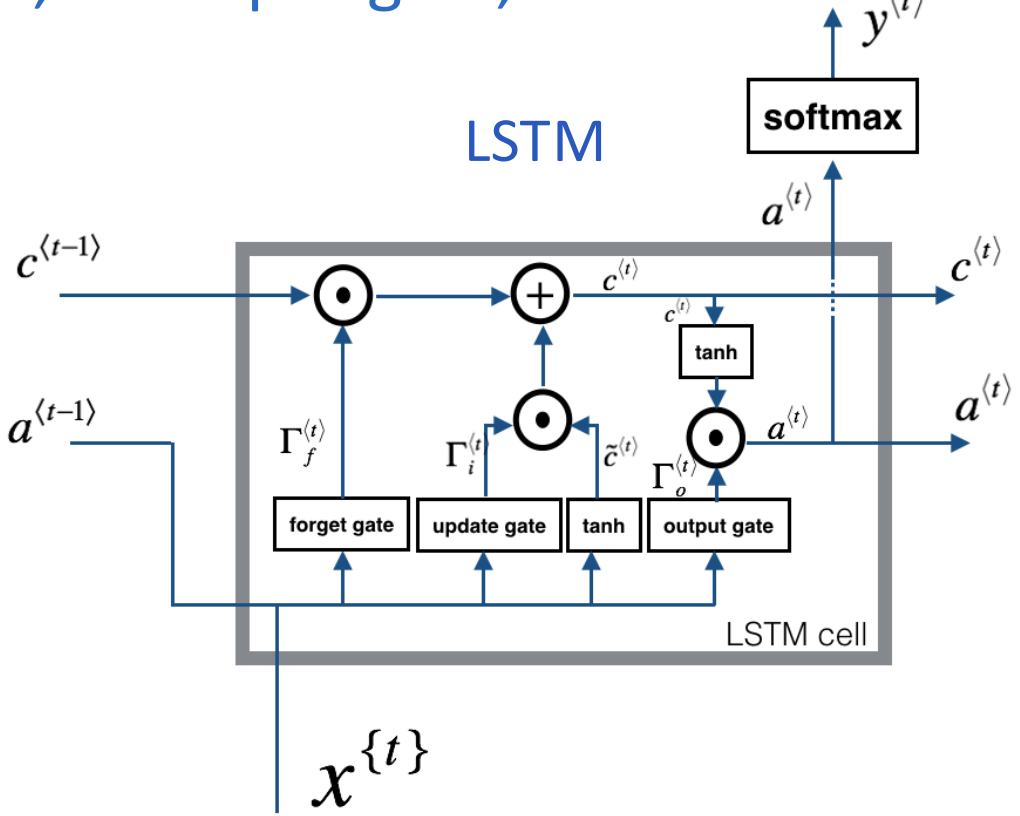
- Vocabulary: [h, e, l, o]
- Example training sequence: “Hello”
- Prediction:



New: Long Short-Term Memory (LSTM)

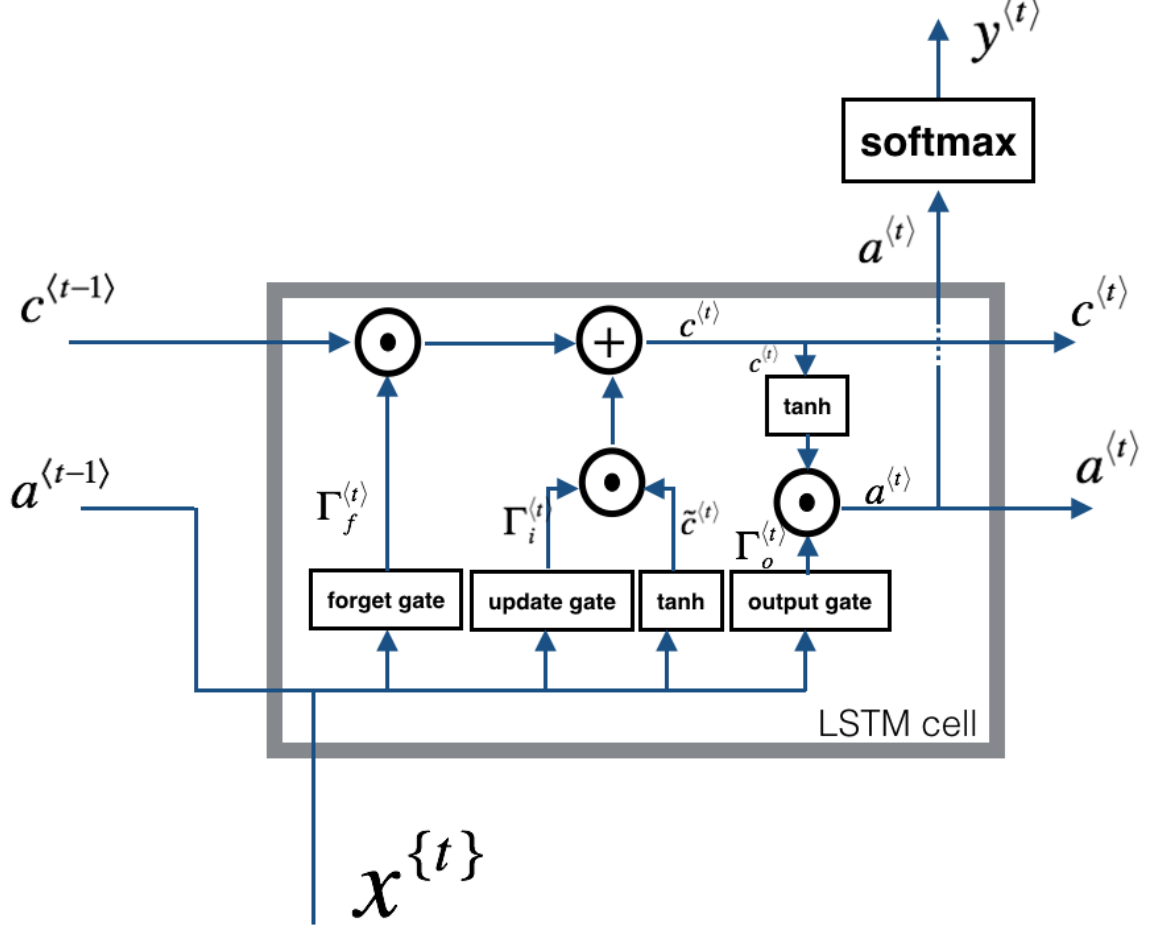
- components: 1. forget gate, 2. update gate, 3. output gate, 4. cell state





Long Short-Term Memory (LSTM)

- Forget gate: controls how much information is retained in the cell state from the previous timestep.
- Update gate: controls how much information to update cell state
- Output gate: controls how much information to the next hidden state
- Cell state: a memory to remember information over long sequences of data.



$$\Gamma_f^{\langle t \rangle} = \sigma(W_f[a^{\langle t-1 \rangle}, x^{\langle t \rangle}] + b_f) \qquad c^{\langle t \rangle} = \Gamma_f^{\langle t \rangle} \circ c^{\langle t-1 \rangle} + \Gamma_u^{\langle t \rangle} \circ \tilde{c}^{\langle t \rangle}$$

$$\Gamma_u^{\langle t \rangle} = \sigma(W_u[a^{\langle t-1 \rangle}, x^{\langle t \rangle}] + b_u) \qquad \Gamma_o^{\langle t \rangle} = \sigma(W_o[a^{\langle t-1 \rangle}, x^{\langle t \rangle}] + b_o)$$

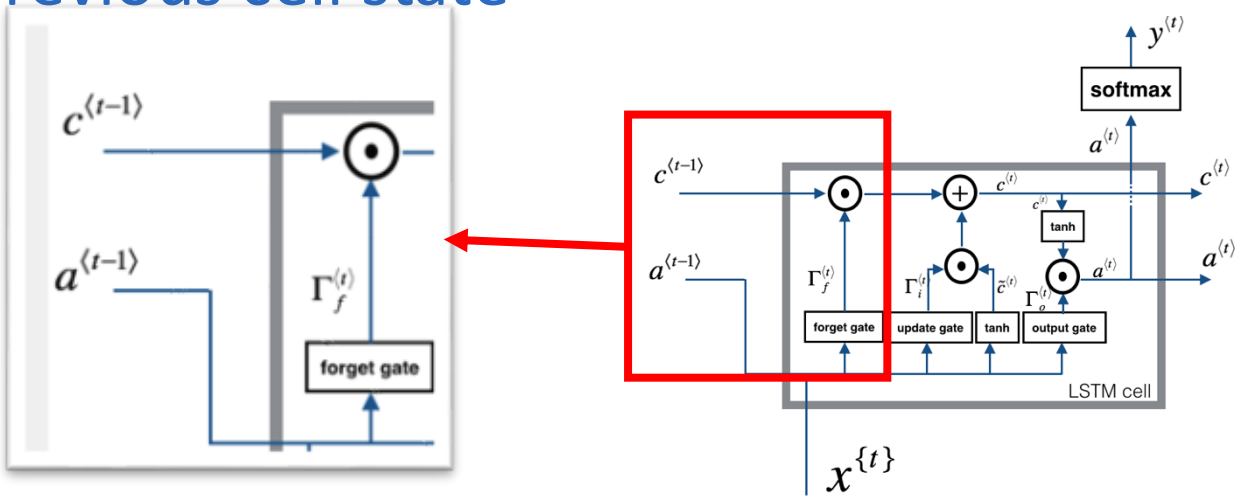
$$\tilde{c}^{\{t\}} = \tanh(W_c[a^{\langle t-1 \rangle}, x^{\langle t \rangle}] + \qquad a^{\langle t \rangle} = \Gamma_o^{\langle t \rangle} \circ \tanh(c^{\langle t \rangle})$$

Forget Gate

- For keeping track of grammatical structures (e.g., singular or plural):
  - a way to get rid of our previously stored memory value of the singular/plural state.

$$\Gamma_f^{\langle t \rangle} = \sigma(W_f[a^{\langle t-1 \rangle}, x^{\langle t \rangle}] + b_f)$$

- The equation above results in a vector with values between 0 and 1 and will be multiplied element-wise by the previous cell state

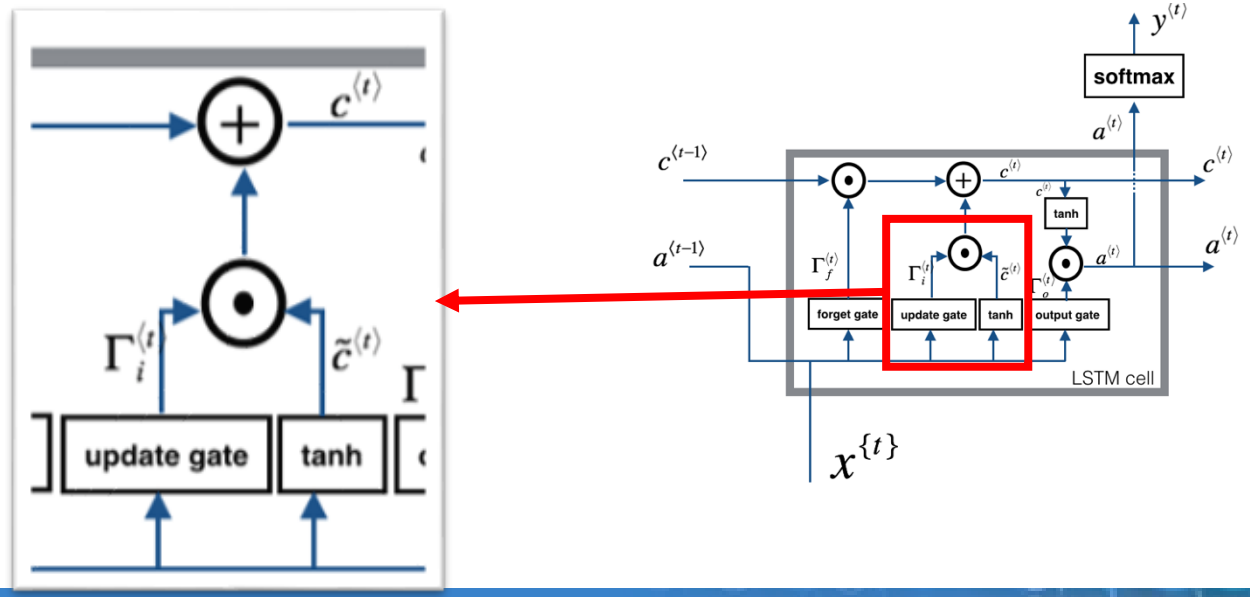


Update Gate

- To update it to reflect that the new subject is now singular or plural:

$$\Gamma_u^{\langle t \rangle} = \sigma(W_u[a^{\langle t-1 \rangle}, x^{\{t\}}] + b_u)$$

- Similar to the forget gate, here is a vector of values between 0 and 1.



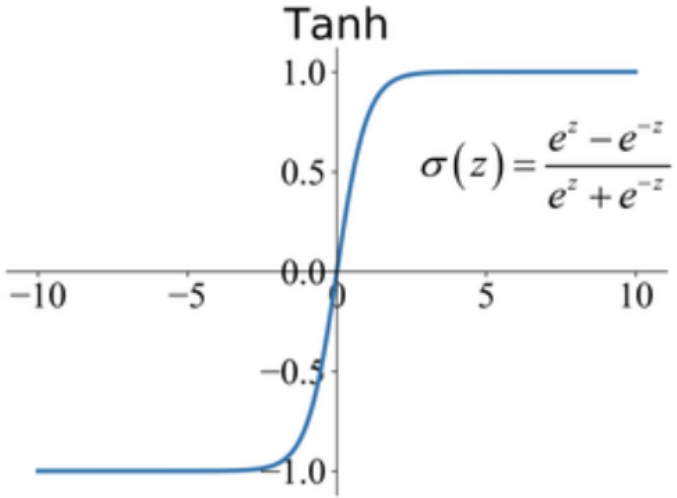


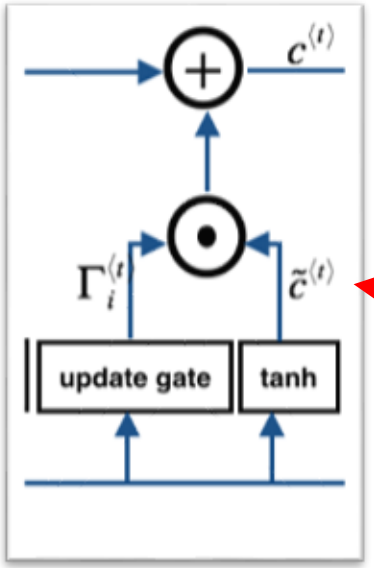
Updating the cell

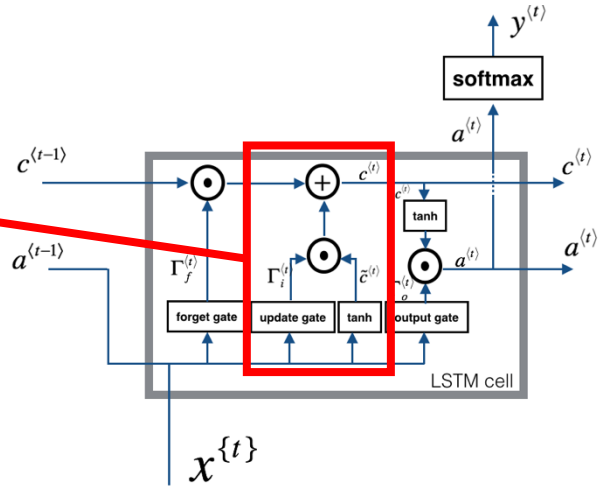
- To update the new subject we need to create a new vector of numbers that we can add to our previous cell state

$$\tilde{c}^{\langle t \rangle} = \tanh(W_c[a^{\langle t-1 \rangle}, x^{\langle t \rangle}] + b_c)$$

- Finally, the new cell state is: $c^{\langle t \rangle} = \Gamma_f^{\langle t \rangle} * c^{\langle t-1 \rangle} + \Gamma_u^{\langle t \rangle} * \tilde{c}^{\langle t \rangle}$







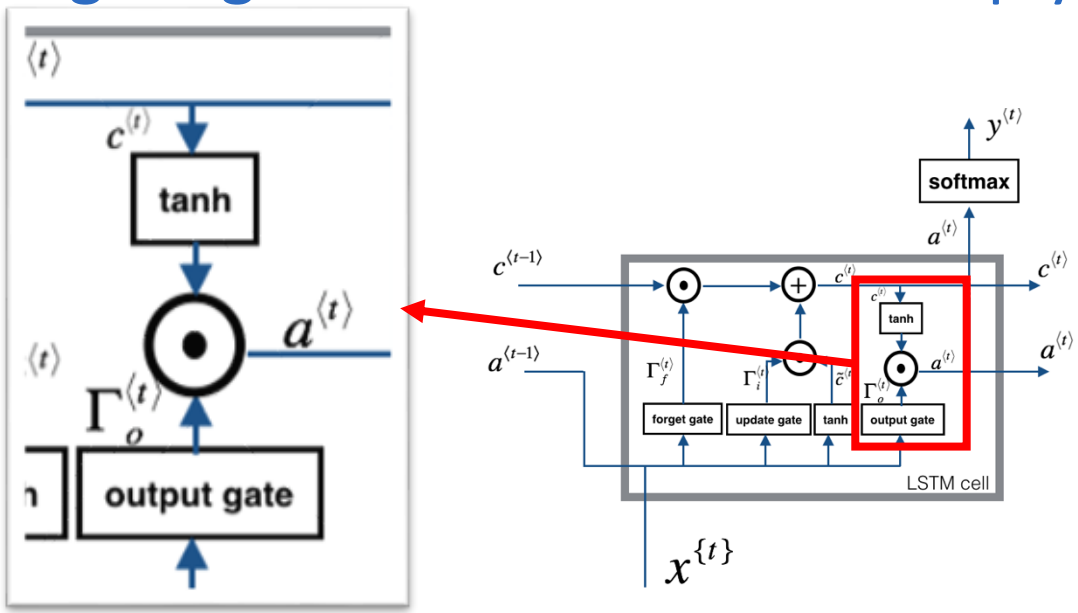
Output gate

- To decide which outputs we will use, we will use the following two formulas:

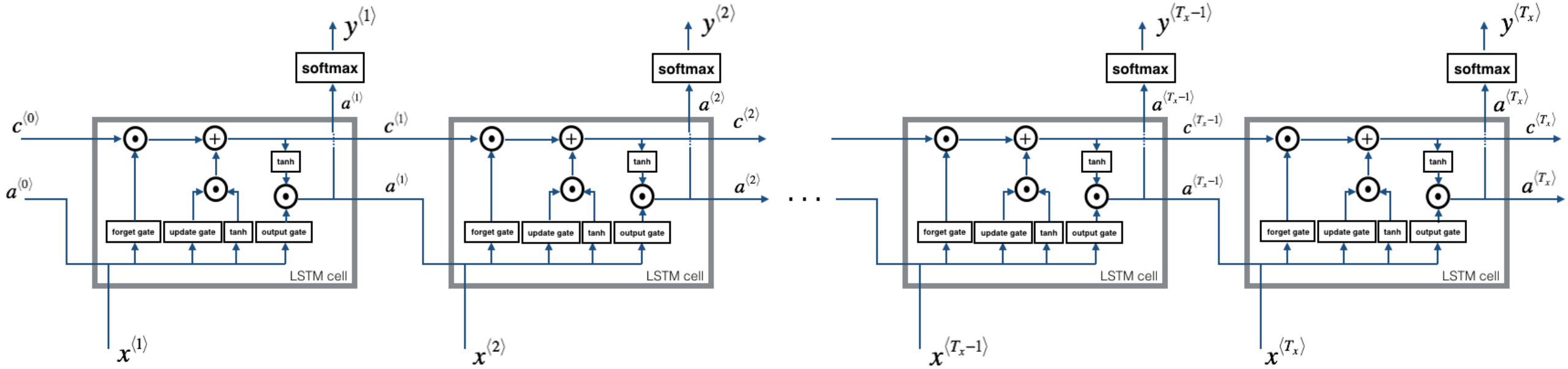
$$\Gamma_o^{\langle t \rangle} = \sigma(W_o[a^{\langle t-1 \rangle}, x^{\langle t \rangle}] + b_o)$$

$$a^{\langle t \rangle} = \Gamma_o^{\langle t \rangle} * \tanh(c^{\langle t \rangle})$$

- Where you decide what to output using a sigmoid function and multiply that by the tanh of the previous state.



LSTM Example



LSTM vs. Traditional RNN

Aspect	LSTM	Traditional RNN
Structure	Contains memory cell and gates (input, forget, output)	Simple recurrent layer with a hidden state
Handling Long-Term Dependencies	Effective due to the memory cell and gating mechanisms	Struggles due to the vanishing/exploding gradient problem
Gradient Flow	Preserves gradient flow over long sequences	Suffers from vanishing/exploding gradient problem
Handling Short-Term Dependencies	Performs well due to the gating mechanisms	Performs reasonably well
Memory Retention	Retains information over long sequences	Tends to forget information quickly
Training Time	Longer training time due to the complex structure	Faster training time
Computational Complexity	Higher computational complexity	Lower computational complexity
Usage	Preferred choice for tasks requiring long-term memory	Commonly used for simple sequential tasks

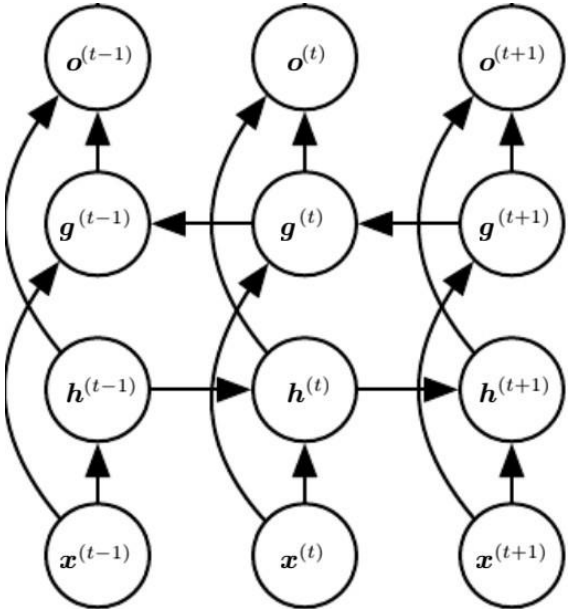


Bidirectional RNNs

- In one directional RNN, $\mathbf{h}^{(t)}$ only capture information from the past $(\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(t)})$, and we use $\mathbf{h}^{(t)}$ to predict $\mathbf{y}^{(t)}$

- Sometime, future information is needed to predict $\mathbf{y}^{(t)}$.

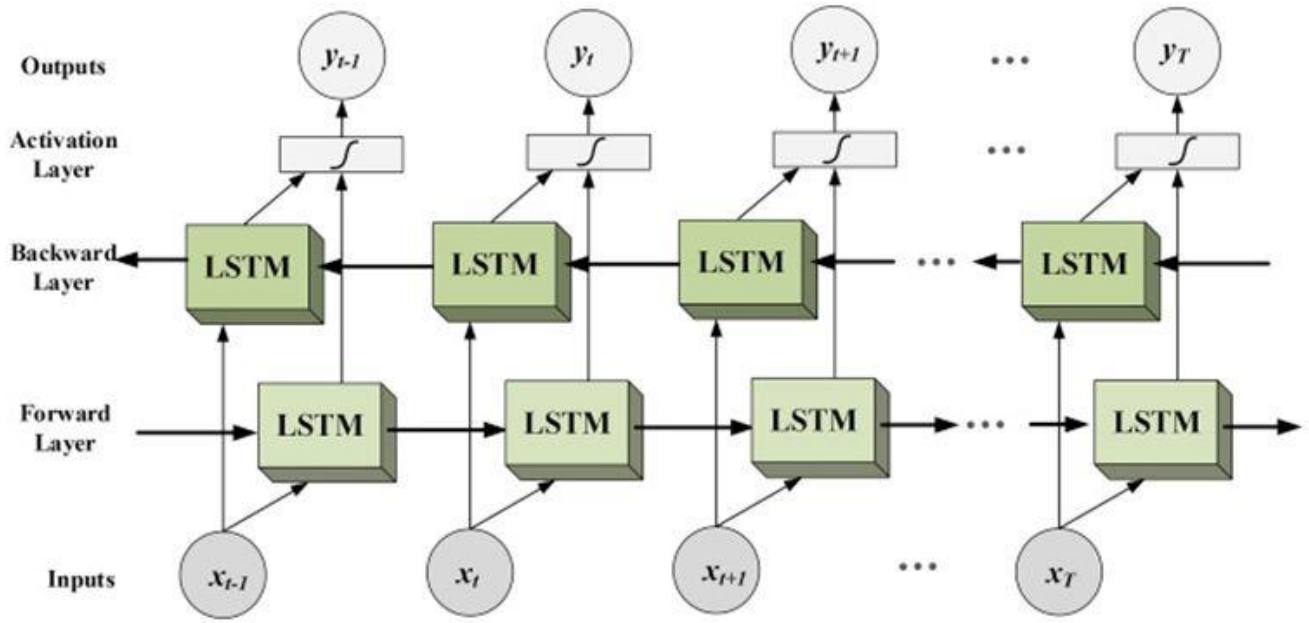
- Useful in handwriting recognition, speech recognition, and bioinformatics.



Deep RNNs

- we use multiple layers of hidden units => **deep recurrent neural network**.

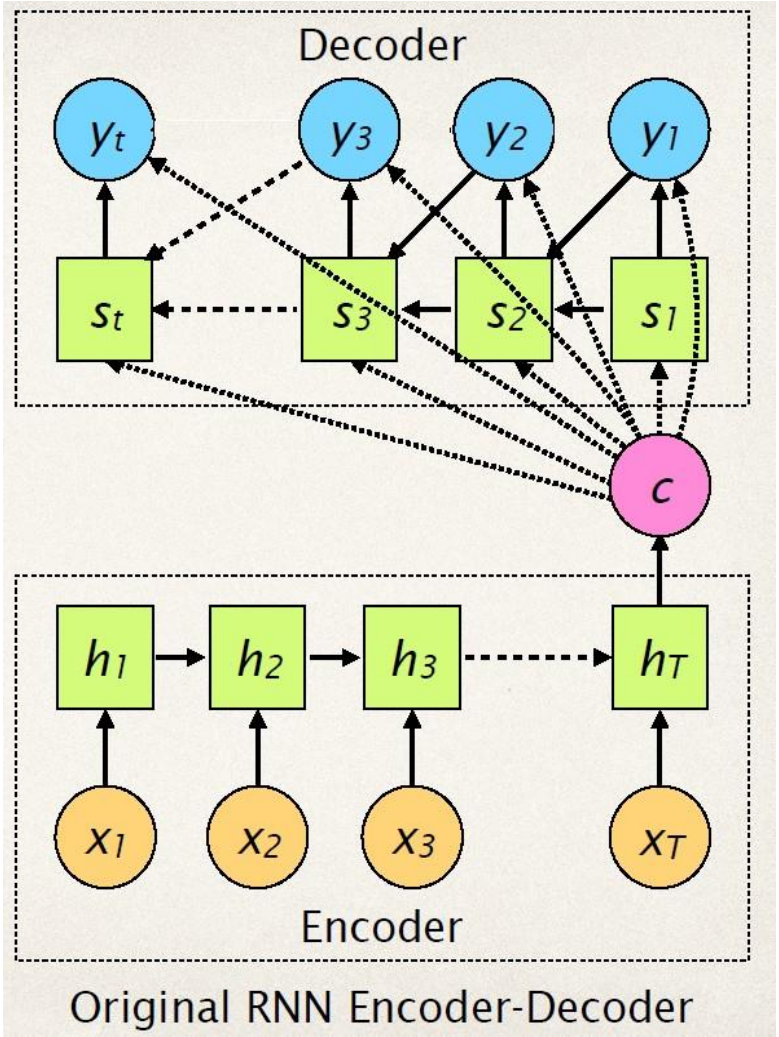
- More layers usually implies better performance.





The Encoder-Decoder Architecture

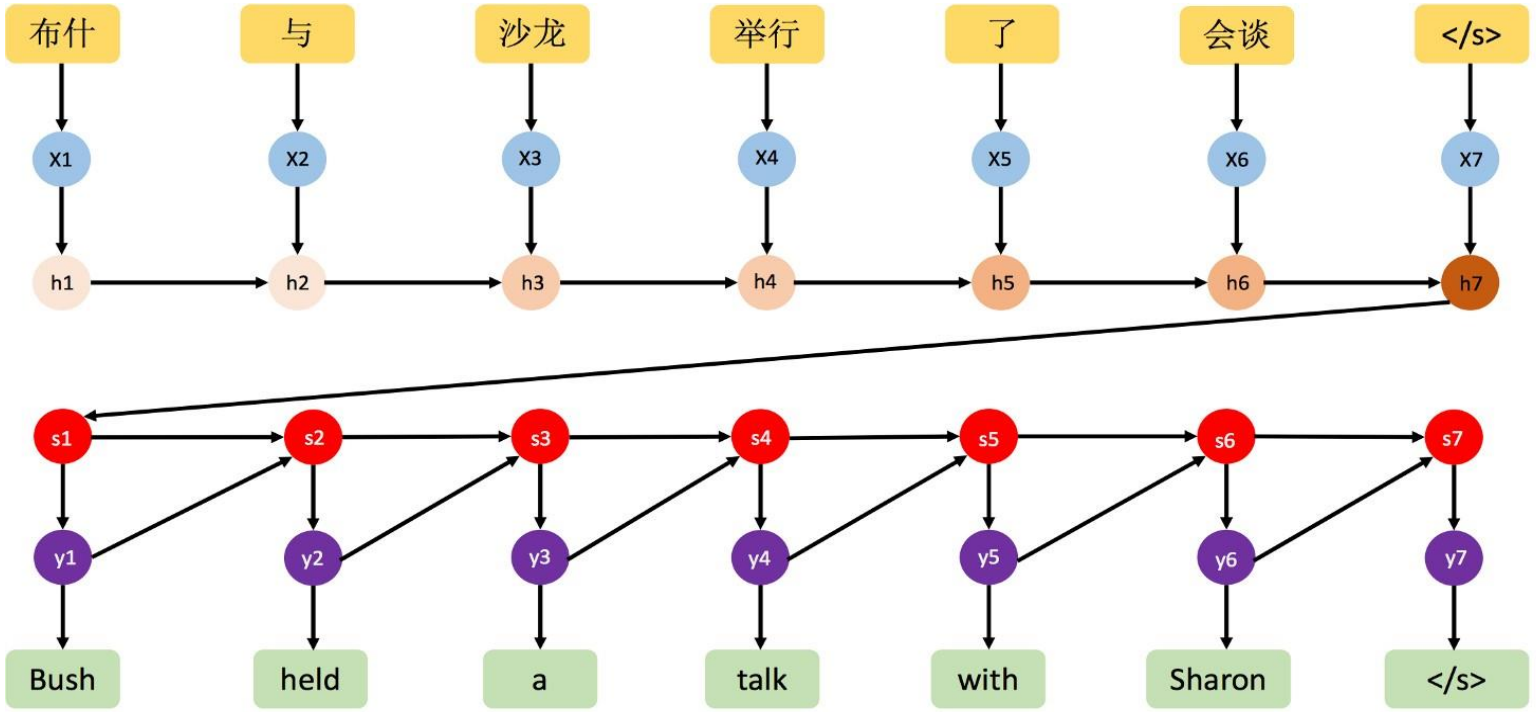
- $(y^{(1)}, \ldots, y^{(n_y)})$ in response to $(x^{(1)}, \ldots, x^{(n_x)})$
- The **context variable** C represents the input sequence
- The decoder defines a conditional distribution $P(y^{(1)}, \ldots, y^{(n_y)}|C)$ over sequences
- The architecture is used in dialogue systems and machine translation



Original RNN Encoder-Decoder

Seq2seq for machine translation

- The generated sequence can be a translation of the input sequence in machine translation



(Sutskever et al., 2014)

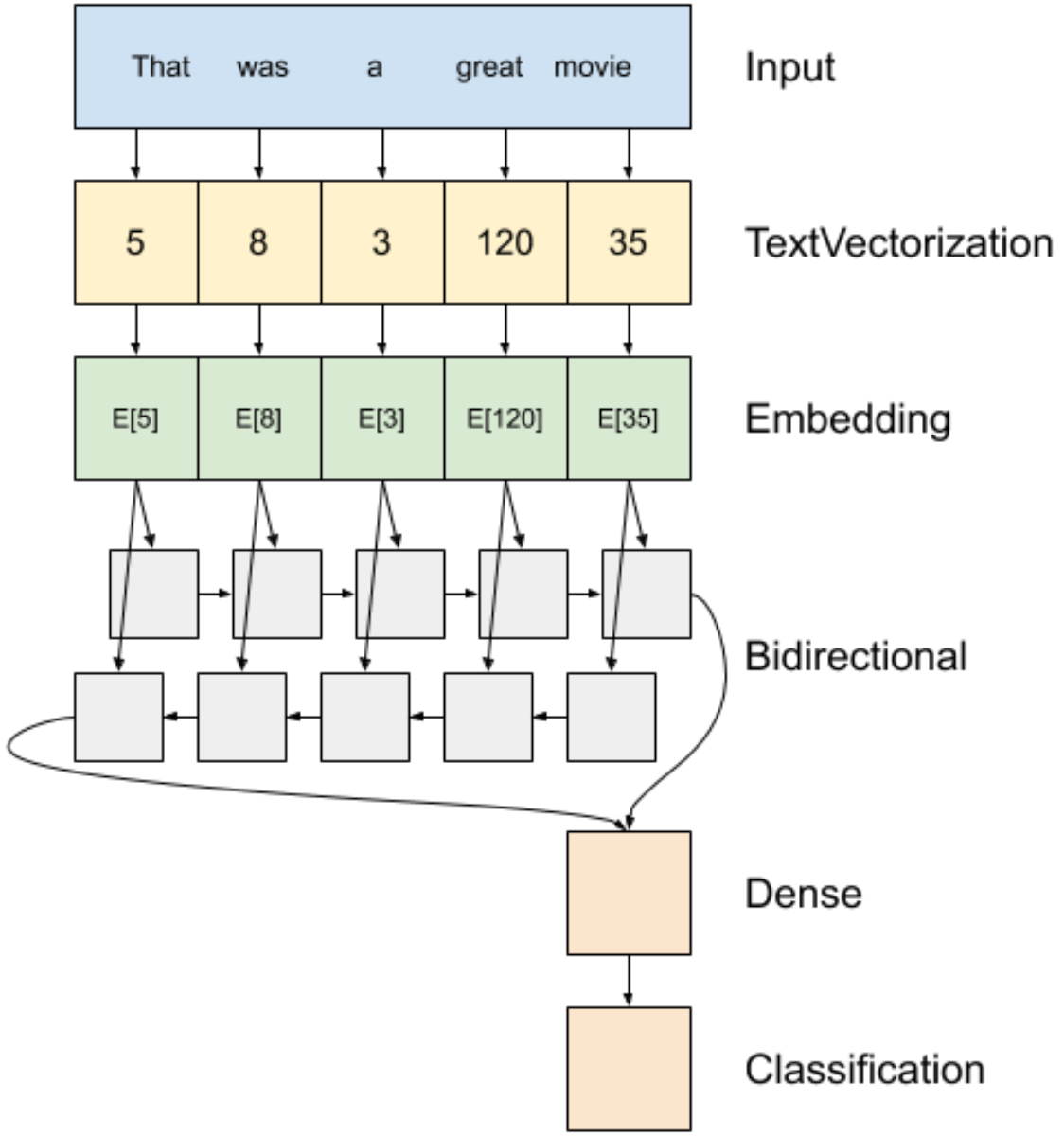
Seq2Seq model

```python
# Efficient implementation equivalent to the following with bidirectional=False
def forward(x, h_0=None):
    if batch_first:
        x = x.transpose(0, 1)
    seq_len, batch_size, _ = x.size()
    if h_0 is None:
        h_0 = torch.zeros(num_layers, batch_size, hidden_size)
    h_t_minus_1 = h_0
    h_t = h_0
    output = []
    for t in range(seq_len):
        for layer in range(num_layers):
            h_t[layer] = torch.tanh(
                x[t] @ weight_ih[layer].T
                + bias_ih[layer]
                + h_t_minus_1[layer] @ weight_hh[layer].T
                + bias_hh[layer]
            )
        output.append(h_t[-1])
        h_t_minus_1 = h_t
    output = torch.stack(output)
    if batch_first:
        output = output.transpose(0, 1)
    return output, h_t
```

- **input_size** – The number of expected features in the input *x*
- **hidden_size** – The number of features in the hidden state *h*
- **num_layers** – Number of recurrent layers. E.g., setting `num_layers=2` would mean stacking two RNNs together to form a *stacked RNN*, with the second RNN taking in outputs of the first RNN and computing the final results. Default: 1
- **nonlinearity** – The non-linearity to use. Can be either `'tanh'` or `'relu'`. Default: `'tanh'`
- **bias** – If `False`, then the layer does not use bias weights *b_ih* and *b_hh*. Default: `True`
- **batch_first** – If `True`, then the input and output tensors are provided as (*batch, seq, feature*) instead of (*seq, batch, feature*). Note that this does not apply to hidden or cell states. See the Inputs/Outputs sections below for details. Default: `False`
- **dropout** – If non-zero, introduces a *Dropout* layer on the outputs of each RNN layer except the last layer, with dropout probability equal to `dropout`. Default: 0
- **bidirectional** – If `True`, becomes a bidirectional RNN. Default: `False`



RNN for Classification





RNN for Classification

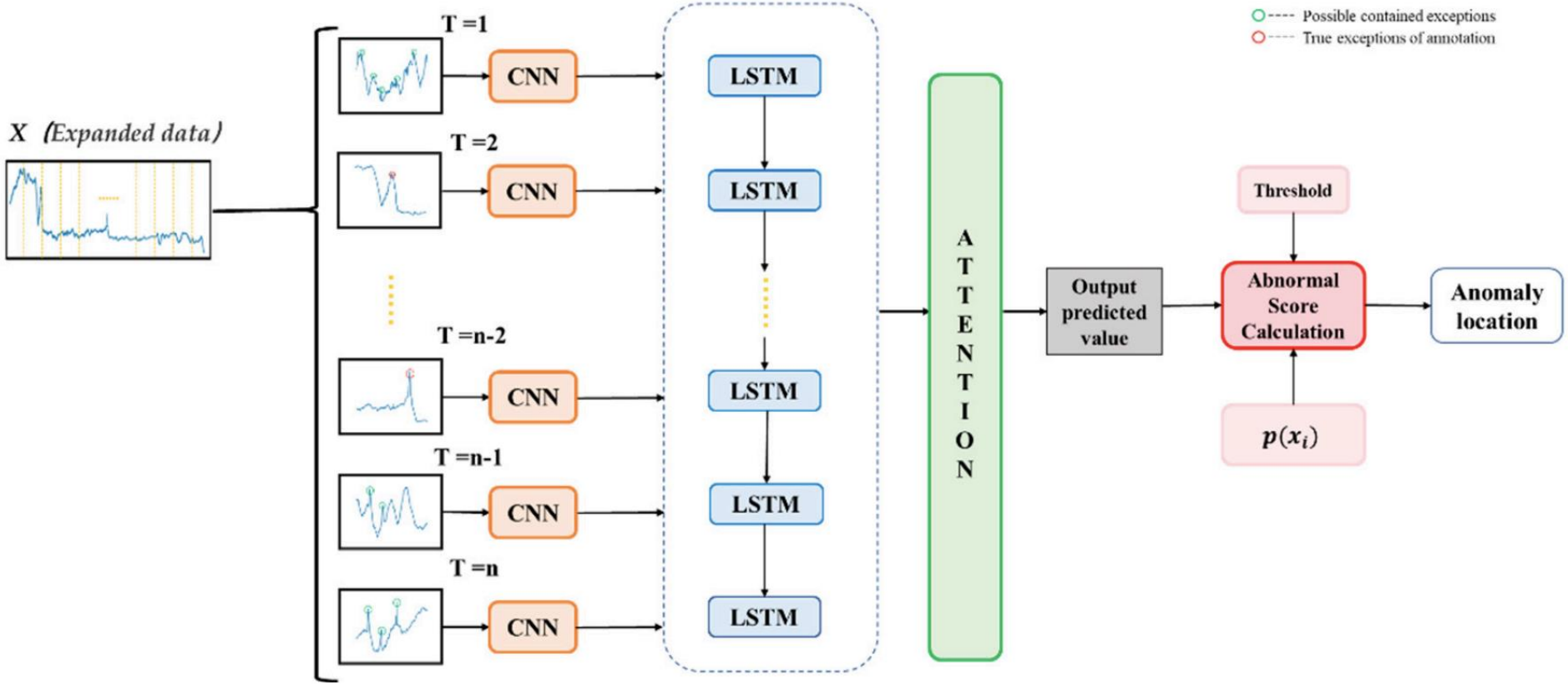
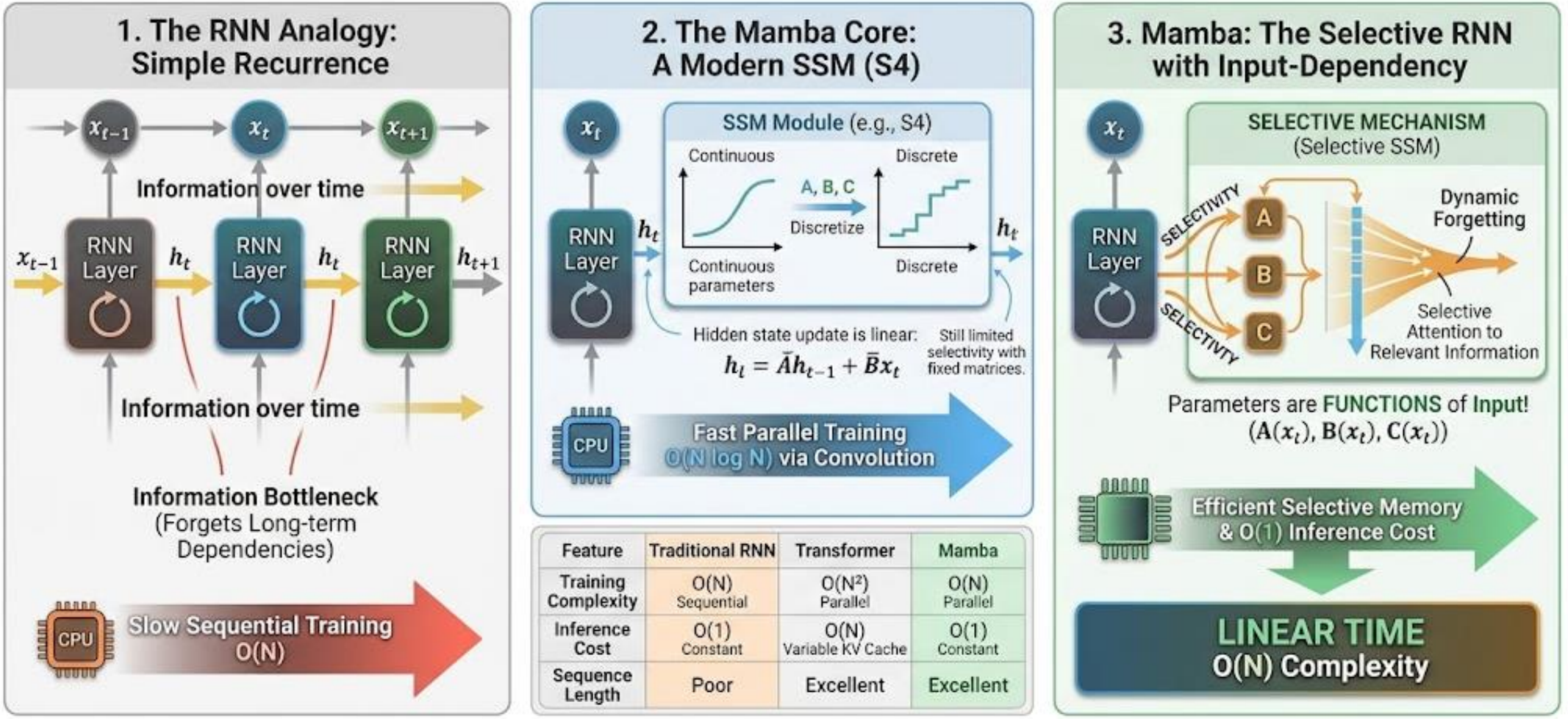


Figure 4. The simple flow chart of the proposed anomaly detection.

Mamba: RNN-LIKE structure in LLM



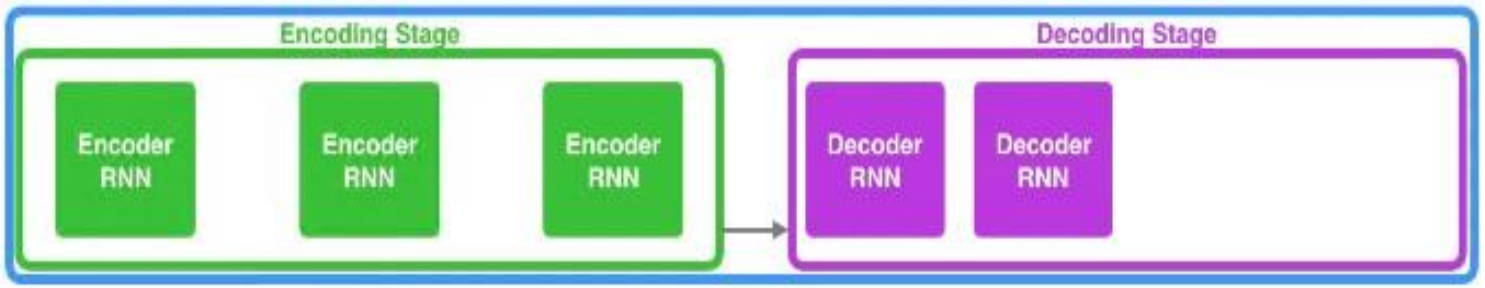
Agenda

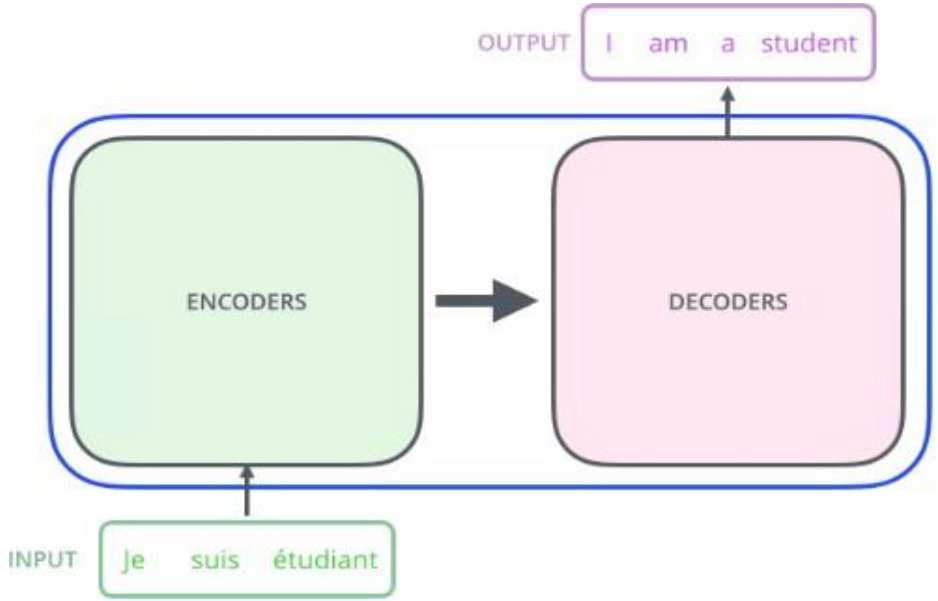
- RNN & LSTM
- Transformer
- KV caches & Model efficiency



Two Seq2Seq Models

- **RNN** is sequential. It precludes parallelization within training examples
- **Transformer** allows significantly more parallelization:
  - Based solely on attention, dispensing with recurrence.
  - Requires less time to train and achieves better results than Seq2Seq
  - Based on attention



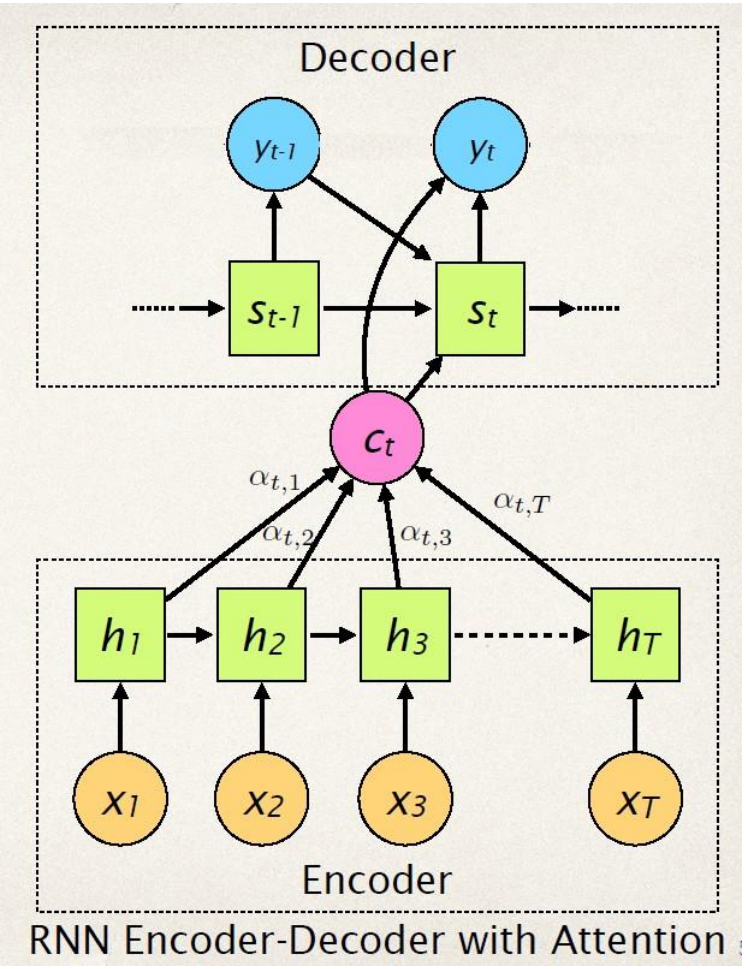


Attention

- Consider $\mathbf{C}_t$ for each time step of the decoder:

$$C_t = \sum_j a_{t,j} h_j$$

- a linear combinations of the hidden states from the encoder
- At time t, different of $\alpha_{t,j}$, to focus on different parts
- For different input sequences, the weights $\alpha_{t,j}$ are different. So, attention means **dynamic weights**.



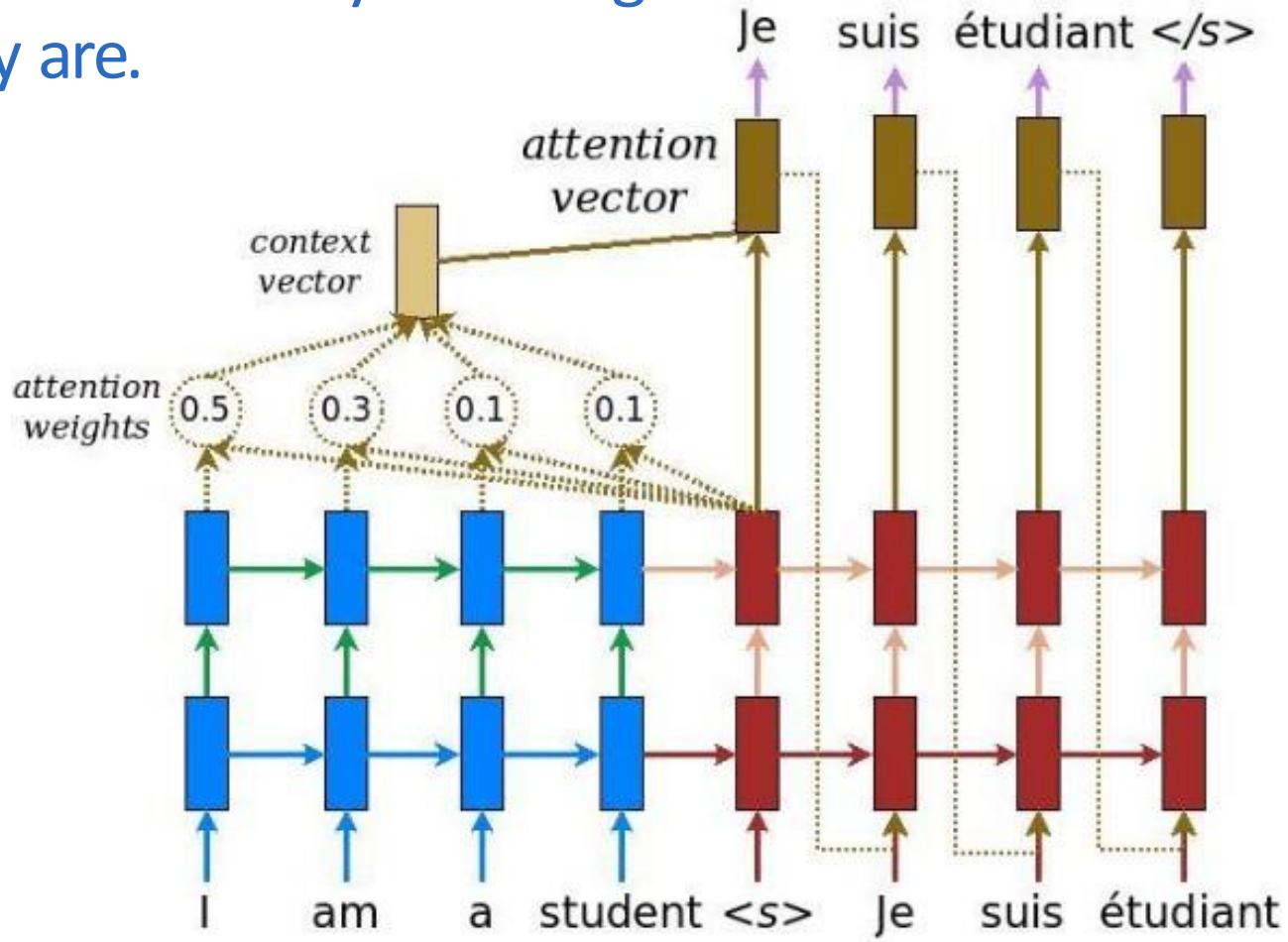
RNN Encoder-Decoder with Attention

Attention Example

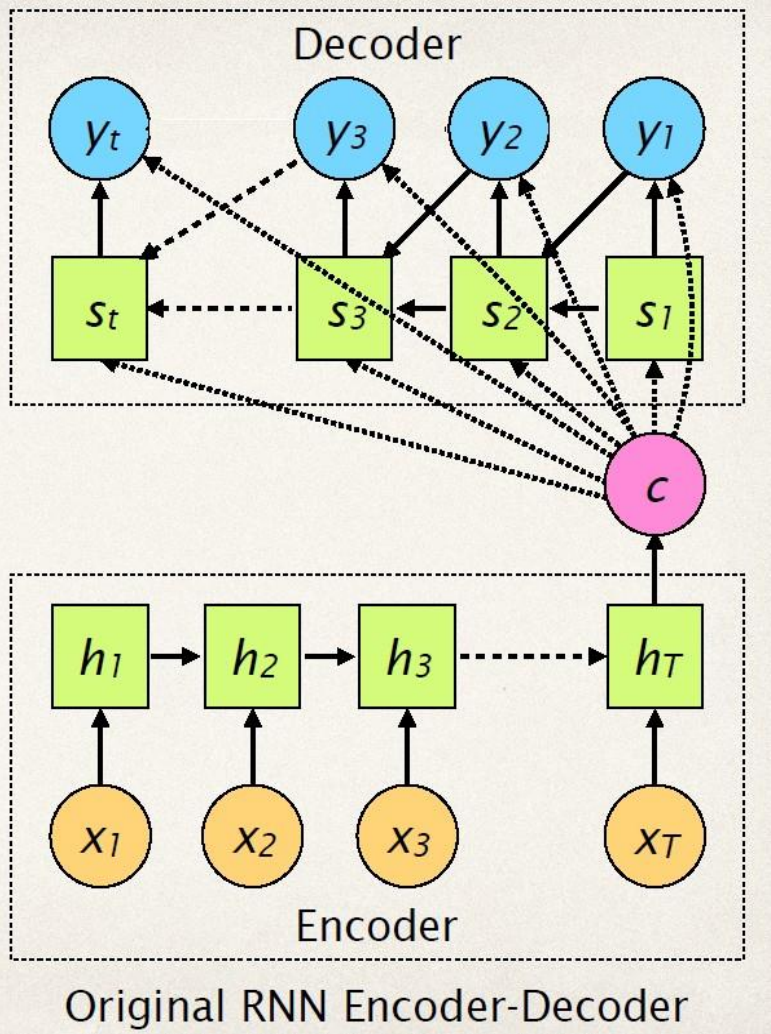
- The attention weight, $a_{t,j}$, is to be determined by assessing how compatible the query and the key are.

- Computed from the input sequence

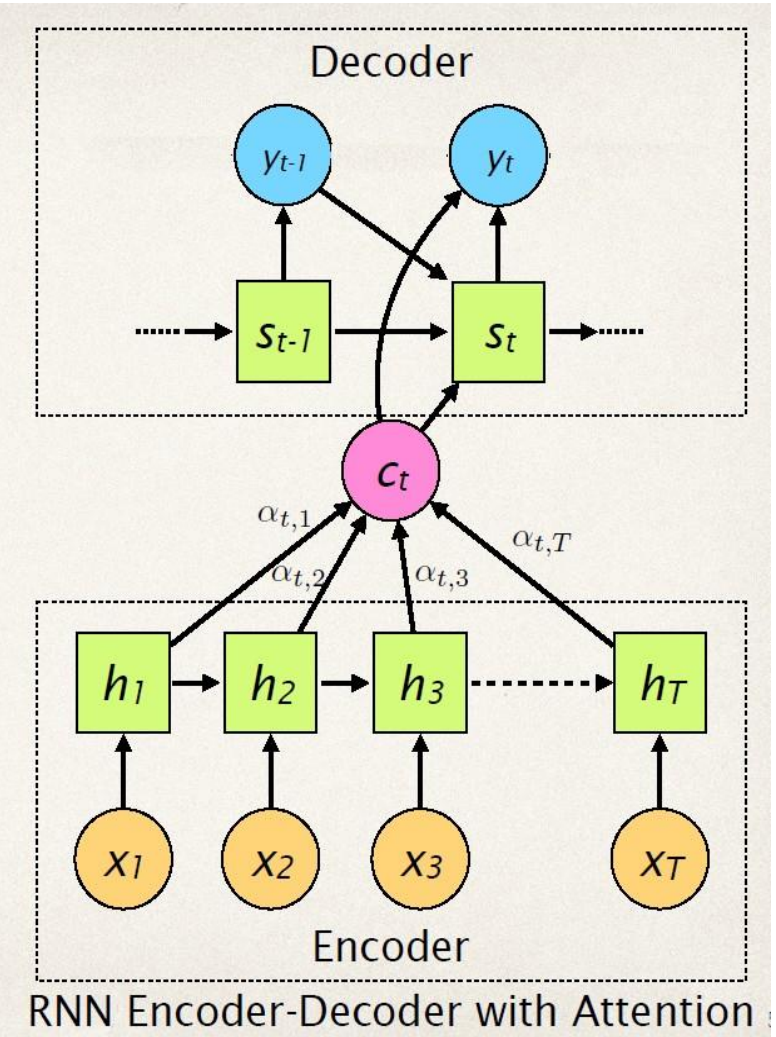
Recall: $C_t = \sum_j a_{t,j} h_j$



RNN vs. Attention



Original RNN Encoder-Decoder



RNN Encoder-Decoder with Attention

The Softmax—Similarity to Probability

- Suppose we have a vector of scores: [2.0, 1.0, 0.5]. To apply softmax:
  - e^2.0 ≈ 7.389
  - e^1.0 ≈ 2.718
  - e^0.5 ≈ 1.649
  - Sum: 7.389 + 2.718 + 1.649 = 11.756
- Finally, the values:
  - 7.389 / 11.756 ≈ 0.628
  - 2.718 / 11.756 ≈ 0.231
  - 1.649 / 11.756 ≈ 0.140

Softmax activation function

$$\frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Softmax in LLM

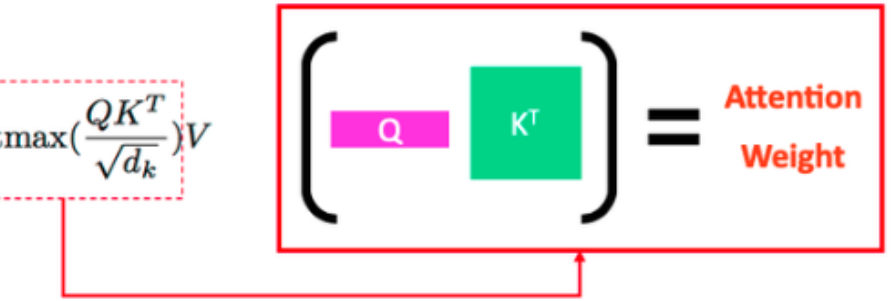
$$\frac{e^{\frac{y_i}{T}}}{\sum_{k=1}^{n} e^{\frac{y_k}{T}}}$$

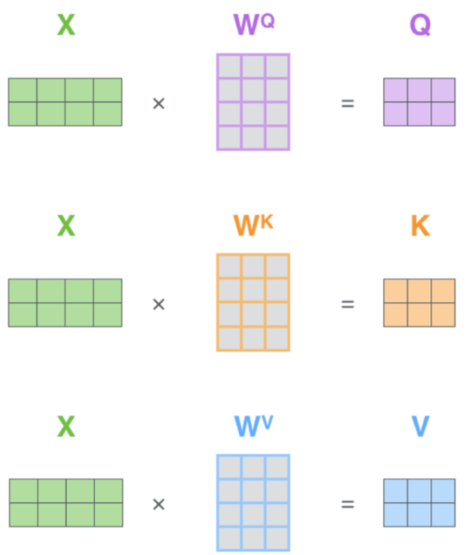


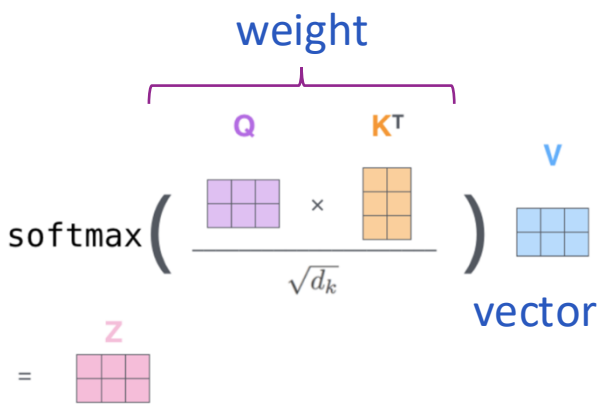
Computing attention weight

- The Q, K and V are formed using **the same input X** with different weight, and a softmax is used to compute **how much attention is put on V**.
- All the weight can be trained using some targets
- It is followed by a feed forward network

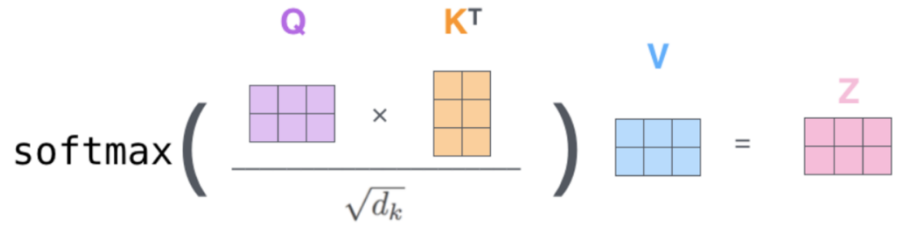
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$







Computing attention weight



Step 1

Without cache

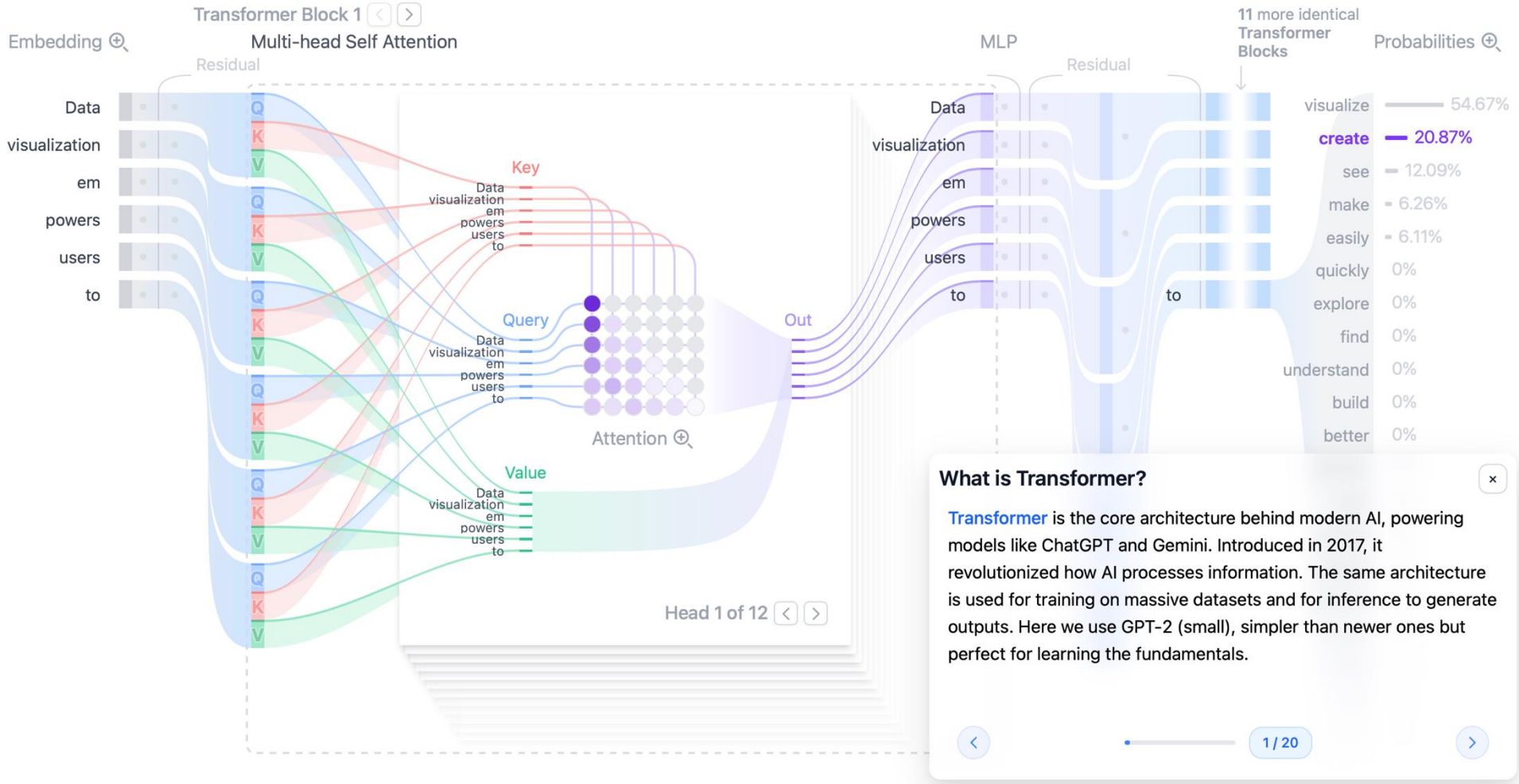
Q		K^T		QK^T		V		Attention
Query Token 1	x	Key Token 1	=	$Q_1.K_1$	x	Value Token 1	=	Token 1
(1, emb_size)		(emb_size, 1)		(1, 1)		(1, emb_size)		(1, emb_size)







Demo: transformer

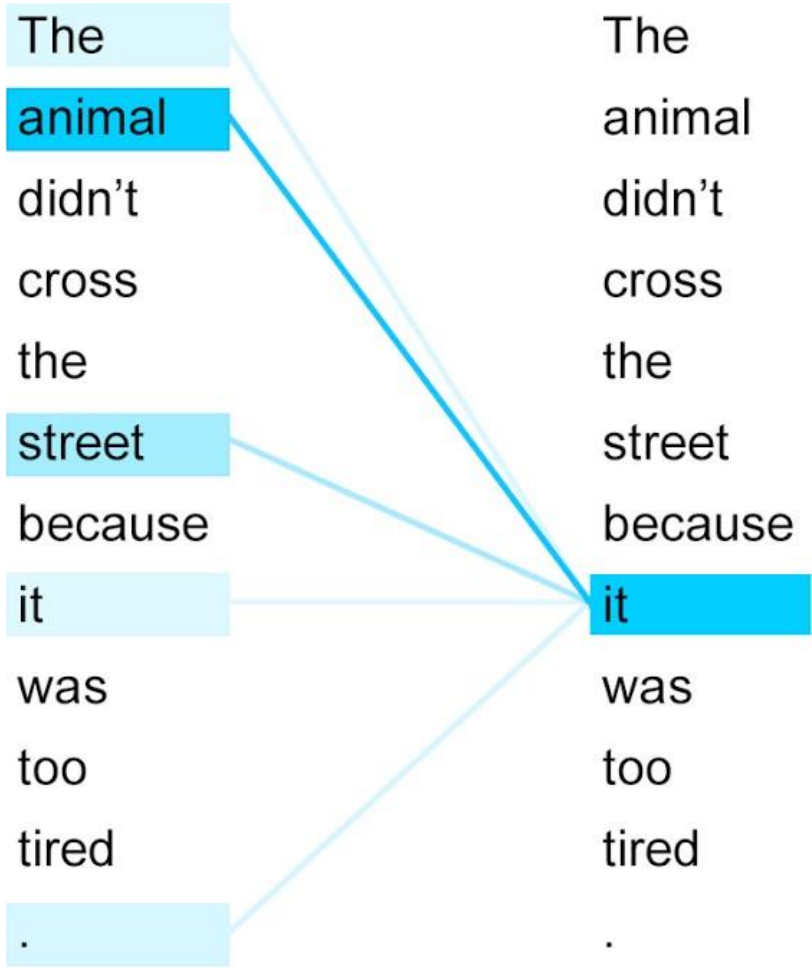


https://poloclub.github.io/transformer-explainer/

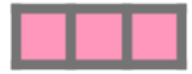
Self-attention Layer

- "improve" the representation using other tokens
- Example:
  - What does "it" refer to? The street or the animal?
  - Self-attention: associate "it" with "animal"

- calculates the attention weight of a Query token (token : something like a word) and a Key token and multiplies the Value associated with each Key



Self-attention Layer

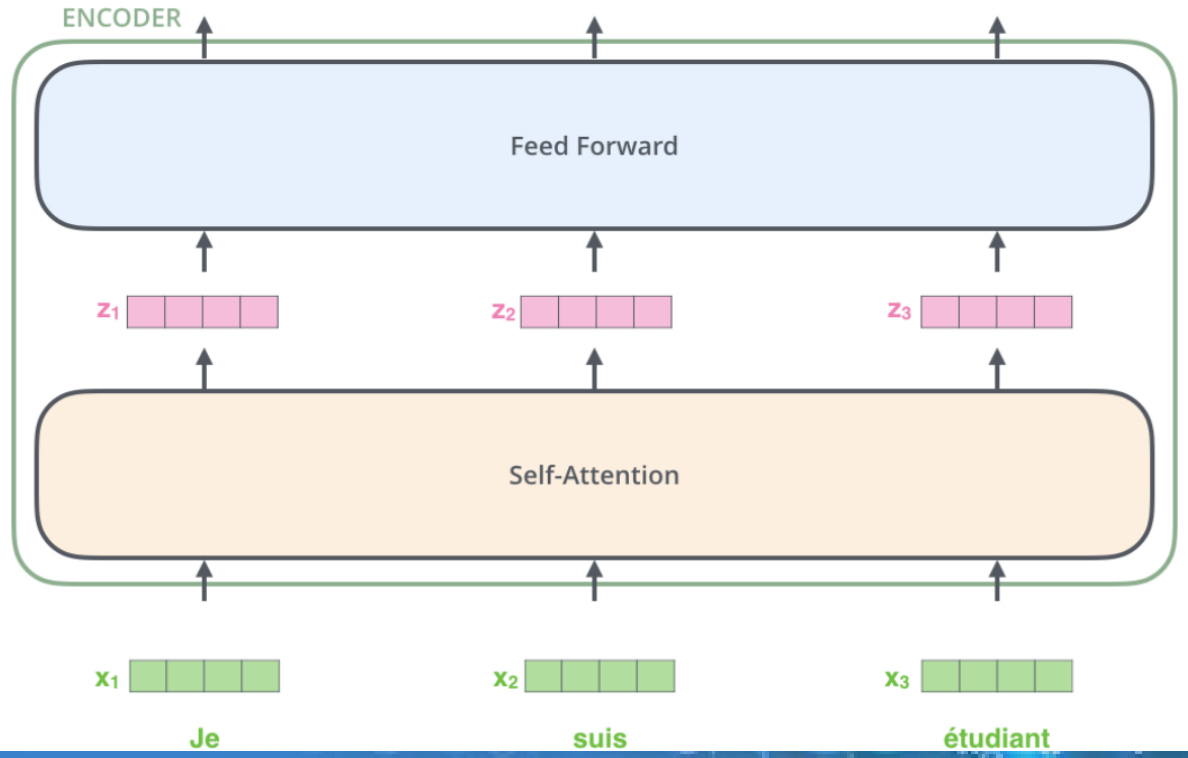
Word	Value vector	Score	Value X Score
<s>		0.001	
a		0.3	
robot		0.5	
must		0.002	
obey		0.001	
the		0.0003	
orders		0.005	
given		0.002	
it		0.19	
		Sum:	



Self-attention Layer

- The input sequence to the encoder of Transformer are the embedding vectors

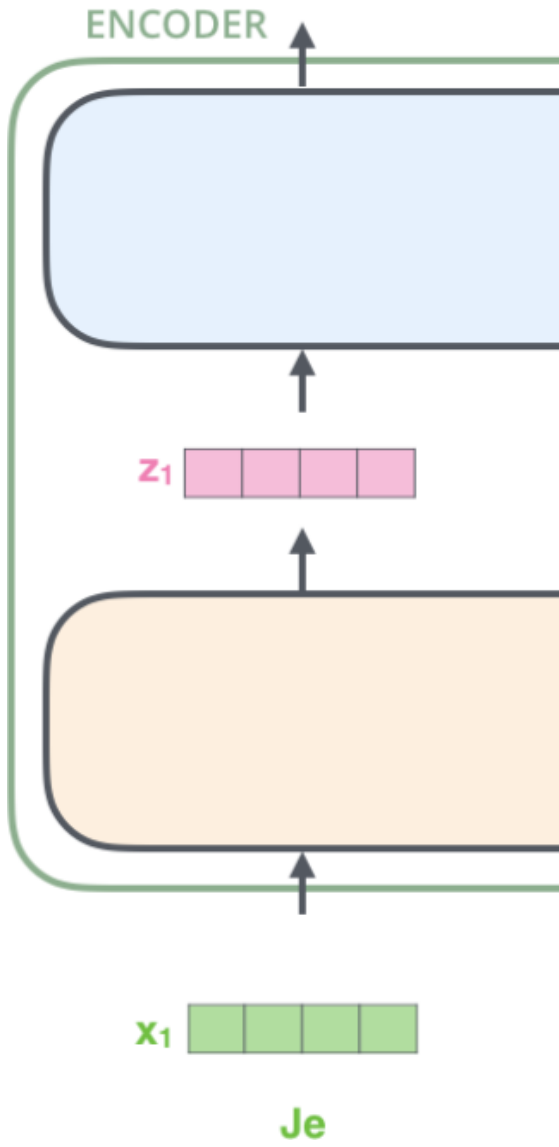
- The **self-attention layer** processes the embedding vectors in parallel to obtain new representations of the tokens.





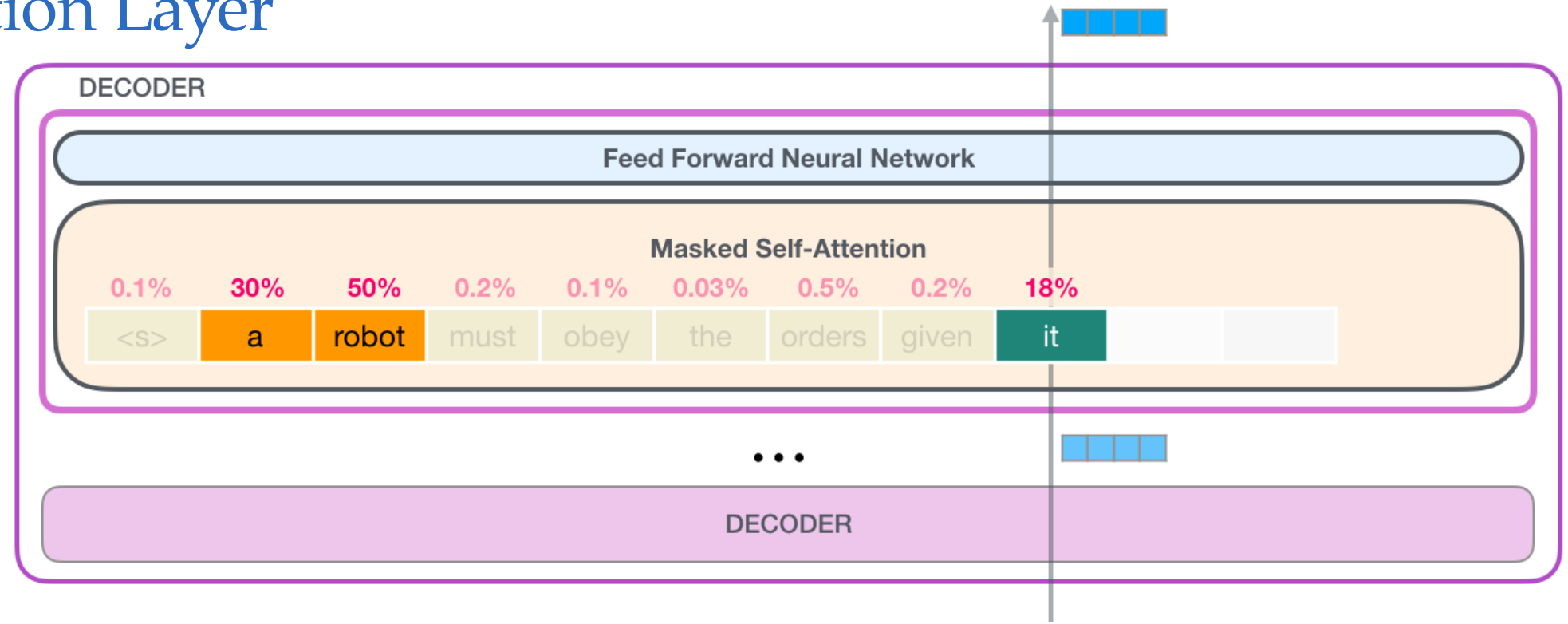
Self-attention Layer

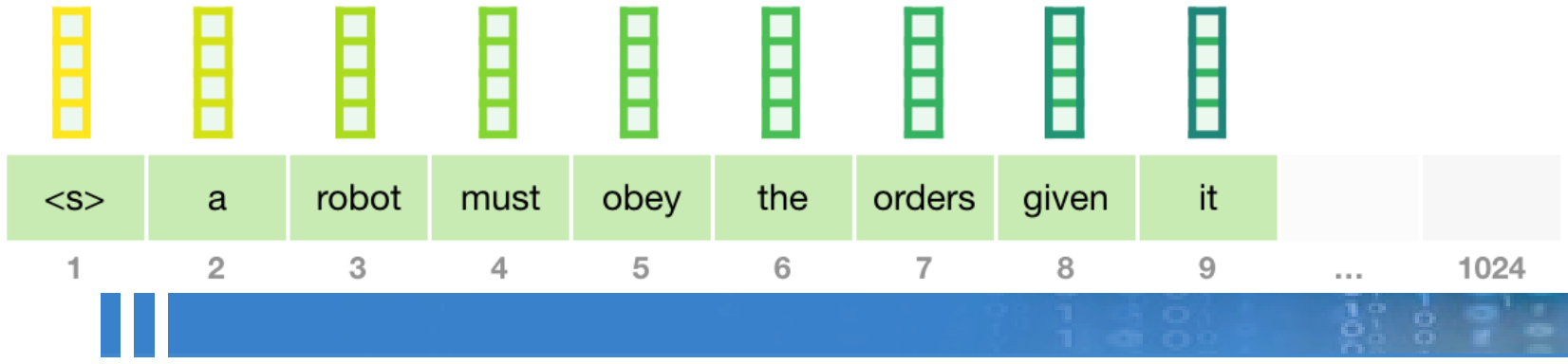
- Project x_i and x_j to get query, key, and value:
  - $q_i = x_i W^Q$: **query** vector of dimension d_k.
  - $k_j = x_j W^K$: **key** vector of dimension d_k.
  - $v_j = x_j W^v$: **value** vector of dimension d_v.
- Compute attention weights:
  - Dot-product attention: $\alpha_{i,j} \leftarrow q_i k_j^\top$.
  - **Scaled dot-product attention**: $\alpha_{i,j} \leftarrow \frac{\alpha_{i,j}}{\sqrt{d_k}}$.
  - Apply softmax: $\alpha_{i,j} \leftarrow \frac{e^{\alpha_{i,j}}}{\sum_j e^{\alpha_{i,j}}}$
- Obtain z_i (a vector of dimension d_v) by: $z_i = \sum_j \alpha_{i,j} v_j$





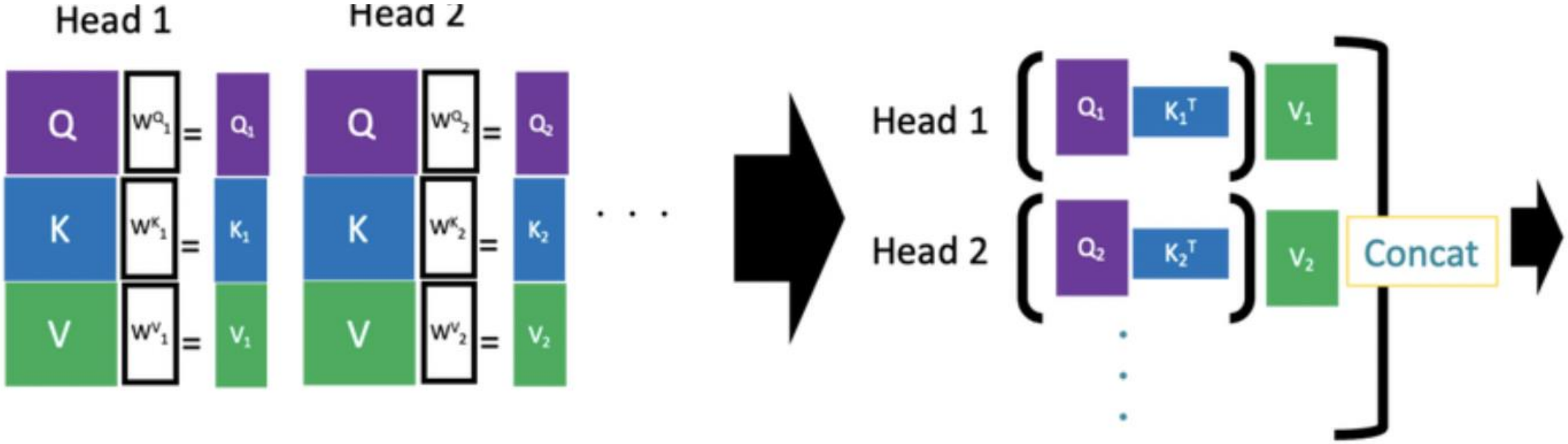
Self-attention Layer





Multi-head Attention

- The blocks take inputs to form multiple attention so to make sure that it can capture most information using different attentions



Transformer (a neural network architecture)

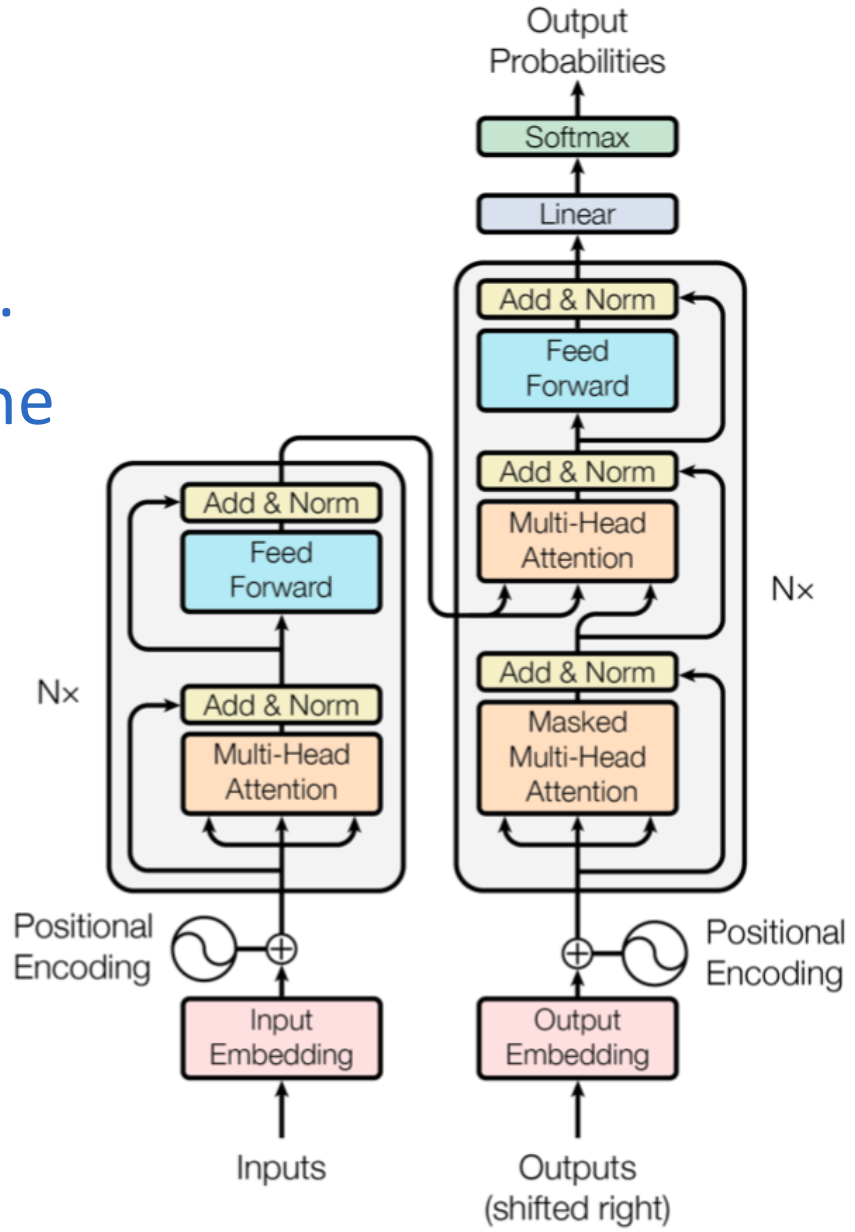
- Using self-attention to capture relationships between different parts of the input sequence
- Multi-head attention is used to capture multifaceted relationships

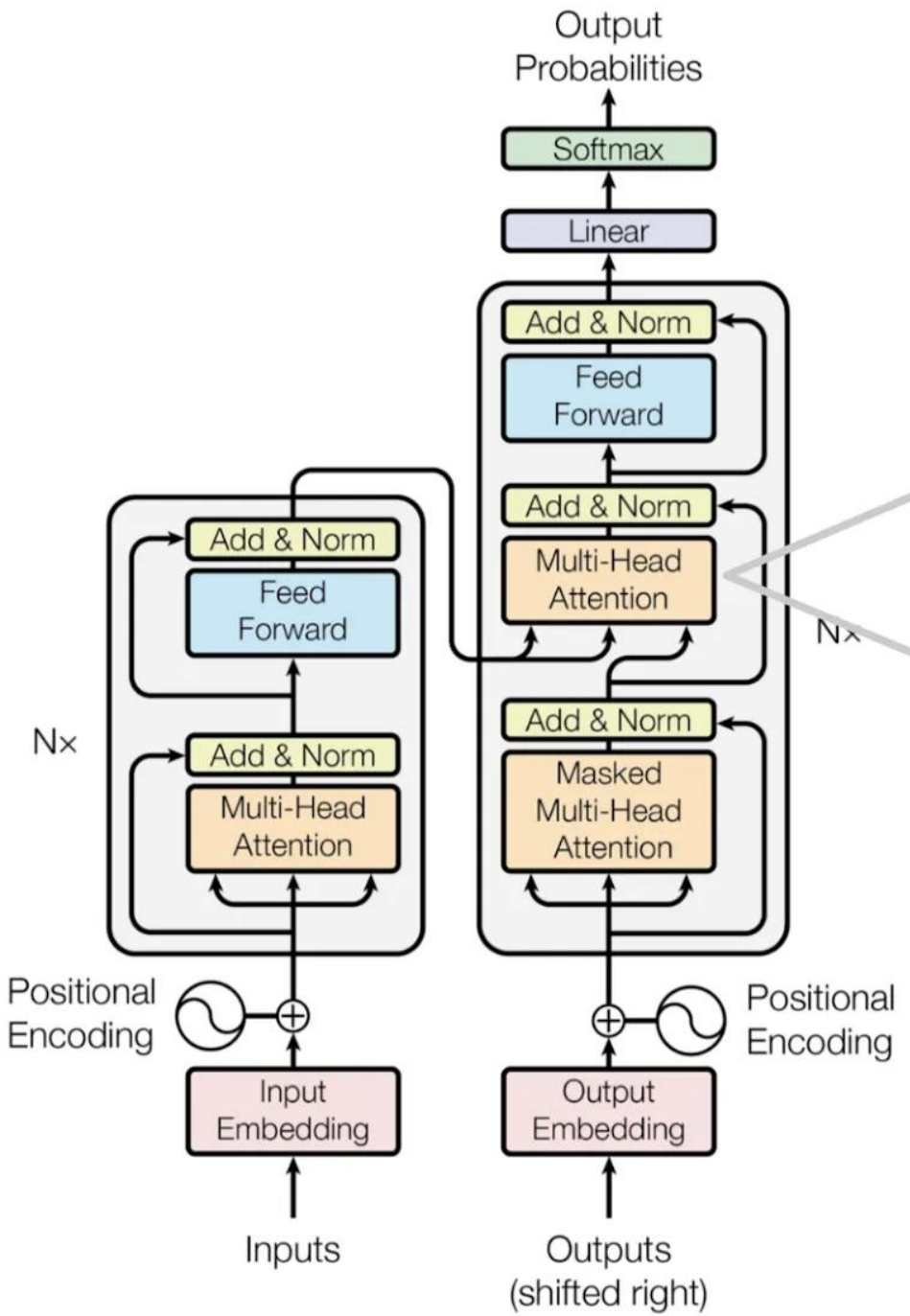
- **How it Works**:
  - Text is converted into numerical representations called tokens
  - Each token is converted into a vector via a word embedding table
  - The model identifies the relevance and relationship between the words and uses this knowledge to generate an output

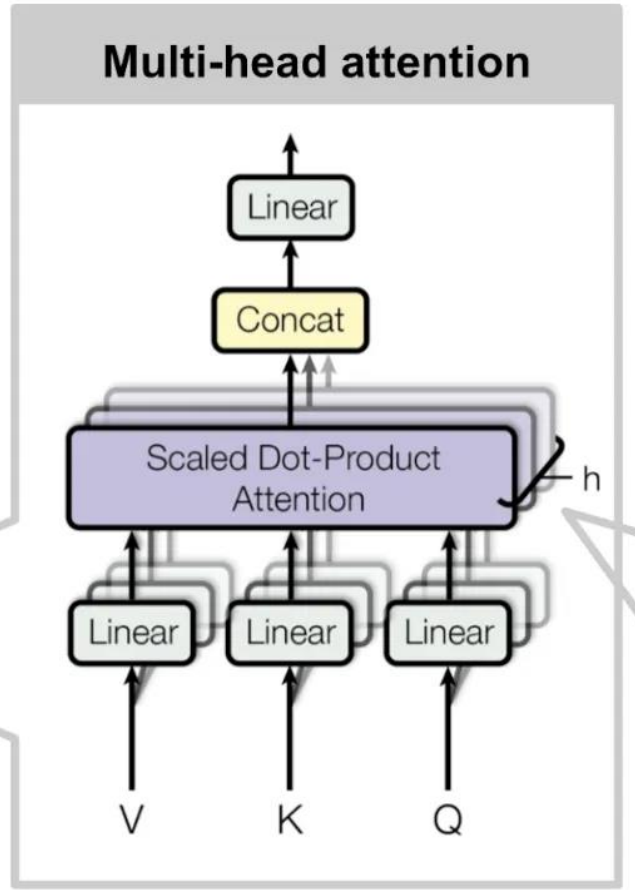


Encoder-decoder Structure

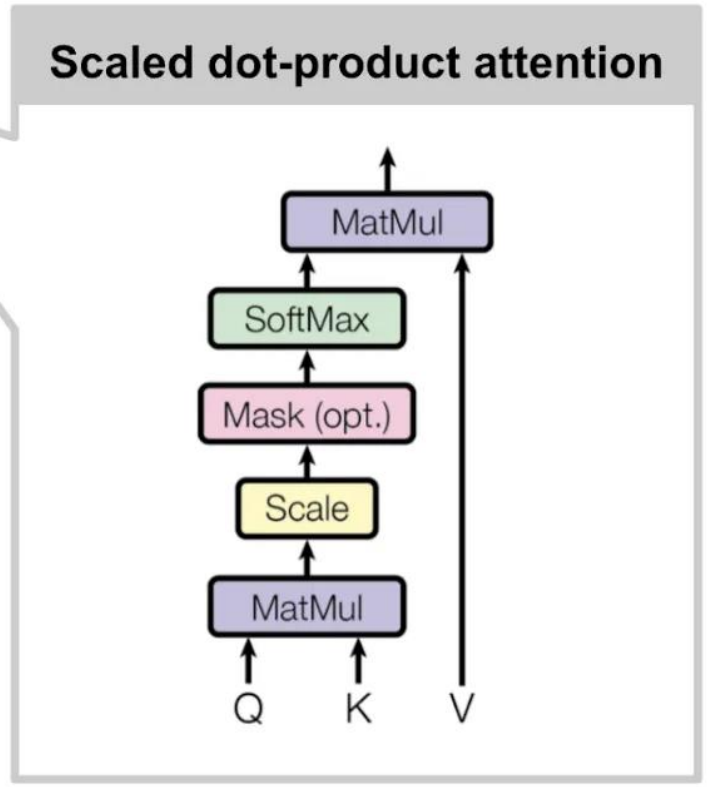
- Input & output: 2 text sequent in diff languages.
- The decoder will take the previous outputs as the input, as well as the output of encoders to generate an output.
- The structure is similar to an encoder.
- Positional vector is added to represent the position. No weight share with the encoder.





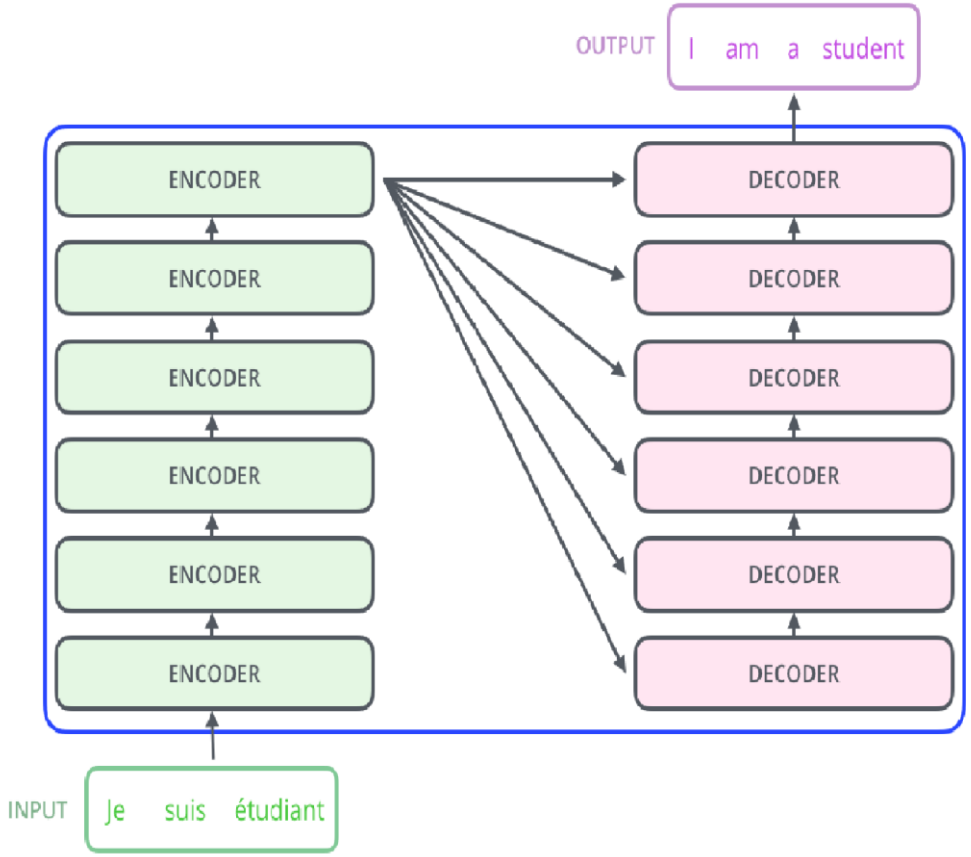


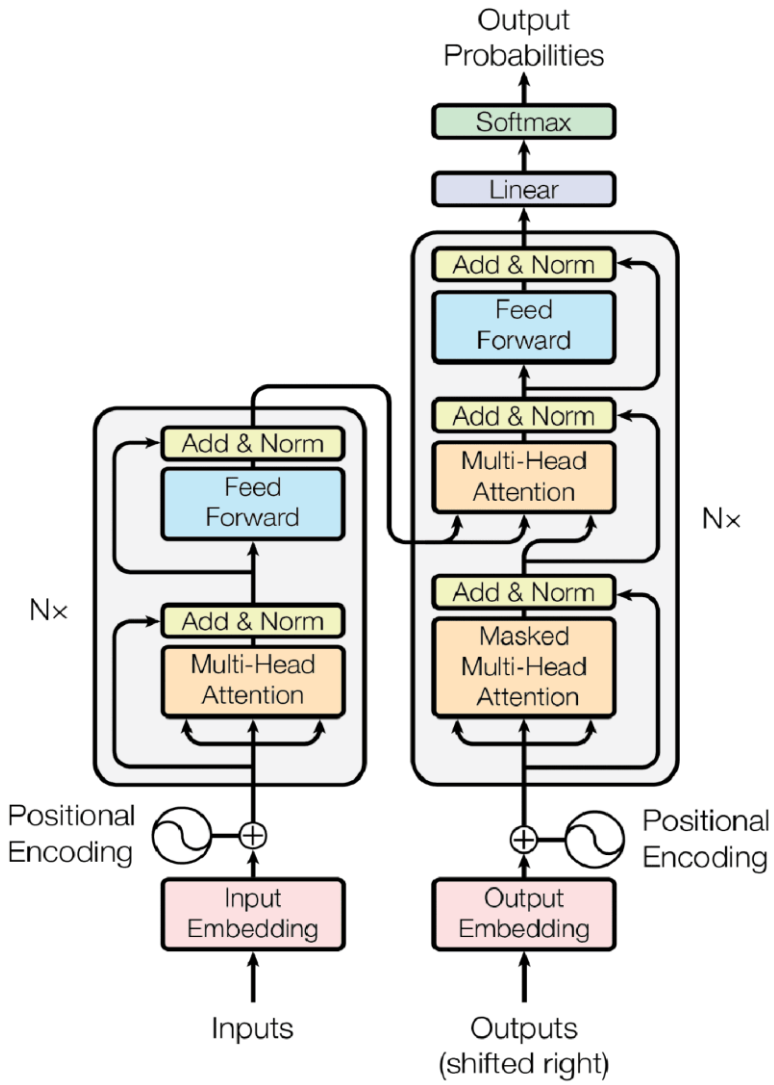
Zoom-In!



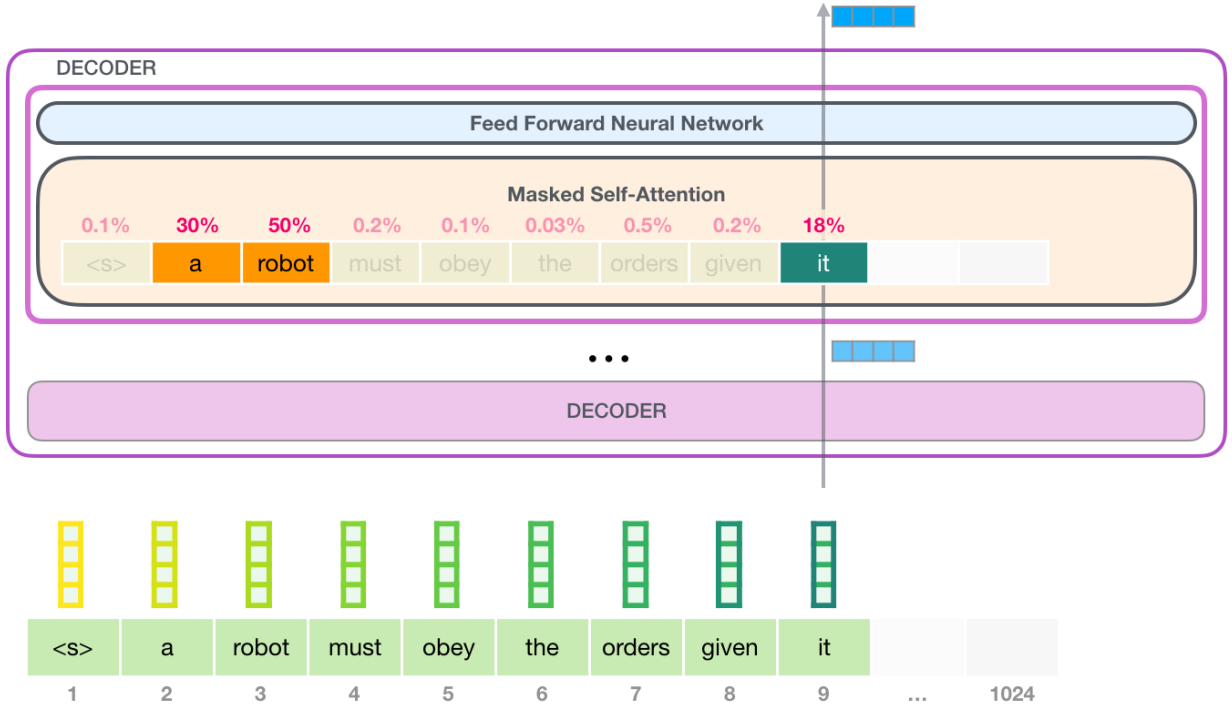
Zoom-In!

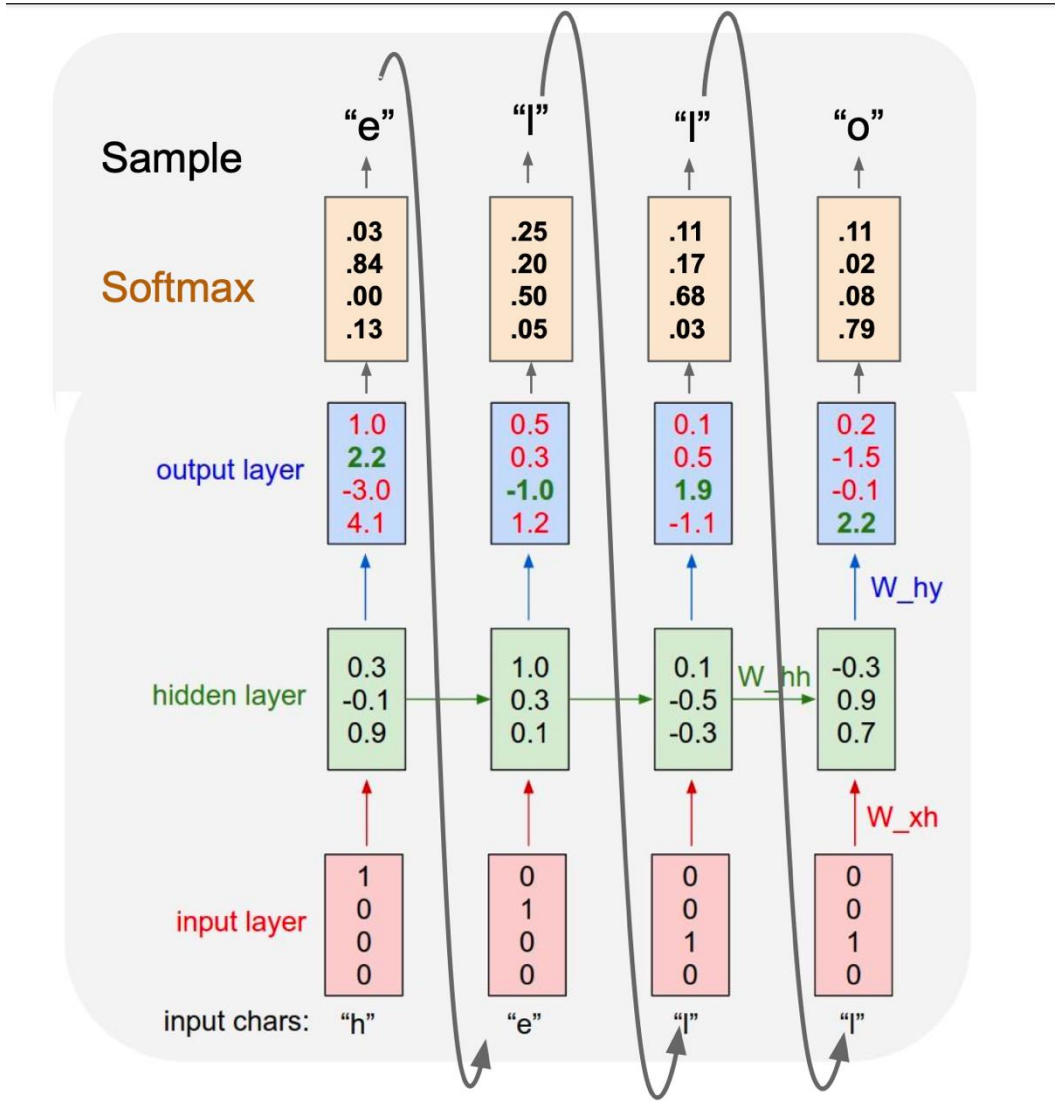
Encoder-decoder Structure





Transformer vs. LSTM





Transformer vs. LSTM

Aspect	Transformer	LSTM
Architecture	Based on self-attention and feed-forward networks	Based on recurrent connections and hidden states
Long-Term Dependencies	Capable	Struggles
Positional Information	Uses positional encoding	Assumes sequential order
Parallelization	Highly parallelizable	Less parallelizable
Training Time	Longer	Faster



PyTorch-based Transformer

```python
class MultiHeadSelfAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super(MultiHeadSelfAttention, self).__init__()
        self.d_model = d_model
        self.n_heads = n_heads
        self.head_dim = d_model // n_heads

        assert (
            self.head_dim * n_heads == d_model
        ), "d_model must be divisible by n_heads"

        self.values = nn.Linear(d_model, d_model, bias=False)
        self.keys = nn.Linear(d_model, d_model, bias=False)
        self.queries = nn.Linear(d_model, d_model, bias=False)
        self.fc_out = nn.Linear(d_model, d_model)

    def forward(self, x):
        N, seq_length, _ = x.shape
        values = self.values(x)
        keys = self.keys(x)
        queries = self.queries(x)

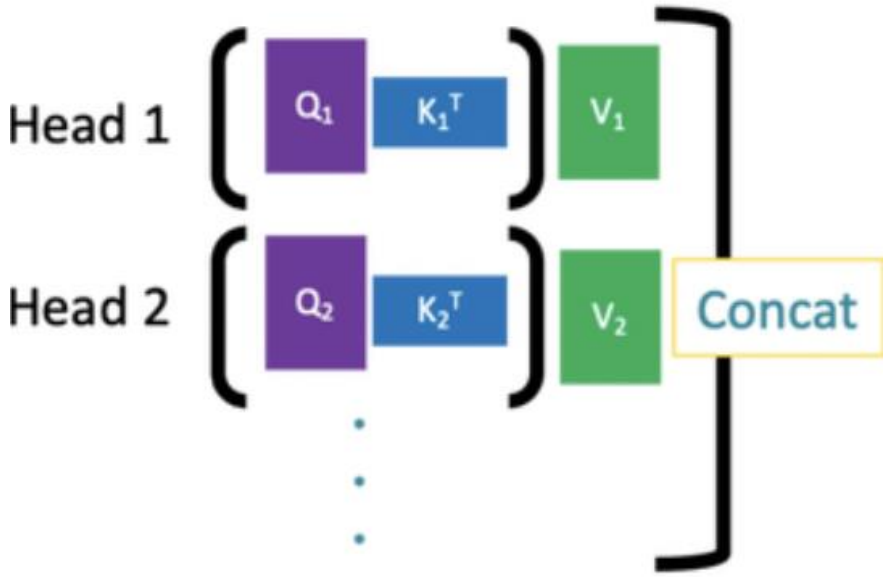
        values = values.view(N, seq_length, self.n_heads, self.head_dim)
        keys = keys.view(N, seq_length, self.n_heads, self.head_dim)
        queries = queries.view(N, seq_length, self.n_heads, self.head_dim)

        values = values.permute(0, 2, 1, 3)
        keys = keys.permute(0, 2, 1, 3)
        queries = queries.permute(0, 2, 1, 3)

        energy = torch.einsum("nqhd,nkhd->nqhk", [queries, keys])
        attention = F.softmax(energy / (self.head_dim ** (1 / 2)), dim=3)

        out = torch.einsum("nqhk,nvhd->nqhd", [attention, values])
        out = out.permute(0, 2, 1, 3).contiguous().view(N, seq_length, self.d_model)

        return self.fc_out(out)
```



PyTorch-based Transformer (Cts)

```python
class FeedForward(nn.Module):
    def __init__(self, d_model, hidden_dim):
        super(FeedForward, self).__init__()
        self.fc1 = nn.Linear(d_model, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, d_model)

    def forward(self, x):
        return self.fc2(F.relu(self.fc1(x)))


class TransformerBlock(nn.Module):
    def __init__(self, d_model, n_heads, hidden_dim, dropout):
        super(TransformerBlock, self).__init__()
        self.attention = MultiHeadSelfAttention(d_model, n_heads)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.feed_forward = FeedForward(d_model, hidden_dim)
        self.dropout1 = nn.Dropout(dropout)
        self.dropout2 = nn.Dropout(dropout)

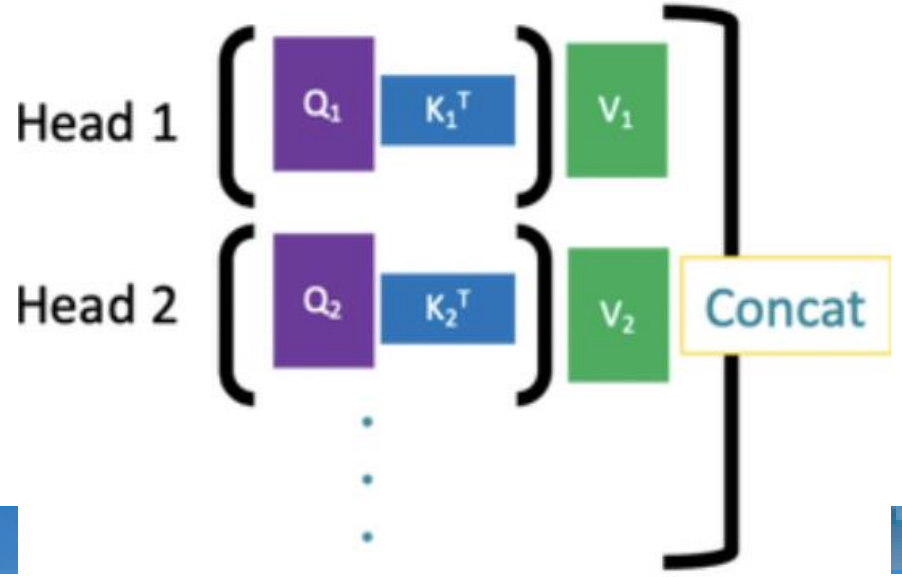
    def forward(self, x):
        attention = self.attention(x)
        x = self.norm1(attention + x)
        forward = self.feed_forward(x)
        x = self.norm2(forward + x)
        return x


class Transformer(nn.Module):
    def __init__(self, d_model, n_heads, hidden_dim, n_layers, dropout):
        super(Transformer, self).__init__()
        self.layers = nn.ModuleList(
            [TransformerBlock(d_model, n_heads, hidden_dim, dropout) for _ in range(n_layers)]
        )
        self.fc_out = nn.Linear(d_model, d_model)

    def forward(self, x):
        for layer in self.layers:
            x = layer(x)
        return self.fc_out(x)
```

```python
# Example usage
if __name__ == "__main__":
    d_model = 512  # Embedding size
    n_heads = 8    # Number of attention heads
    hidden_dim = 2048  # Feedforward hidden layer size
    n_layers = 6   # Number of transformer blocks
    dropout = 0.1  # Dropout rate

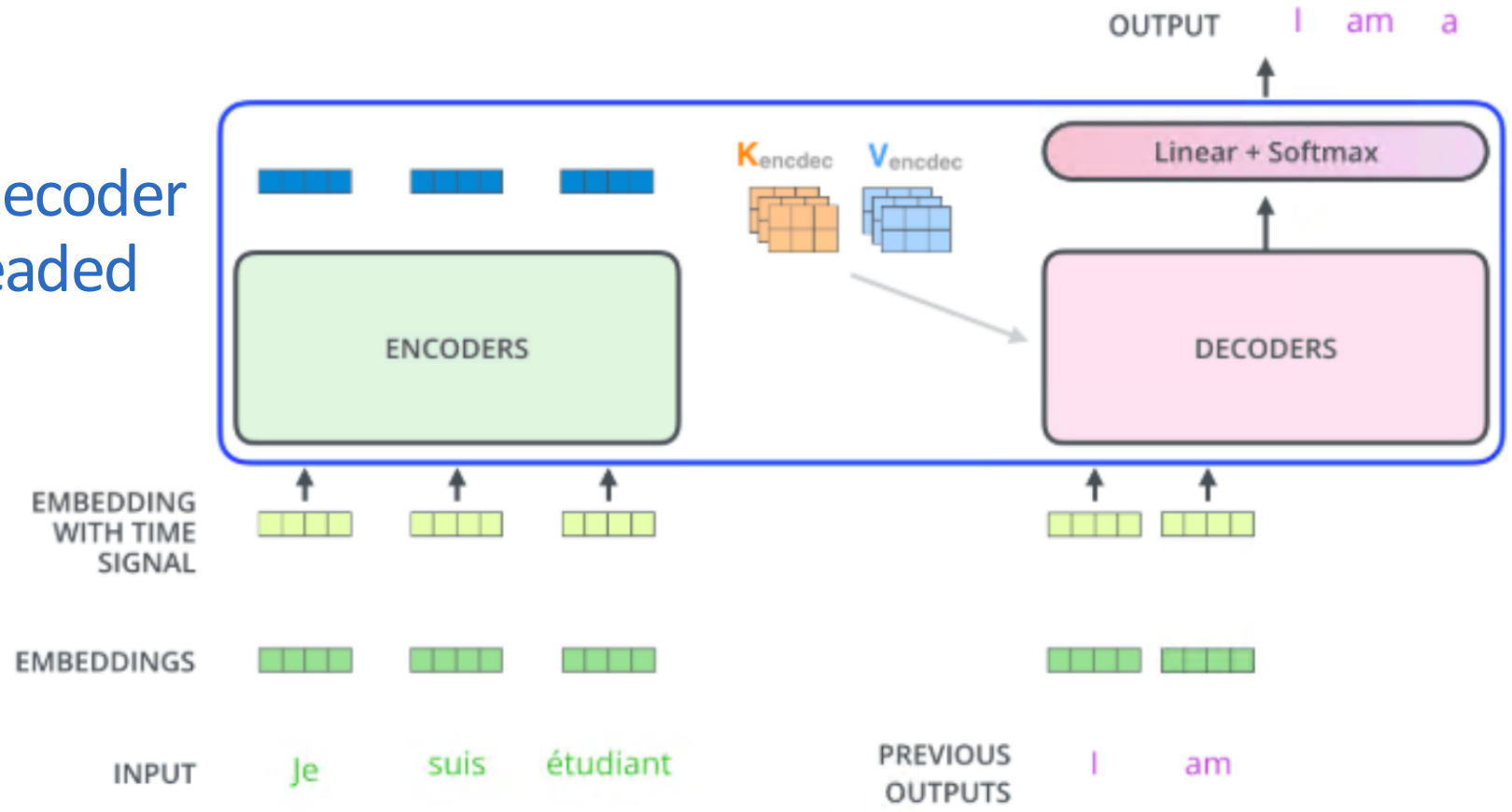
    model = Transformer(d_model, n_heads, hidden_dim, n_layers, dropout)
    x = torch.rand(10, 20, d_model)  # Batch size of 10, sequence length of 20
    out = model(x)
    print(out.shape)  # Should output: torch.Size([10, 20, 512])
```



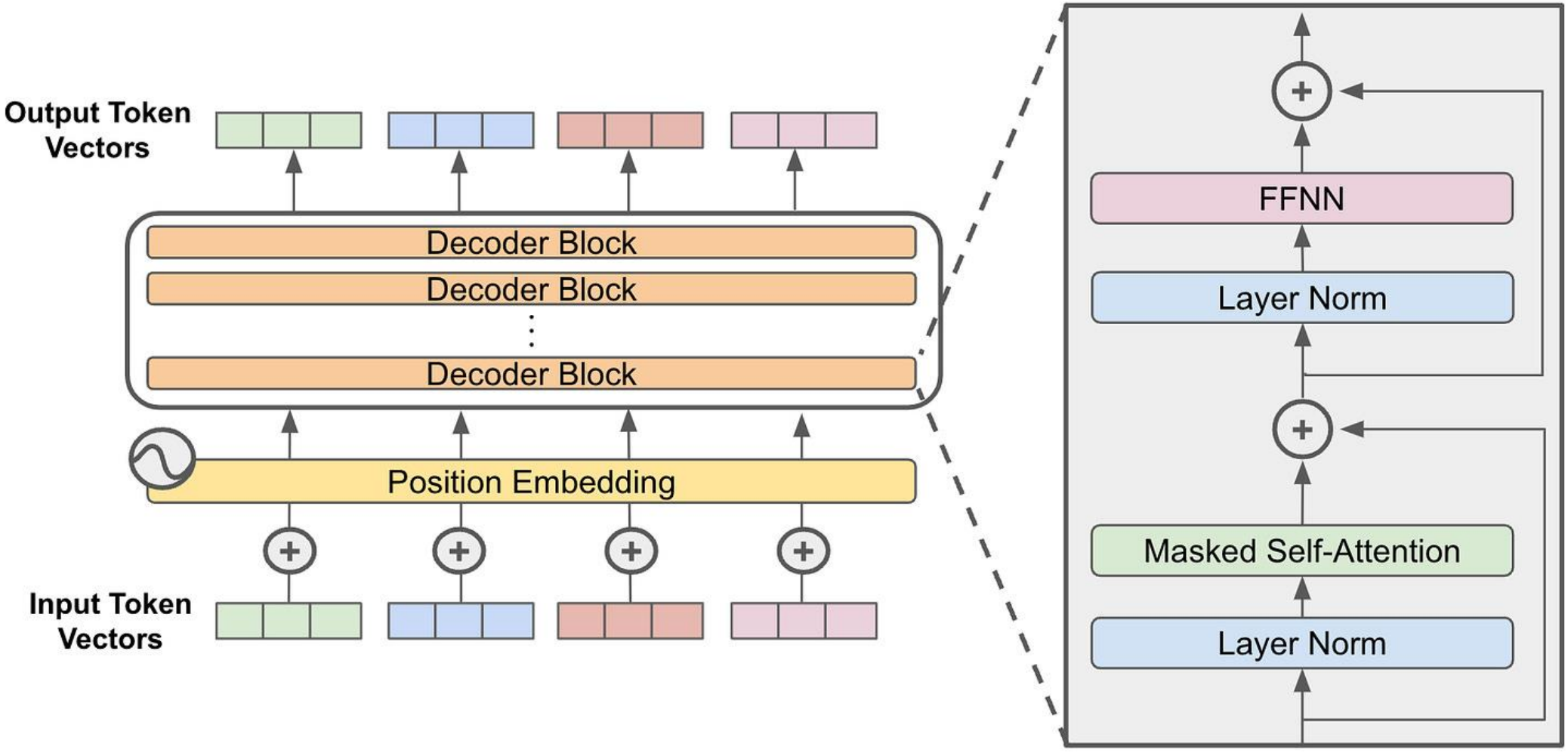
Conclusions on Transformer

- first sequence transduction model based entirely on attention

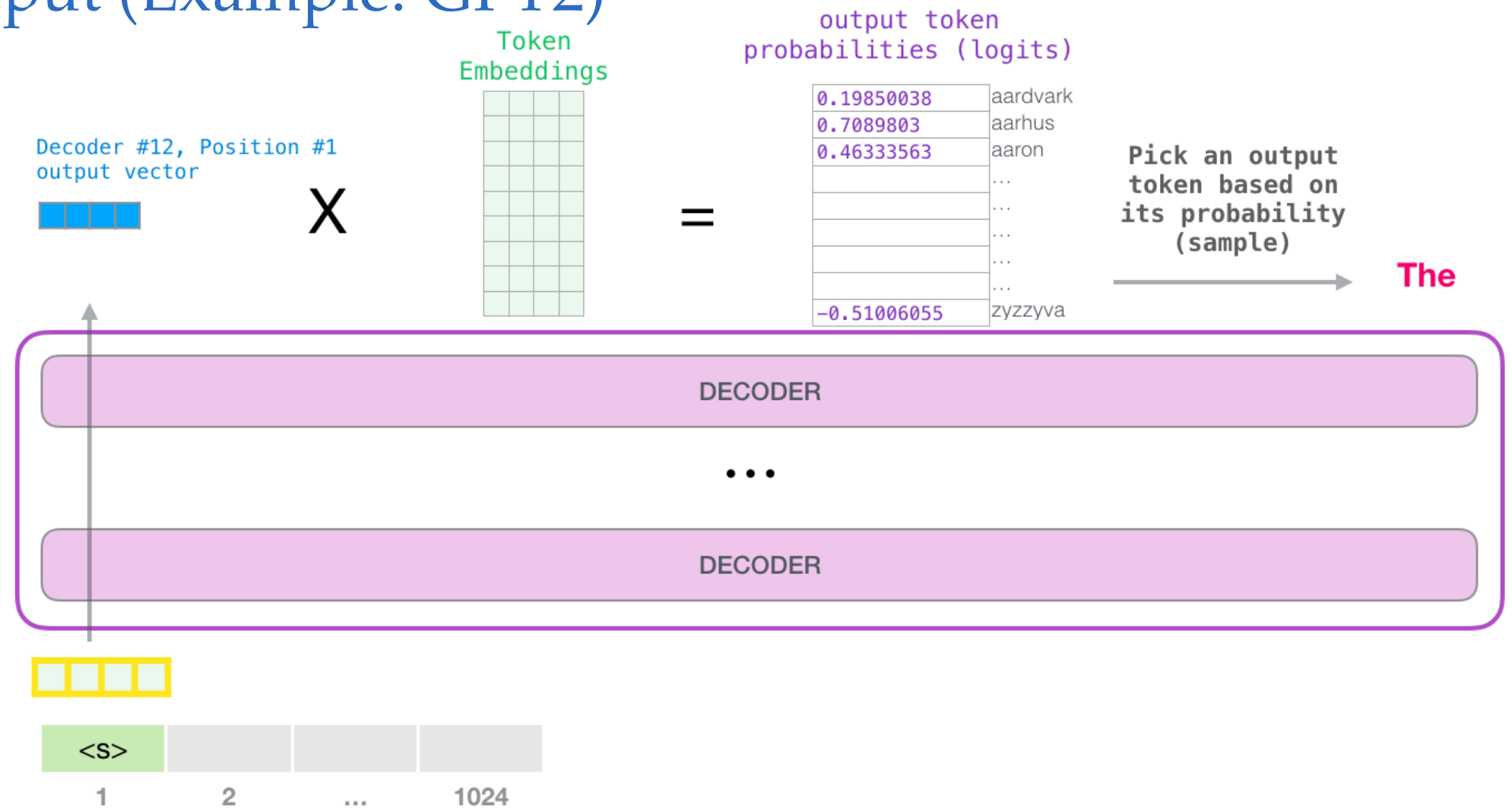
- replace RNN in encoder-decoder architectures with multi-headed self-attention.



Decoder Only Structure (e.g., GPT)



Model Output (Example: GPT2)





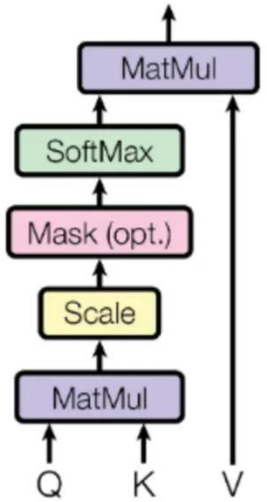
Agenda

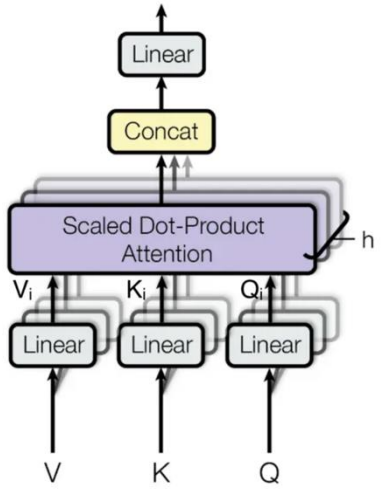
- RNN & LSTM
- Transformer
- KV caches & Model efficiency



Recall: Attention

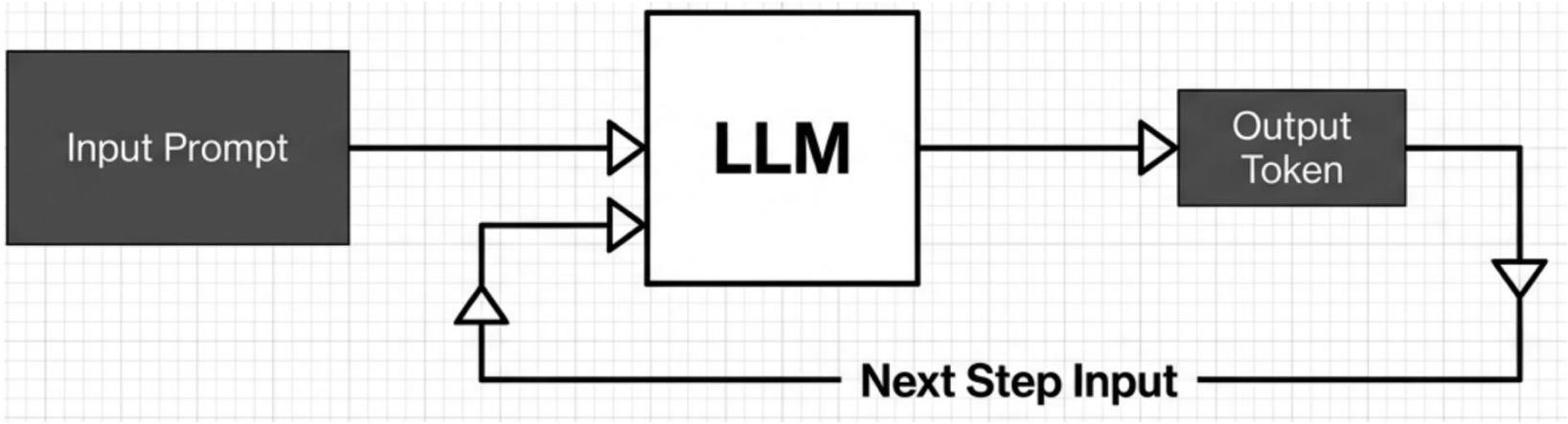
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$





KV caches

- To generate token Tn, the model must evaluate token Tn-1 along with all preceding tokens. The mathematical state of every past token influences the next prediction.

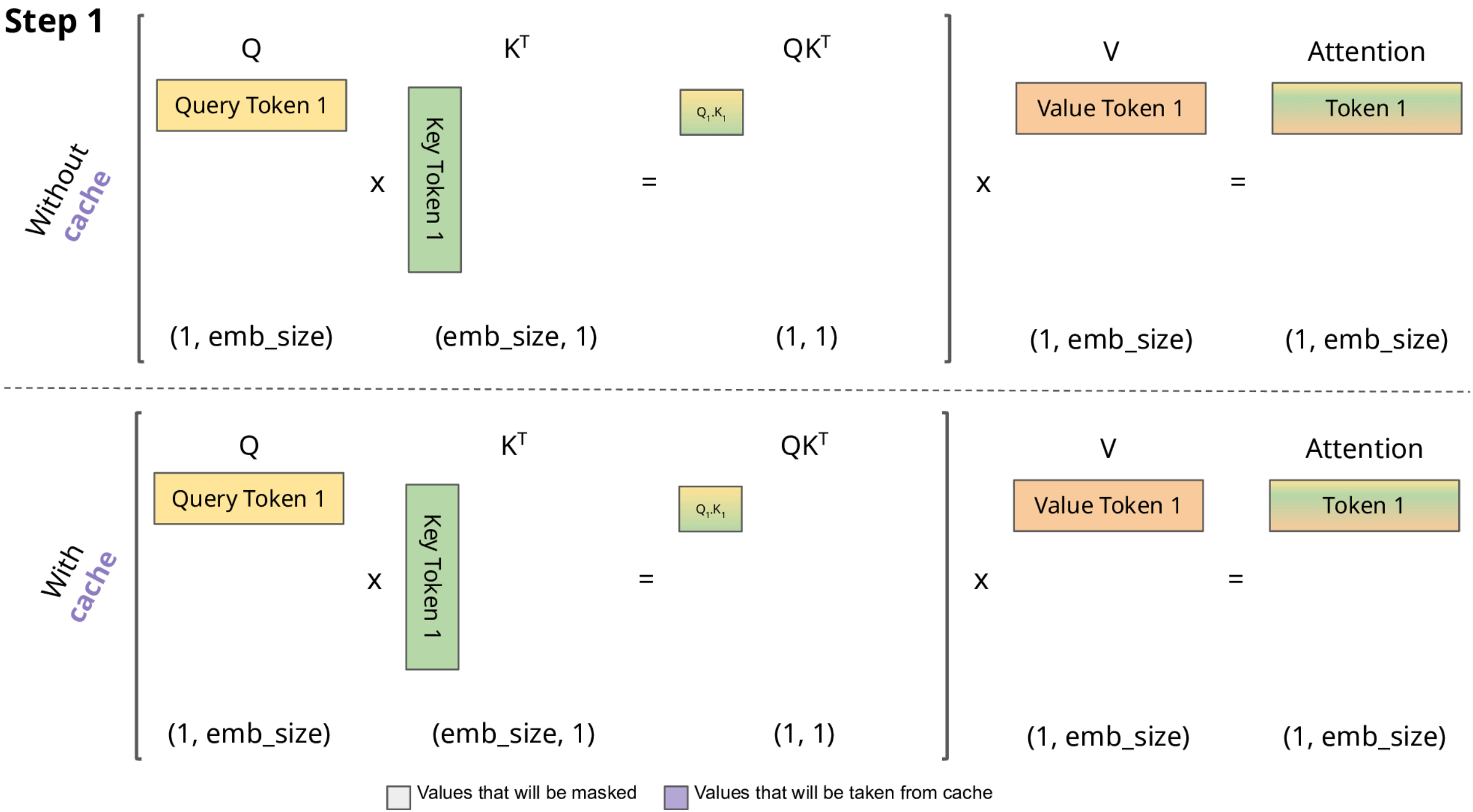


- Key Benefit: Faster inference without sacrificing accuracy.

Transformer-based: KV Cache

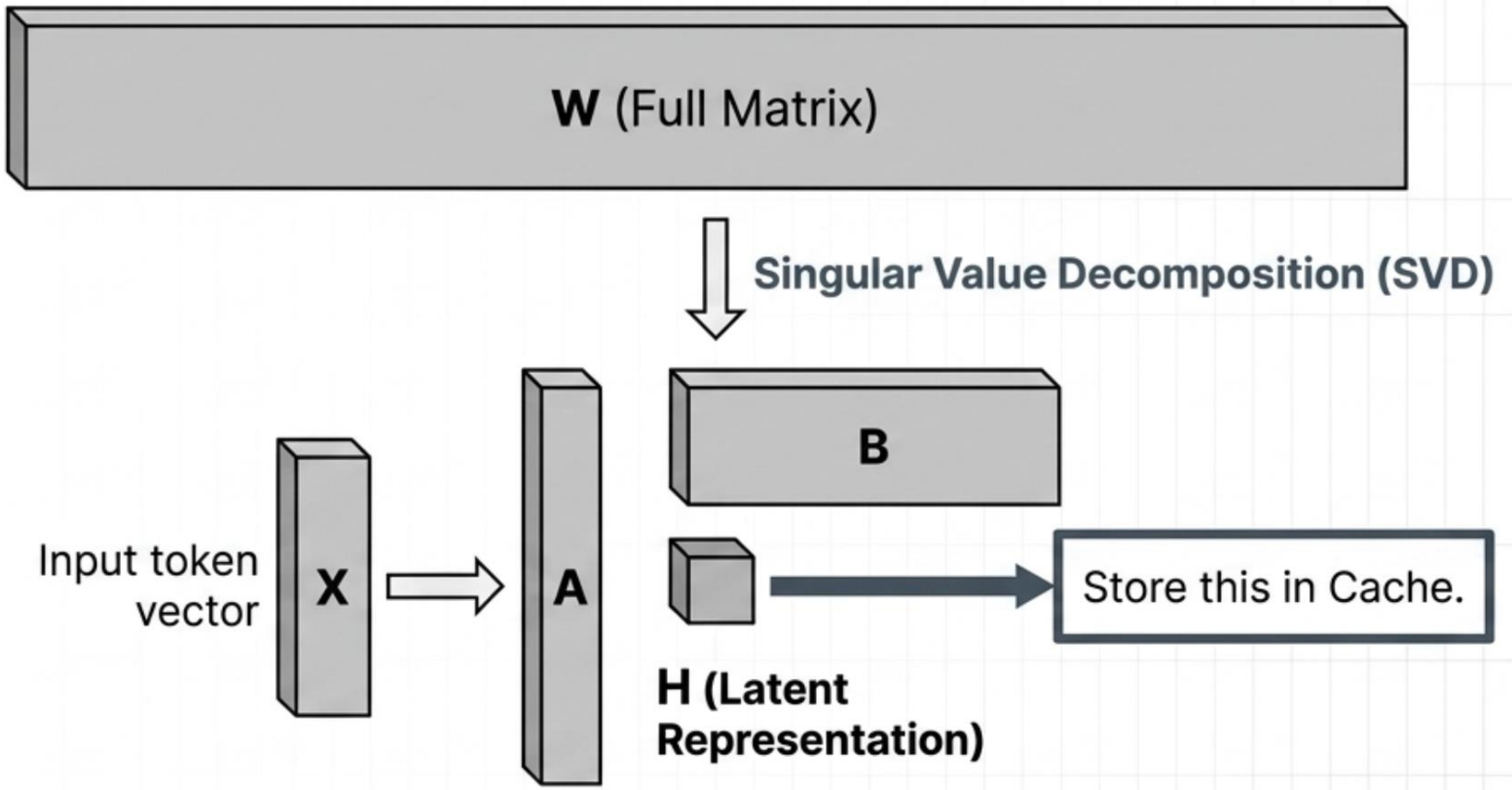
- Efficiency: Eliminates the need to recompute key and value vectors for all tokens in every step.
- Speed: Accelerates inference, especially for long sequences.
- Memory Trade-off: Trades computational savings for additional memory to store cached vectors.
- Commonly applied in large language models (LLMs) for tasks like text generation.




Step 1
Without cache
Q
K^T
QK^T
V
Attention
Query Token 1
Key Token 1
Q1.K1
Value Token 1
Token 1
x
=
x
=
(1, emb_size)
(emb_size, 1)
(1, 1)
(1, emb_size)
(1, emb_size)
With cache
Q
K^T
QK^T
V
Attention
Query Token 1
Key Token 1
Q1.K1
Value Token 1
Token 1
x
=
x
=
(1, emb_size)
(emb_size, 1)
(1, 1)
(1, emb_size)
(1, emb_size)
Values that will be masked
Values that will be taken from cache

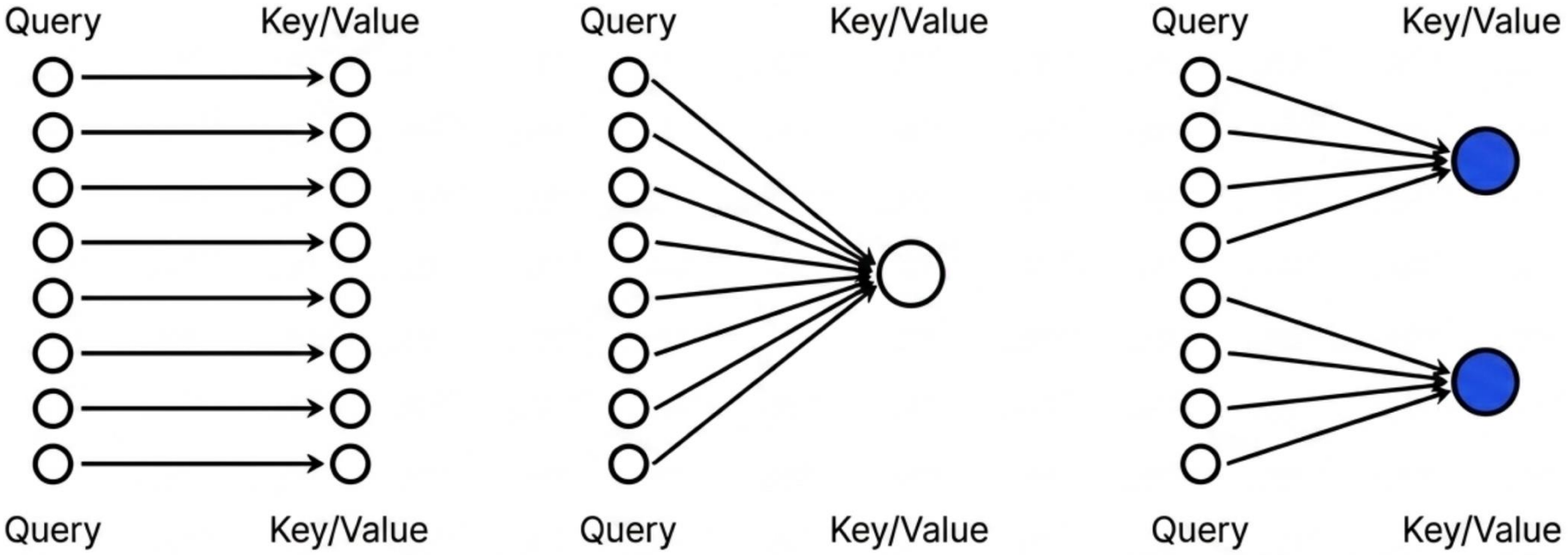
The KV cache

- Rather than storing the full Key/Value, the projection matrix (W) is decomposed into low-rank matrices $\mathbf{A}$ and $\mathbf{B}$
- Only the highly compressed latent representation is cached ($\mathbf{H} = \mathbf{X} \times \mathbf{A}$), and reconstructed only when needed (Full $K/V = \mathbf{H} \times \mathbf{B}$)



Grouped-Query Attention (GQA)

- Standard: 1 to1 mapping,
- MQA: The vector X is multiplied by **only 1** W_K and W_V but different W_{Q_i}
- GQA: shares a single KV head across grouped heads

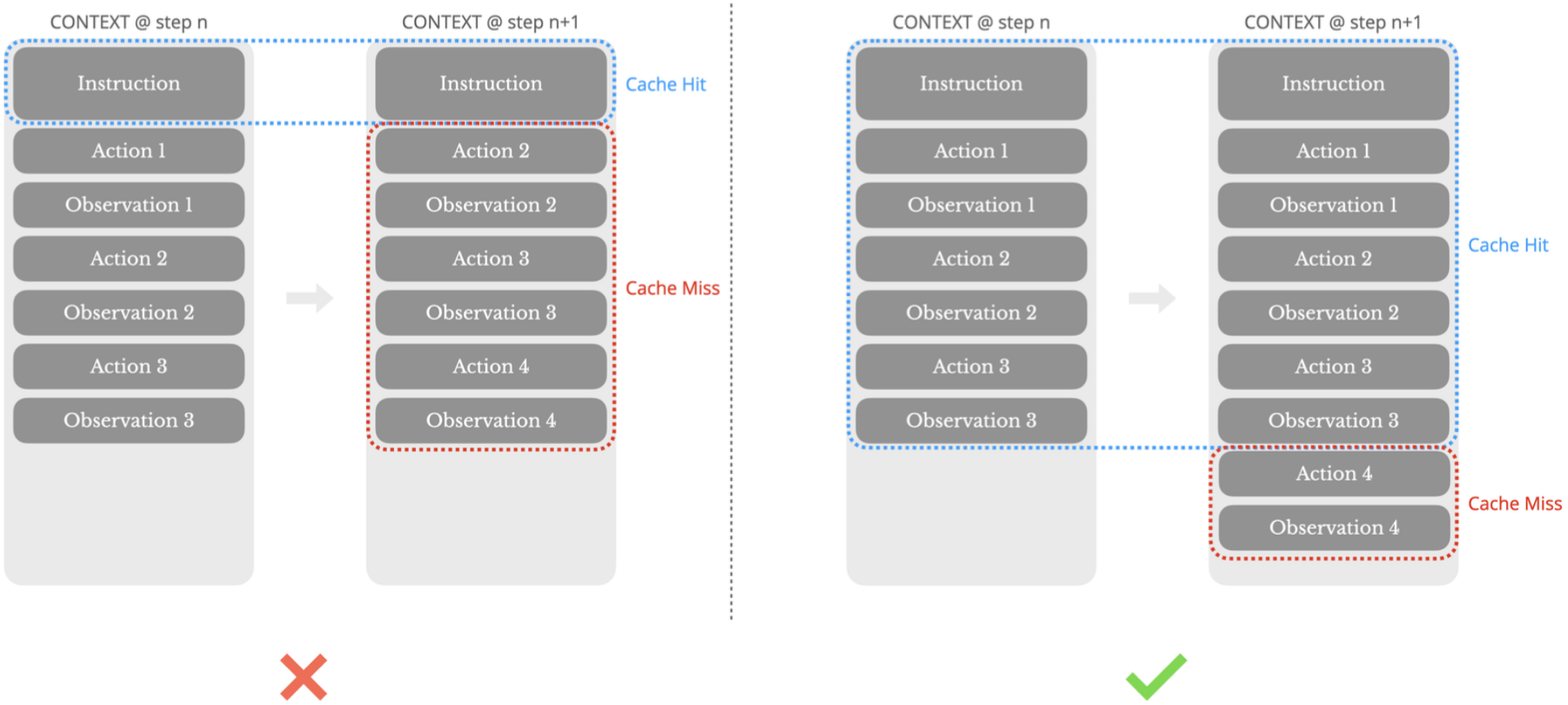


Standard MHA:
Massive Redundancy
$(2\ n_h\ d_h\ l)$

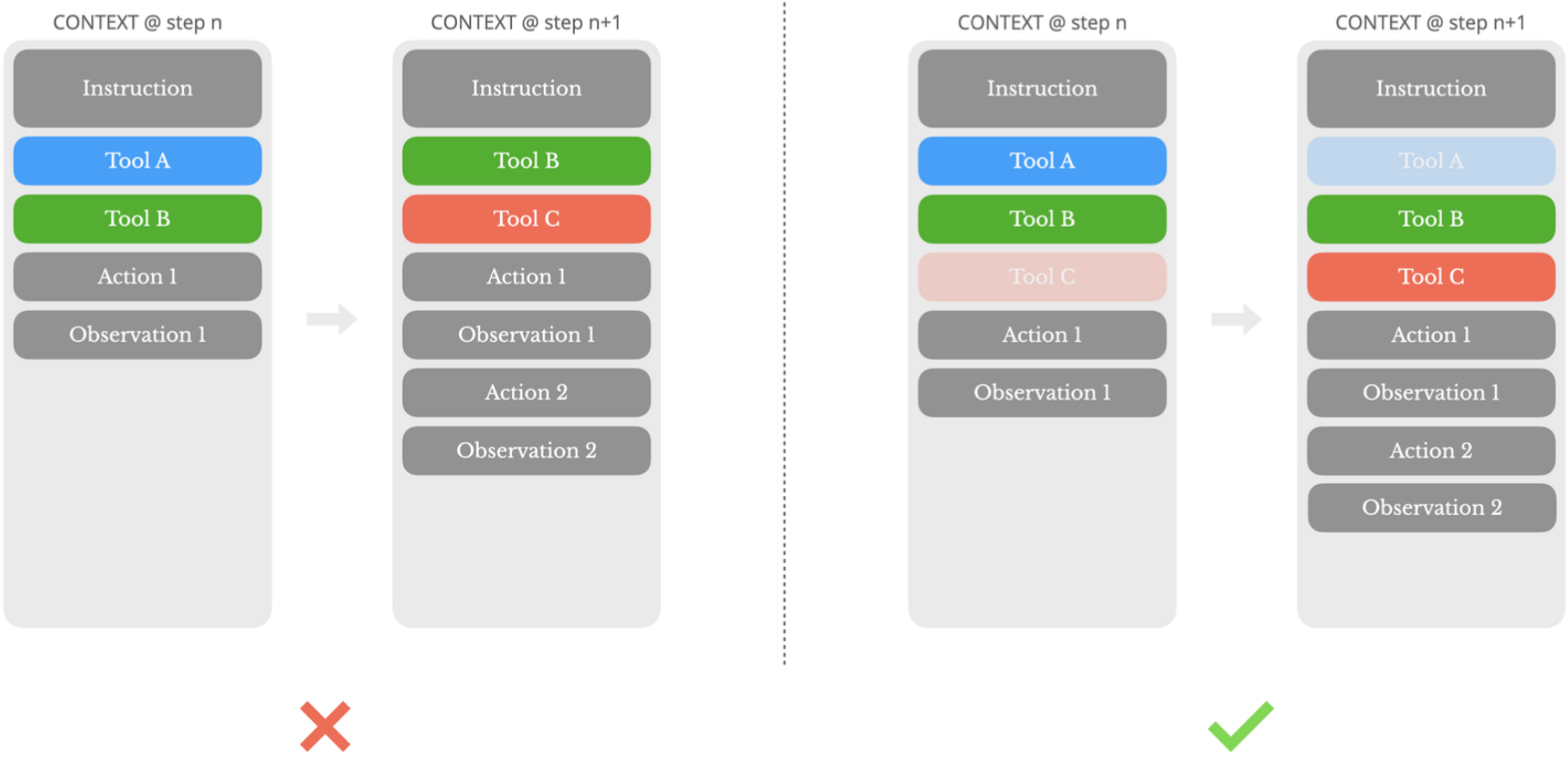
Multi-Query Attention:
Heavy Capacity Loss
$(2\ d_h\ l)$

Grouped-Query Attention:
The Optimal Middle Ground
$(2\ n_g\ d_h\ l)$

KV Caches in Agentic Calls



KV Caches in Agentic Calls



Ref: Context Engineering for AI Agents: Lessons from Building Manus

DeepSeek V4 Pro	GMICloud	https://github.com/features/copilot	587,306 tok	112 tok	$ 0.787
DeepSeek V4 Pro	GMICloud	https://github.com/features/copilot	587,109 tok	118 tok	$ 0.818
DeepSeek V4 Pro	GMICloud	https://github.com/features/copilot	578,576 tok	587 tok	$ 0.80
DeepSeek V4 Pro	GMICloud	https://github.com/features/copilot	571,057 tok	562 tok	$ 0.789
DeepSeek V4 Pro	GMICloud	https://github.com/features/copilot	569,229 tok	1,929 tok	$ 0.791
DeepSeek V4 Pro	GMICloud	https://github.com/features/copilot	40,473 tok	904 tok	$ 0.0107
DeepSeek V4 Pro	GMICloud	https://github.com/features/copilot	39,357 tok	150 tok	$ 0.0482
DeepSeek V4 Pro	GMICloud	https://github.com/features/copilot	37,765 tok	484 tok	$ 0.0469
DeepSeek V4 Pro	GMICloud	https://github.com/features/copilot	32,656 tok	326 tok	$ 0.0392



GPU size requirements

- Model weights
  - 4 bytes * number of parameters for fp32 training
  - 6 bytes * number of params for mixed precision training.
- Optimizer States
  - 8 bytes * number of parameters for normal Adam (maintains 2 states)
  - 2 bytes * number of parameters for 8-bit Adam optimizers
  - 4 bytes * number of parameters for optimizers like SGD with momentum (maintains 1 state)
- Gradients:
  - 4 bytes * number of parameters for either fp32 or mixed precision training
- Other: KV caches, Temporary memory, functionality specific memory, forward activations, etc.



Ref: https://datascience.stackexchange.com/questions/117444/what-size-language-model-can-you-train-on-a-gpu-with-x-gb-of-memory

Memory Challenge

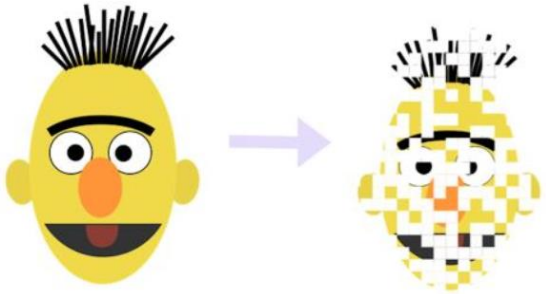
- If you want to estimate the memory required to **serve (Not Train)** a LLaMA model with 70 billion parameters, loaded in 16-bit precision:

$$M = \left(\frac{70 \times 4 \text{ bytes}}{\left(\frac{32}{16}\right)} \right) \times 1.2$$

- This calculation tells you that you would need approximately 168 GB of GPU memory to serve the LLaMA model with 70 billion parameters in 16-bit mode.



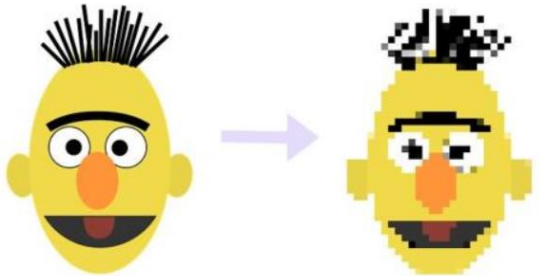
Efficiency beyond Transformer



Pruning
Removing weight connections



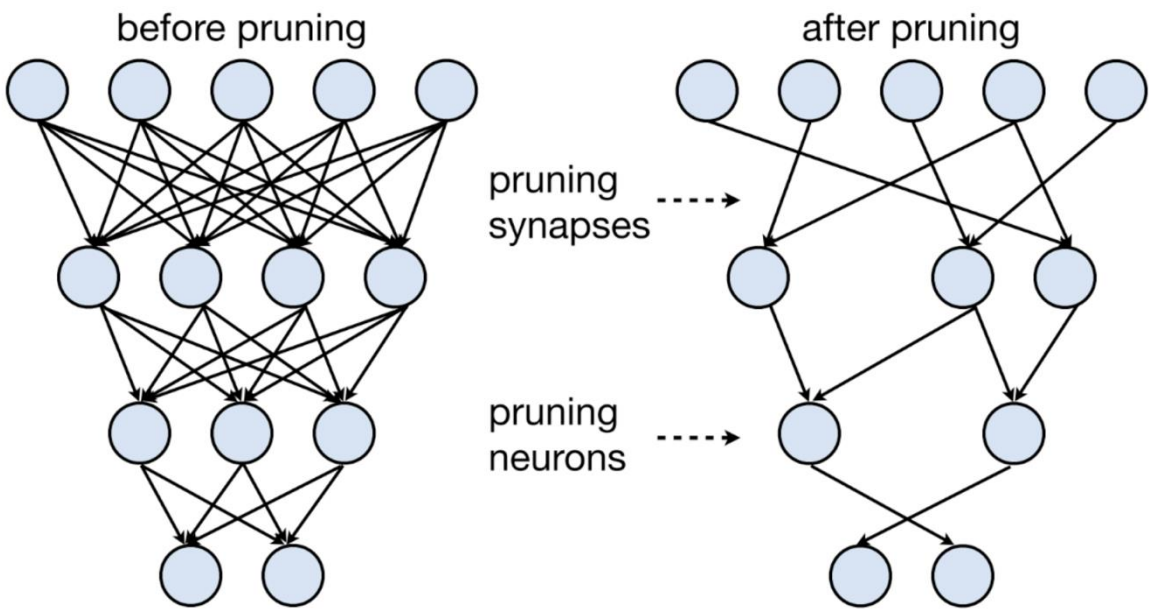
Knowledge distillation
Learning smaller models from big ones

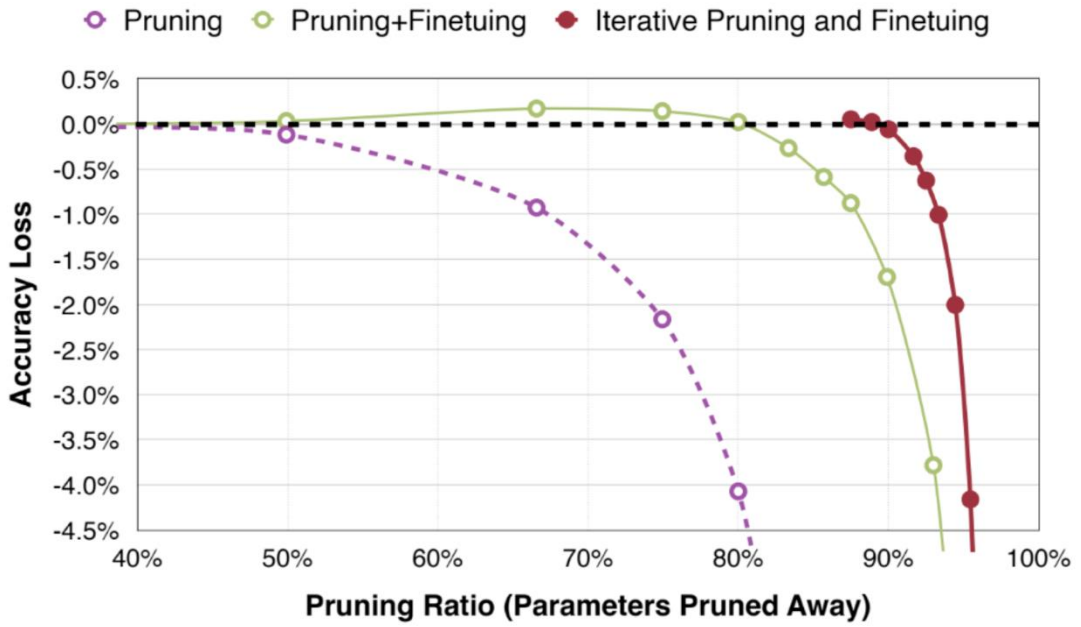


Quantization
“Low resolution”

Neural Network Pruning - What is Pruning?

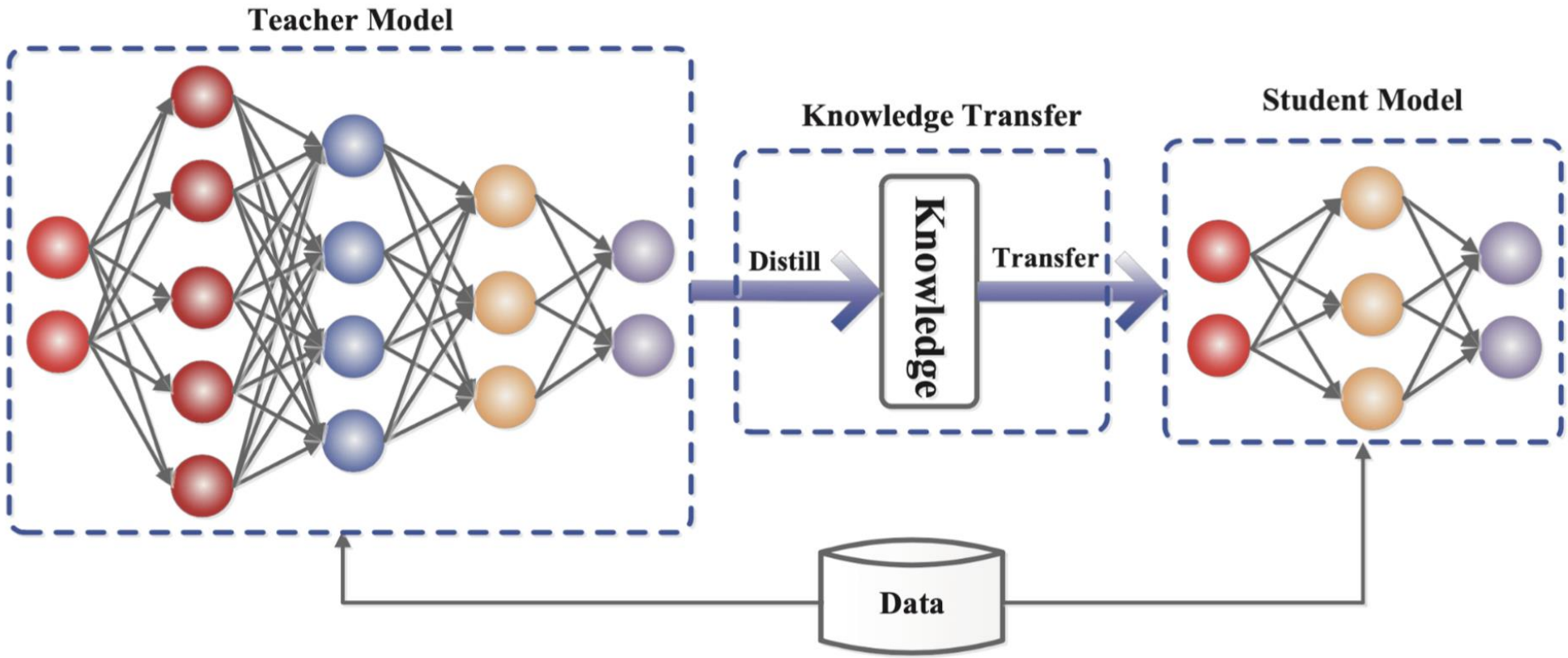
- Make neural network smaller by removing synapses and neurons





Model Distillation

- Using a smaller model to mimic a bigger one





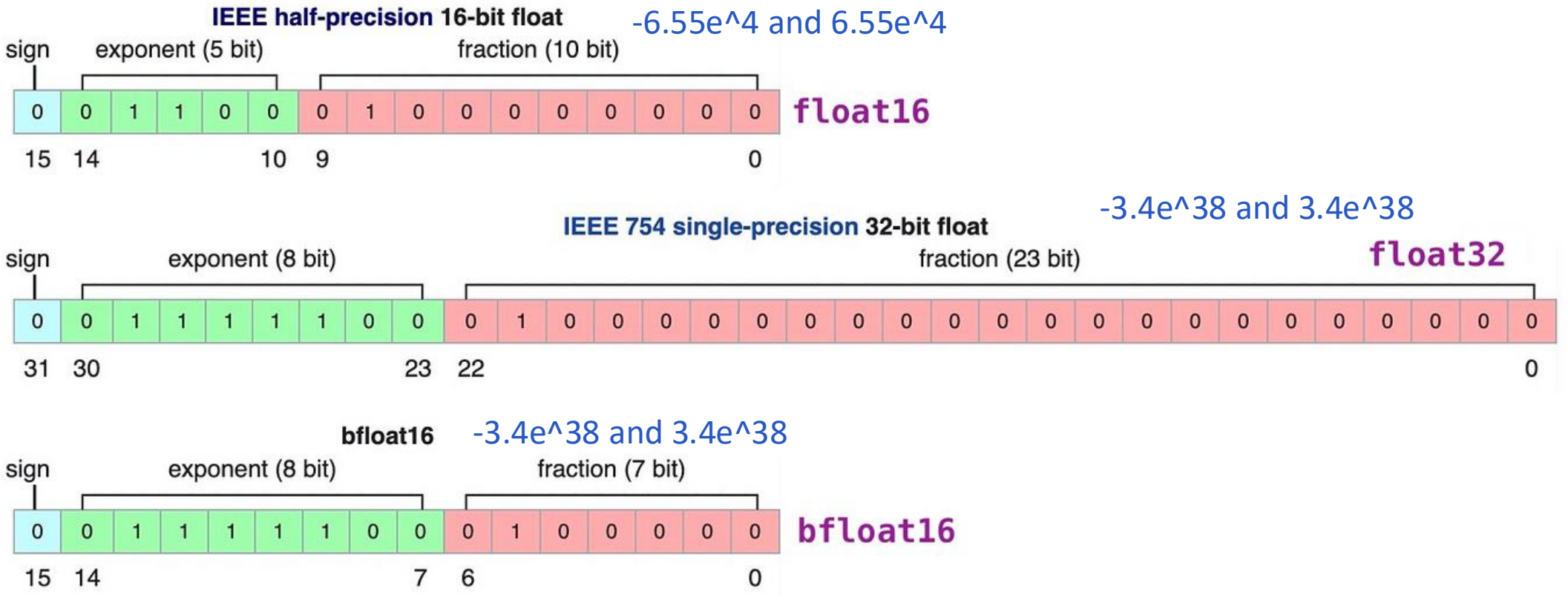
Quantization: using less memory

- A process which reduces the precision of the model's parameters, activations, or gradients to lower memory usage and computational requirements.

- WHY?
  - for deploying large models on devices with limited resources
  - for reducing the carbon footprint of training and inference processes.



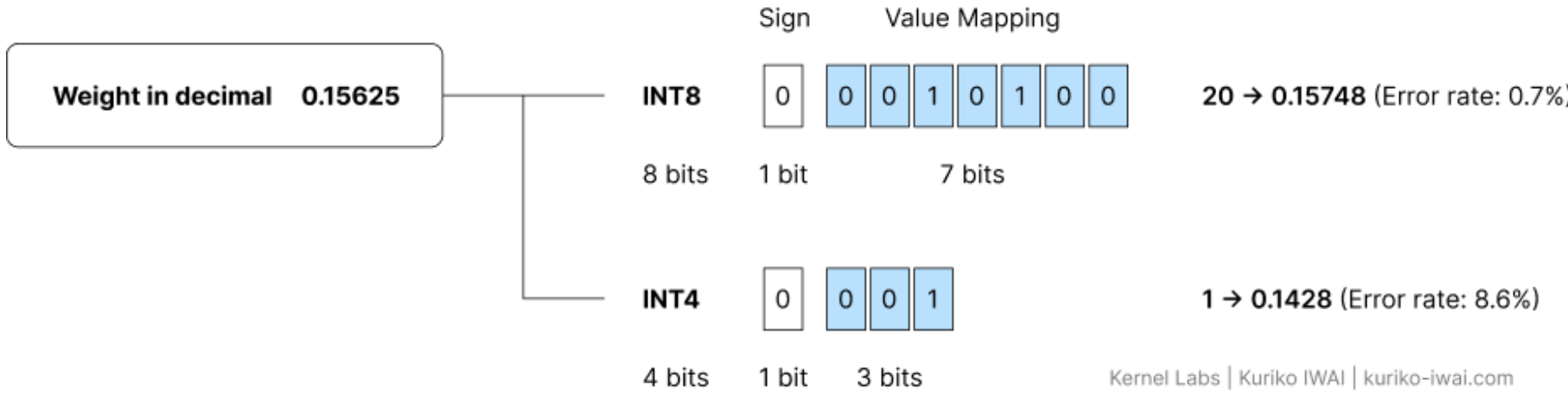
Quantization





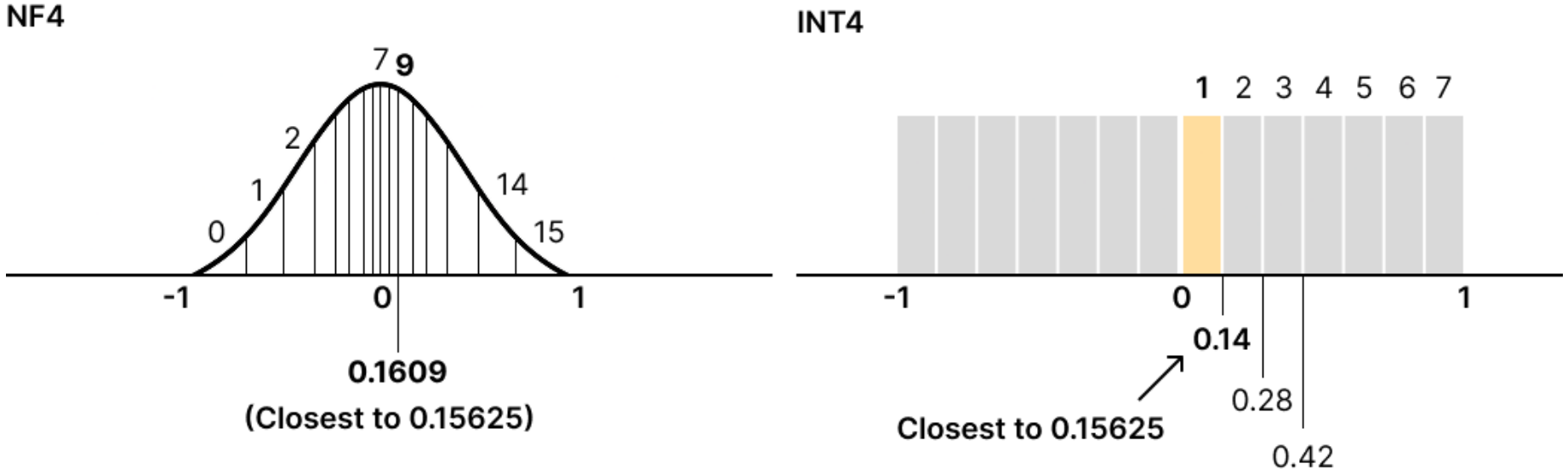
INT8 and INT4

- maps a range from -1.0 to 1.0 onto a fixed grid of whole numbers, ranging up to a maximum integer that the n-bit can represent.
- INT8: Scales the original value 0.15625 to 127 → 19.8437 (127 x 0.15625), the nearest whole number to 19.8437 → 20 → 0.15748
- INT4: Scales the original value 0.15625 to 7 → 1.09375 (7 x 0.15625), the nearest whole number to 1.09375 → 1 → 0.1428.

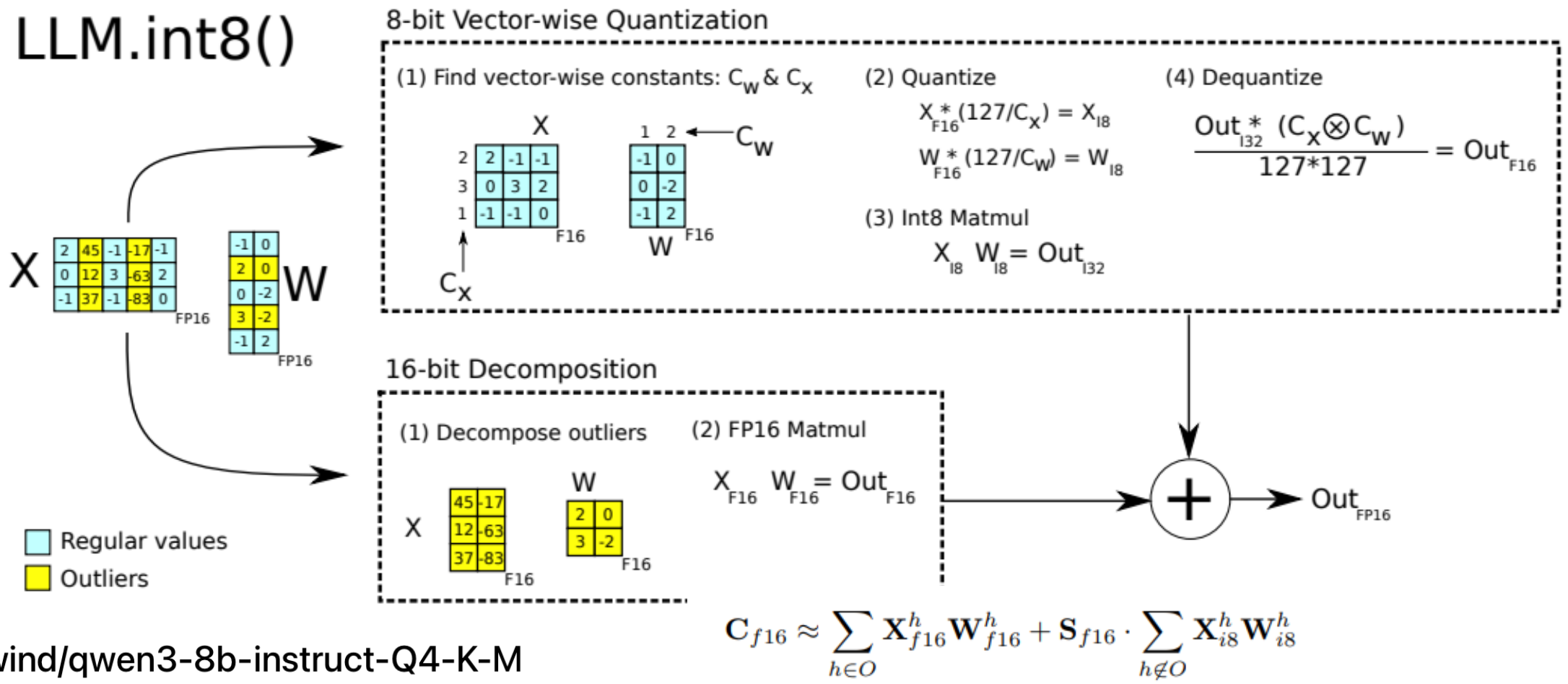


NormalFloat (NF)

- NormalFloat quantization leverages model weights' normal distribution where billions of AI weights are mostly tiny numbers clustered around zero.
- Its primary format, NormalFloat4 (NF4) places most of its 16 notches right in that crowded center around zero to represent the decimal more accurately than INT4:



8-bit Quantization: Mixed Precision Quantization



$$\mathbf{C}_{f16} \approx \sum_{h \in O} \mathbf{X}^h_{f16} \mathbf{W}^h_{f16} + \mathbf{S}_{f16} \cdot \sum_{h \notin O} \mathbf{X}^h_{i8} \mathbf{W}^h_{i8}$$

qcwind/qwen3-8b-instruct-Q4-K-M

qwen3:8b-instruct-q4_K_M

tools thinking

899.3K Pulls 1 Tag Updated 4 months ago

Quantization-aware Training

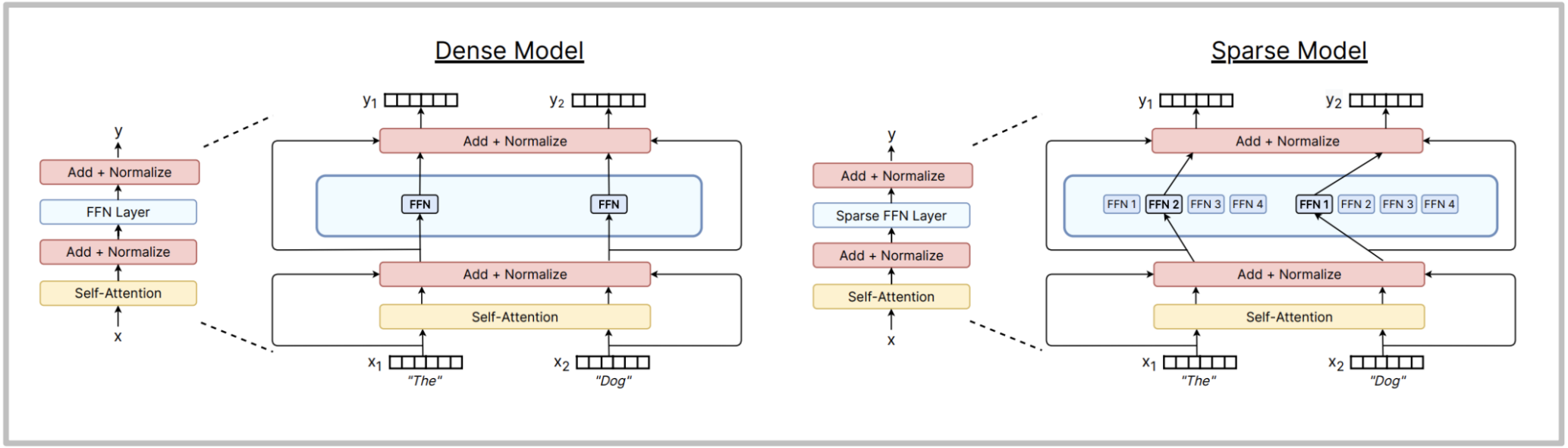
- Trains directly in low precision for forward and backward passes
- Scales tensors per layer to fit FP4 range, retains critical sums in higher precision
- Uses automated FP4 recipes in training stack for efficiency
- Benefits
  - Significant computational and memory savings
  - Improved performance for final quantised model
- Challenges
  - Unstable without optimised hardware
  - Risk of errors in multiplication, addition, and gradient calculations



Mixture of experts vs. Ensemble methods

- Replace big feedforward with (many) big feedforward networks and a selector layer
- You can increase the # experts without affecting FLOPs (Floating-Point Operation)
- Active parameters

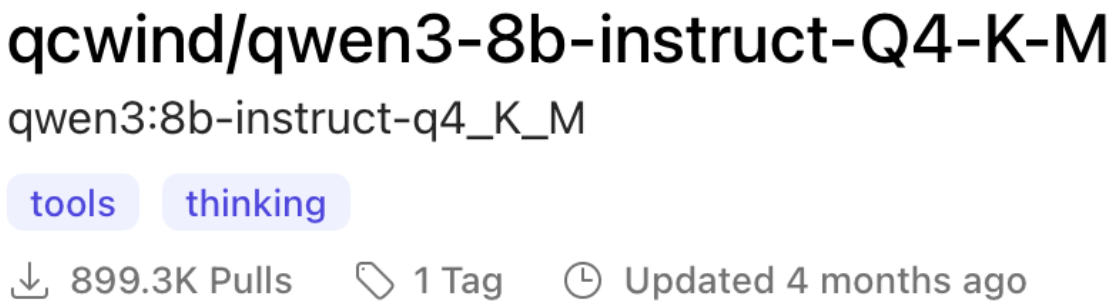
Gemma 4 31B	Gemma 4 26B A4B	Gemma 4 E4B	Gemma 4 E2B	Gemma 3 27B (no think)



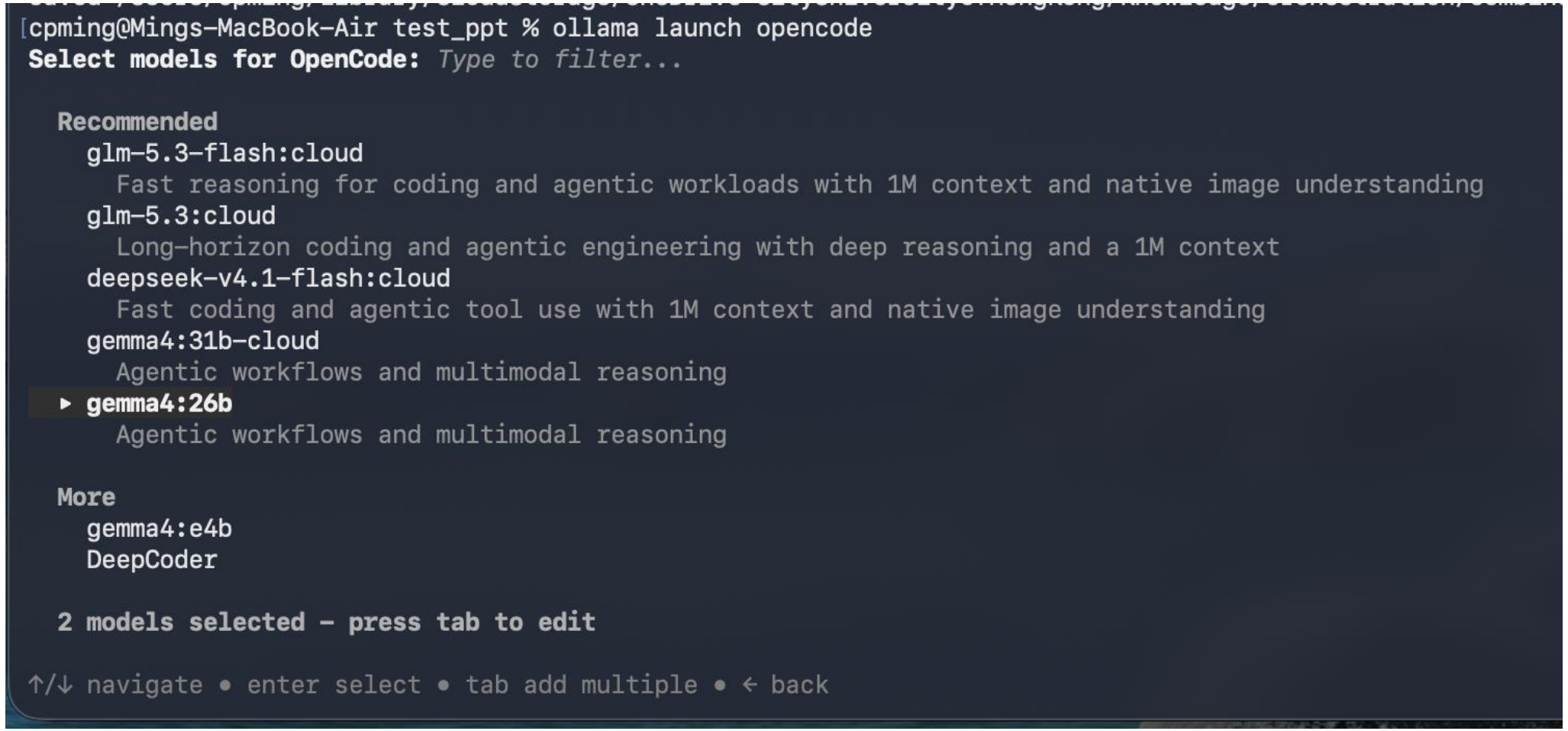
[Fedus et al 2022]

Model Names

- **Q4:** It is a **4-bit** quantization. Instead of using the standard 16-bit precision, it squishes the data down to 4 bits.
- **K:** Indicates that it uses "K-quant" methods, compressing different parts of the model differently to retain accuracy.
- **M:** Stands for **Medium**. It's a specific preset balance. A Q4_K_M model keeps the most important parts of the brain slightly larger and compresses the rest



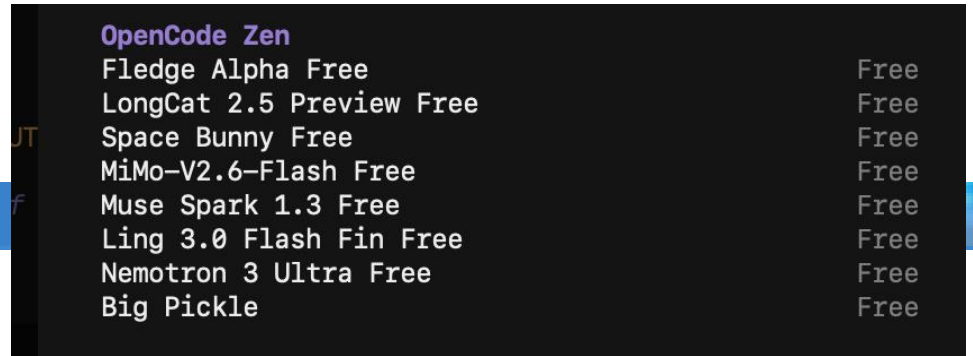
Demo: local models for coding





Assignment 2: Sequent Recognition

- Basic Tasks (50%): Develop a prompt to predict the value of a sequent using LSTM
  - Download the given data on blackboard, and use OpenCode to build models
  - Handle different cases, such as splitting train-test sets (by time) and normalization
  - Plot and check your result (e.g., in MSE), and put them into a report
- Advanced Tasks (50%):
  - (15%) Compare the results using a local LLM
  - (15%) Compare the result with Transformer
  - (20%) Build a prompt to handle the same dataset from different period (not given), on OpenCode with the free model you pick in the report
- Expected outcomes and notes:
  - Submit the prompt, a report in pdf to explain your results, the design of the prompt and the session transcripts to blackboard by ~~7 Oct~~ => 12 Oct
  - List 3 code suggestions OpenCode generated that:
    - were incorrect, sub-optimal, or inefficient for this specific task, and
    - explain how you fixed them using the knowledge in this lecture that support the fixes in your report
  - Run multiple times to capture different behaviors



Questions?

Email: pakmingcheung@cuhk.edu.hk

